



Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών &
Μηχανικών Υπολογιστών
Τομέας Τεχνολογίας Πληροφορικής και
Υπολογιστών

**Συντονισμένη εκτέλεση πολλαπλών μοντέλων
τεχνητής νοημοσύνης με early-exit
σε edge περιβάλλοντα με περιορισμένους
υπολογιστικούς πόρους**

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Ιωάννης Πολυχρονόπουλος

Επιβλέπων: Κωνσταντίνος Τσερπές
Επίκουρος Καθηγητής Ε.Μ.Π.

Αθήνα, Ιούνιος 2026



Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών &
Μηχανικών Υπολογιστών
Τομέας Τεχνολογίας Πληροφορικής και
Υπολογιστών

Συντονισμένη εκτέλεση πολλαπλών μοντέλων τεχνητής νοημοσύνης με early-exit σε edge περιβάλλοντα με περιορισμένους υπολογιστικούς πόρους

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Ιωάννης Πολυχρονόπουλος

Επιβλέπων: Κωνσταντίνος Τσερπές
Επίκουρος Καθηγητής Ε.Μ.Π.

Εγκρίθηκε από την τριμελή εξεταστική επιτροπή την 30η Ιουνίου 2026.

.....
Κωνσταντίνος Τσερπές
Επίκουρος Καθηγητής Ε.Μ.Π.

.....
Αθανάσιος Βουλόδημος
Επίκουρος Καθηγητής Ε.Μ.Π.

.....
Θεοδώρα Βαρβαρίγου
Καθηγήτρια Ε.Μ.Π.

Αθήνα, Ιούνιος 2026

.....

Ιωάννης Πολυχρονόπουλος

Διπλωματούχος Ηλεκτρολόγος Μηχανικός και Μηχανικός Υπολογιστών Ε.Μ.Π.

Copyright ©Ιωάννης Πολυχρονόπουλος, 2026.

Με επιφύλαξη παντός δικαιώματος. All rights reserved.

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Ερωτήματα που αφορούν τη χρήση της εργασίας για κερδοσκοπικό σκοπό πρέπει να απευθύνονται προς τον συγγραφέα.

Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν τον συγγραφέα και δεν πρέπει να ερμηνευθεί ότι αντιπροσωπεύουν τις επίσημες θέσεις του Εθνικού Μετσόβιου Πολυτεχνείου.

Περίληψη

Η εκτέλεση μοντέλων τεχνητής νοημοσύνης σε edge περιβάλλοντα παρουσιάζει σημαντικές τεχνικές προκλήσεις, καθώς οι διαθέσιμες συσκευές χαρακτηρίζονται συχνά από περιορισμένη υπολογιστική ισχύ, μνήμη και ενεργειακή αυτονομία. Παράλληλα, η αυξανόμενη πολυπλοκότητα των βαθιών νευρωνικών δικτύων καθιστά αναγκαίες τεχνικές που μειώνουν το κόστος inference χωρίς σημαντική υποβάθμιση της ακρίβειας. Τα early-exit νευρωνικά δίκτυα εισάγουν έναν μηχανισμό προσαρμοστικής εκτέλεσης, ώστε τα εύκολα δείγματα να τερματίζουν σε ενδιάμεσες εξόδους αντί να διατρέχουν ολόκληρο το μοντέλο. Σε καταναμημένα edge συστήματα, όμως, και ειδικά σε σενάρια με πολλαπλές ταυτόχρονες ροές inference, η απόδοση εξαρτάται όχι μόνο από την αρχιτεκτονική του μοντέλου αλλά και από τη διάσπαση του δικτύου και την τοποθέτηση των partitions στους διαθέσιμους workers.

Η παρούσα διπλωματική εργασία μελετά τη συντονισμένη εκτέλεση early-exit ResNet μοντέλων σε περιορισμένο Raspberry Pi edge cluster. Χρησιμοποιούνται ResNet-18 και ResNet-34 μοντέλα, σε baseline και early-exit μορφή, και αξιολογούνται σε CIFAR-10 και CIFAR-10.1 χωρίς αλλαγή των εκπαιδευμένων βαρών ή της πολιτικής εξόδου. Η πειραματική μελέτη οργανώνεται κλιμακωτά: αρχικά εξετάζεται η single-node εκτέλεση, στη συνέχεια η καταναμημένη εκτέλεση ενός μοντέλου σε τρία partitions, και τέλος η πολυ-μοντελική εκτέλεση δύο logical model instances πάνω στους ίδιους workers. Για την πολυ-μοντελική περίπτωση συγκρίνεται μια αφελής κοινή διαδρομή με τρεις memory-aware στρατηγικές placement.

Τα αποτελέσματα δείχνουν ότι το early-exit μειώνει σημαντικά το μέσο latency και την ενέργεια ανά εργασία inference σε single-node εκτέλεση, με αναμενόμενο συμβιβασμό στην ακρίβεια. Αντίθετα, η καταναμημένη εκτέλεση δεν οδηγεί αυτόματα σε καλύτερη αξιοποίηση του cluster, επειδή τα πρώτα partitions δέχονται σχεδόν όλα τα δείγματα ενώ τα βαθύτερα partitions ενεργοποιούνται μόνο για δυσκολότερες εισόδους. Στα multi-model σενάρια, η αναπαραγωγή της ίδιας pipeline διαδρομής δημιουργεί bottleneck στον πρώτο worker και περιορίζει την αύξηση του throughput. Οι memory-aware τοποθετήσεις μειώνουν τη συμφόρηση, το communication/runtime overhead και την ενέργεια ανά εργασία, βελτιώνοντας συνολικά τη συμπεριφορά του συστήματος. Η σύγκριση με το CIFAR-10.1 δείχνει επιπλέον ότι το βέλτιστο placement εξαρτάται από την κατανομή των exits και, συνεπώς, από τη ροή των δεδομένων εισόδου.

Λέξεις Κλειδιά: early-exit νευρωνικά δίκτυα, edge computing, καταναμημένο inference, split computing, partitioning, placement, multi-model εκτέλεση, Raspberry Pi cluster, ResNet, CIFAR-10, CIFAR-10.1, latency, throughput, communication/runtime overhead, ενεργειακή αποδοτικότητα

Abstract

Executing artificial intelligence models in edge environments is a critical challenge, as available devices are often characterized by limited computational power, memory, and energy autonomy. At the same time, the increasing complexity of deep neural networks makes it necessary to employ techniques that reduce inference cost without significantly degrading accuracy. Early-exit neural networks provide a form of adaptive computation by allowing easy samples to terminate at intermediate exits instead of traversing the entire model. However, when such models are deployed in distributed edge systems, especially in scenarios with multiple concurrent inference streams, performance depends not only on the model architecture but also on how the network is partitioned and how its partitions are placed across the available workers.

This diploma thesis studies the coordinated execution of early-exit ResNet models on a resource-constrained Raspberry Pi edge cluster. ResNet-18 and ResNet-34 models are used in both baseline and early-exit form and are evaluated on CIFAR-10 and CIFAR-10.1 without changing the trained weights or the exit policy. The experimental study is organized incrementally: first, single-node execution is evaluated; then, a single early-exit model is executed as a three-partition distributed pipeline; finally, two logical model instances are executed concurrently on the same workers. For the multi-model case, a naive common pipeline is compared against three memory-aware placement strategies.

The results show that early exits significantly reduce average latency and energy per inference task in single-node execution, at the expected cost of lower accuracy. In contrast, simply distributing one model across multiple workers does not automatically improve cluster utilization, because the first partitions process almost all samples while deeper partitions are activated only for harder inputs. In multi-model scenarios, replicating the same pipeline route creates a bottleneck on the first worker and limits throughput scaling. The memory-aware placements reduce congestion, communication/runtime overhead, and energy per task, improving the overall behavior of the system. The comparison with CIFAR-10.1 further shows that the best placement depends on the exit distribution and, therefore, on the input data stream.

Keywords: early-exit neural networks, edge computing, distributed inference, split computing, partitioning, placement, multi-model execution, Raspberry Pi cluster, ResNet, CIFAR-10, CIFAR-10.1, latency, throughput, communication/runtime overhead, energy efficiency

Περιεχόμενα

Περίληψη	5
Abstract	7
Κατάλογος Εικόνων	13
Κατάλογος Πινάκων	15
1 Εισαγωγή	17
1.1 Κίνητρο και Σημασία του Προβλήματος	17
1.1.1 Η άνοδος των βαθιών νευρωνικών δικτύων	17
1.1.2 Προκλήσεις εκτέλεσης σε edge περιβάλλοντα	19
1.1.3 Υφιστάμενες τεχνικές βελτιστοποίησης	19
1.1.4 Ανάγκη για ολοκληρωμένες λύσεις	20
1.2 Στόχοι της Διπλωματικής	20
1.3 Δομή της Εργασίας	21
2 Θεωρητικό Υπόβαθρο	25
2.1 Βαθιά Νευρωνικά Δίκτυα	25
2.1.1 Συνελικτικά Δίκτυα	26
2.1.2 Inference σε DNNs	27
2.1.3 Συμβιβασμός Ακρίβειας και Πόρων	28
2.2 Early-Exit Νευρωνικά Δίκτυα	29
2.2.1 Αρχιτεκτονική early-exit	29
2.2.2 Εκπαίδευση Multi-Exit Μοντέλων	30
2.2.3 Κριτήρια Τερματισμού	31
2.2.4 Οφέλη και Περιορισμοί	32
2.3 Edge Computing και Κατανεμημένα Edge Συστήματα	33
2.3.1 Πόροι και Περιορισμοί στο Edge	33
2.3.2 Τοπική, Κατανεμημένη και Πολυμοντελική Εκτέλεση	34
2.4 Partitioning και Split Computing σε Edge Workers	35
2.4.1 Βασικό Μοντέλο Split Computing	35
2.4.2 Confidence-Based Routing	36
2.4.3 Συνδυασμός Split Computing και Early-Exit Εκτέλεσης	36
2.4.4 Resource-Aware Τοποθέτηση	37
2.4.5 Προοδευτική Εκτέλεση	37
3 Σχετική Βιβλιογραφία	39
3.1 Early-Exit Αρχιτεκτονικές και Adaptive Inference	39
3.1.1 Multi-exit DNNs και δυναμικό βάθος εκτέλεσης	40

3.1.2	Από το BranchyNet στην ταξινόμηση των early-exit DNNs	41
3.1.3	Στρατηγικές εκπαίδευσης και αξιοπιστία ενδιάμεσων εξόδων . . .	41
3.1.4	Πολιτικές εξόδου και adaptive inference	42
3.2	Split Computing και Κατανεμημένη Edge Εκτέλεση DNNs	43
3.2.1	Split Inference και Κόστος Ενδιάμεσων Αναπαραστάσεων	43
3.2.2	Συμπίεσμένες Αναπαραστάσεις και Δυναμική Επιλογή Split Point	44
3.2.3	Split Computing και Early Exits σε Edge Pipelines	45
3.3	Resource-Aware Placement για Dynamic DNNs	45
3.3.1	Το Deployment ως Constrained Optimization Πρόβλημα	45
3.3.2	FIN και Σύνδεση με την Παρούσα Εργασία	46
3.4	Σύνθεση Βιβλιογραφίας και Θέση της Παρούσας Εργασίας	47
4	Μεθοδολογία και Πειραματικό Πλαίσιο	49
4.1	Σκοπός και Μεθοδολογική Προσέγγιση	49
4.2	Λογισμικό και Βιβλιοθήκες Υλοποίησης	50
4.3	Πειραματική Υποδομή	50
4.4	Σύνολα Δεδομένων και Προεπεξεργασία	51
4.4.1	Προεπεξεργασία Εισόδου	52
4.4.2	Warmup και Πειραματική Συνέπεια	52
4.5	Αρχιτεκτονικές ResNet και Early-Exit Επέκταση	52
4.5.1	Residual Blocks	53
4.5.2	Baseline Μοντέλα	54
4.5.3	Early-Exit Μοντέλα	54
4.6	Εκπαίδευση, Αποθηκευμένα Βάρη και Πολιτική early-exit	56
4.6.1	Εκπαίδευση Baseline Μοντέλων	56
4.6.2	Εκπαίδευση Multi-Exit Μοντέλων	56
4.6.3	Entropy-Based Απόφαση Εξόδου	57
4.7	Partitioning και Κατανεμημένη Εκτέλεση	58
4.7.1	Three-Partition Model Split	58
4.7.2	Πολυ-μοντελική Τοποθέτηση	58
4.8	Πειραματικές Τοπολογίες και Σενάρια	59
4.8.1	Experiment 1: Single-Model Εκτέλεση	59
4.8.2	Experiment 2: Multi-Model Εκτέλεση με Κοινή Διαδρομή	60
4.8.3	Experiment 3: Memory-Aware Placement	60
4.8.4	Κατάλογος Runs	62
4.9	Distributed Runtime και Επικοινωνία	62
4.9.1	Ροή Inference	62
4.9.2	Πρωτόκολλο Επικοινωνίας	63
4.9.3	Έλεγχος Συμβατότητας Workers	63
4.10	Μετρικές Αξιολόγησης	63
4.10.1	Ακρίβεια και Exit Distribution	65
4.10.2	Latency, Throughput και Utilization	66
4.10.3	Remote Compute και Communication/Runtime Overhead	67
4.10.4	Ενέργεια και Εκπομπές	68
4.11	Περιορισμοί	69
5	Πειραματικά Αποτελέσματα	71
5.1	Οργάνωση της Πειραματικής Αξιολόγησης	71

5.2	Experiment 1: Single-Model Εκτέλεση	71
5.3	Experiment 2: Αφελής Multi-Model Κατανομή	73
5.4	Experiment 3: Memory-Aware Placement	74
5.5	Ενεργειακή Αποδοτικότητα	77
5.6	Συμπληρωματική Αξιολόγηση στο CIFAR-10.1	80
6	Συζήτηση	87
6.1	Ερωτήματα Συζήτησης και Άξονες Ερμηνείας	87
6.2	Το Early-Exit ως Μηχανισμός Προσαρμοστικού Υπολογισμού	88
6.3	Κατανεμημένη Εκτέλεση και Ανισορροπία Φόρτου	88
6.4	Placement ως Πολυκριτήριο Πρόβλημα Βελτιστοποίησης	89
6.5	Σχεδιαστικές Αρχές	91
7	Συμπεράσματα και Μελλοντική Εργασία	93
7.1	Μελλοντική Εργασία	93
7.2	Συνολικά Συμπεράσματα	95
	Βιβλιογραφία	97
	Δήλωση χρήσης εργαλείων Τεχνητής Νοημοσύνης	99

Κατάλογος Εικόνων

1.1	Εξέλιξη CNN αρχιτεκτονικών και σχέση μεγέθους–ακρίβειας.	18
2.1	Αρχιτεκτονική βαθιού νευρωνικού δικτύου.	26
2.2	Ροή επεξεργασίας σε συνελκτικό νευρωνικό δίκτυο.	27
2.3	Παράδειγμα CNN για ανάλυση εικόνας.	28
2.4	Συμβατική και early-exit εκτέλεση DNN.	29
2.5	Joint training σε early-exit DNN.	31
2.6	Παράδειγμα split computing.	35
3.1	Αρχιτεκτονική BranchyNet με πλευρικούς κλάδους.	40
3.2	Early-exit DNN με ενδιάμεσους κλάδους.	42
3.3	Προσεγγίσεις split computing.	44
3.4	Resource-aware deployment δυναμικών DNNs.	46
4.1	Residual learning block.	53
4.2	Ροή inference baseline ResNet.	54
4.3	Δομή early-exit κεφαλής.	54
4.4	Αρχιτεκτονική early-exit ResNet.	55
4.5	Ροή ενεργοποιήσεων ResNet-18 EE.	55
4.6	Ροή ενεργοποιήσεων ResNet-34 EE.	56
4.7	Κοινή διαδρομή των model instances στο Experiment 2.	60
4.8	Στρατηγικές τοποθέτησης του Experiment 3.	61
5.1	Αξιοποίηση workers στο Experiment 1.3.	73
5.2	Throughput speedup στρατηγικών Experiment 3.	75
5.3	Αξιοποίηση workers: Experiment 2 vs Exp3.2.	75
5.4	Αξιοποίηση workers σε distributed πειράματα.	76
5.5	Communication/runtime overhead στα Experiments 2 και 3.	76
5.6	Ενέργεια ανά inference εργασία.	78
5.7	Ανάλυση ενέργειας σε master και workers.	78
5.8	Energy-Delay Product ανά inference εργασία.	79
5.9	Αξιοποίηση workers στο Experiment 1.3 (CIFAR-10.1).	81
5.10	Throughput speedup Experiment 3 (CIFAR-10.1).	82
5.11	Αξιοποίηση workers: Experiment 2 vs Exp3.3 (CIFAR-10.1).	83
5.12	Αξιοποίηση workers σε distributed πειράματα (CIFAR-10.1).	84
5.13	Communication/runtime overhead (CIFAR-10.1).	84
5.14	Ενέργεια ανά inference εργασία (CIFAR-10.1).	85
5.15	Ανάλυση ενέργειας σε master και workers (CIFAR-10.1).	85
5.16	Energy-Delay Product (CIFAR-10.1).	86

Κατάλογος Πινάκων

4.2	Βασικές βιβλιοθήκες και ρόλος τους στην πειραματική υλοποίηση.	50
4.4	Υλικό και βασικά χαρακτηριστικά των κόμβων της πειραματικής υποδομής.	51
4.5	Διάσπαση του early-exit μοντέλου σε partitions και αντίστοιχες έξοδοι.	58
4.7	Σύνοψη των αποθηκευμένων CIFAR-10 και CIFAR-10.1 πειραματικών runs.	62
4.9	Συνοπτική αντιστοίχιση μετρικών, υπολογισμού και ρόλου.	64
5.1	Experiment 1 core metrics.	72
5.2	Σύγκριση του Experiment 1.3 με το αφελές multi-model Experiment 2.	74
5.3	Experiment 3 latency και throughput σε σχέση με το Experiment 2.	74
5.4	Experiment 1 core metrics (CIFAR-10.1).	80
5.5	Σύγκριση του Experiment 1.3 με το αφελές multi-model Experiment 2 στο CIFAR-10.1.	81
5.6	Experiment 3 latency και throughput σε σχέση με το Experiment 2 στο CIFAR-10.1.	82

Κεφάλαιο 1

Εισαγωγή

1.1 Κίνητρο και Σημασία του Προβλήματος

Τα τελευταία χρόνια, η Τεχνητή Νοημοσύνη (Artificial Intelligence – AI) και η Μηχανική Μάθηση (Machine Learning – ML) έχουν γνωρίσει ραγδαία ανάπτυξη, τόσο σε ερευνητικό όσο και σε βιομηχανικό επίπεδο. Η εξέλιξη αυτή έχει οδηγήσει στην ενσωμάτωση των αλγορίθμων μάθησης σε πλήθος εφαρμογών της καθημερινότητας, όπως η πρόβλεψη ζήτησης σε υπηρεσίες και εφοδιαστικές αλυσίδες, η ανίχνευση απάτης σε χρηματοοικονομικές συναλλαγές, τα συστήματα συστάσεων και εξατομίκευσης περιεχομένου σε ψηφιακές πλατφόρμες και η υποστήριξη λήψης αποφάσεων σε συστήματα υγείας και ενέργειας.

Κομβικό ρόλο σε αυτήν την πρόοδο έχουν διαδραματίσει τα νευρωνικά δίκτυα, μια από τις πιο βασικές κατηγορίες μοντέλων μηχανικής μάθησης. Η ιδέα τους εμφανίστηκε ήδη στις δεκαετίες του 1940 και 1950, με τα πρώτα υπολογιστικά μοντέλα τεχνητών νευρώνων και το perceptron. Για μεγάλο χρονικό διάστημα, όμως, η πρακτική τους αξιοποίηση παρέμεινε περιορισμένη, κυρίως λόγω της ανεπαρκούς υπολογιστικής ισχύος και της έλλειψης μεγάλων συνόλων δεδομένων. Με την πάροδο του χρόνου, η εμφάνιση ισχυρών GPUs, η αύξηση των διαθέσιμων δεδομένων και η ανάπτυξη πιο αποδοτικών αλγορίθμων εκπαίδευσης οδήγησαν στη ραγδαία εξέλιξη των βαθιών νευρωνικών δικτύων.

1.1.1 Η άνοδος των βαθιών νευρωνικών δικτύων

Τα βαθιά νευρωνικά δίκτυα (Deep Neural Networks – DNNs) αποτελούν σήμερα βασική τεχνολογία για μεγάλο φάσμα εφαρμογών τεχνητής νοημοσύνης, όπως η υπολογιστική όραση (computer vision), η επεξεργασία φυσικής γλώσσας (Natural Language Processing – NLP), η επεξεργασία σήματος και η αναγνώριση ομιλίας. Η επιτυχία τους συνδέεται με την ικανότητά τους να μαθαίνουν ιεραρχικές αναπαραστάσεις απευθείας από τα δεδομένα, περιορίζοντας την ανάγκη για χειροκίνητο σχεδιασμό χαρακτηριστικών.

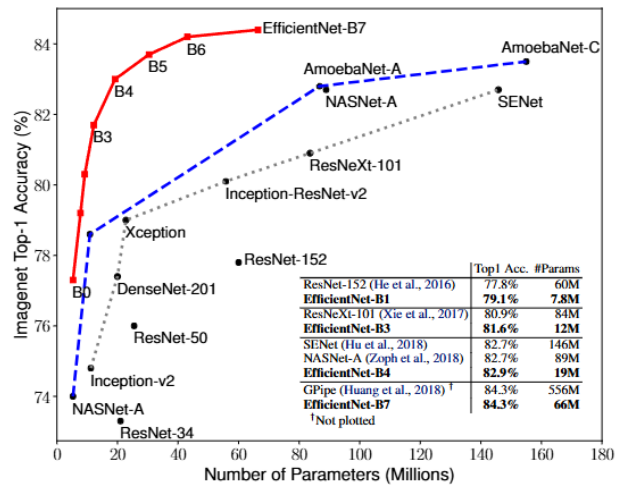
Μία από τις πιο χαρακτηριστικές εκφράσεις αυτής της προόδου είναι τα συνελκτικά νευρωνικά δίκτυα (Convolutional Neural Networks – CNNs). Τα CNNs αξιοποιούν τη χωρική δομή της εισόδου, όπως συμβαίνει στις εικόνες, και αποτέλεσαν τον βασικό

μοχλό της προόδου στην υπολογιστική όραση. Για τον λόγο αυτό προσφέρουν ένα αντιπροσωπευτικό παράδειγμα του γενικότερου φαινομένου που απασχολεί την παρούσα εργασία: όσο τα μοντέλα γίνονται βαθύτερα και πιο σύνθετα, αυξάνεται η εκφραστική τους ικανότητα, αλλά ταυτόχρονα αυξάνονται και οι απαιτήσεις τους σε υπολογισμό, μνήμη και ενέργεια.

Η ιστορική πορεία των CNNs αποτυπώνει αυτή τη μεταβολή: από τα πρώιμα LeNet [1] και το AlexNet [2], η έρευνα οδηγήθηκε σε βαθύτερες αρχιτεκτονικές όπως τα VGG και ResNet [3], [4], καθώς και σε πιο συστηματικές μεθόδους κλιμάκωσης όπως το EfficientNet [5]. Η Εικόνα 1.1 συνοψίζει ενδεικτικά αυτήν την πορεία και τη συνδέει με τη σχέση μεγέθους μοντέλου και ακρίβειας στο ImageNet.¹

Μοντέλο	Έτος	Βάθος	Παράμ.
LeNet-5	1998	5	~ 60K
AlexNet	2012	8	~ 60M
VGG-16	2015	16	~ 138M
ResNet-152	2016	152	~ 60M
AmoebaNet-C	2019	–	~ 155M
EfficientNet-B7	2019	–	~ 66M

(a) Ενδεικτική εξέλιξη CNN αρχιτεκτονικών.



(b) Μέγεθος μοντέλου και ακρίβεια στο ImageNet.

Εικόνα 1.1: Εξέλιξη σημαντικών CNN αρχιτεκτονικών και ενδεικτική σχέση μεγέθους μοντέλου–ακρίβειας στο ImageNet. Τα στοιχεία του πίνακα βασίζονται στις αντίστοιχες δημοσιεύσεις και στη σύγκριση του EfficientNet [1], [2], [3], [4], [5]. Το διάγραμμα προέρχεται από το EfficientNet paper [5].

Το βασικό συμπέρασμα είναι ότι η ακρίβεια δεν βελτιώνεται γραμμικά με το μέγεθος ή το υπολογιστικό κόστος ενός μοντέλου. Όπως φαίνεται στην Εικόνα 1.1b, μοντέλα με πολύ διαφορετικό αριθμό παραμέτρων μπορούν να επιτυγχάνουν παρόμοια ακρίβεια, ενώ μικρές επιπλέον βελτιώσεις στο ImageNet συχνά συνοδεύονται από δυσανάλογη αύξηση σε παραμέτρους, μνήμη και αριθμητικές πράξεις [5]. Επομένως, η επιλογή αρχιτεκτονικής δεν μπορεί να βασίζεται αποκλειστικά στην ακρίβεια, αλλά πρέπει να συνεκτιμά το κόστος εκτέλεσης.

Η παρατήρηση αυτή συνδέει άμεσα την εξέλιξη των DNNs με το πρόβλημα της εκτέλεσης σε edge περιβάλλοντα, όπου οι διαθέσιμοι πόροι είναι περιορισμένοι και κάθε επιπλέον πράξη ή μετάδοση δεδομένων επηρεάζει το latency, την ενεργειακή κατανάλωση και τη συνολική βιωσιμότητα της εφαρμογής.

¹Οι αριθμοί παραμέτρων είναι στρογγυλοποιημένοι, καθώς μπορούν να διαφέρουν ελαφρά ανάλογα με την υλοποίηση και τη διαμόρφωση της τελικής ταξινομητικής κεφαλής.

1.1.2 Προκλήσεις εκτέλεσης σε edge περιβάλλοντα

Η εκτέλεση μοντέλων τεχνητής νοημοσύνης κοντά στην πηγή των δεδομένων, δηλαδή σε edge περιβάλλοντα, προσφέρει σημαντικά πλεονεκτήματα. Μπορεί να μειώσει την καθυστέρηση επικοινωνίας, να περιορίσει τη μετάδοση ευαίσθητων δεδομένων εκτός του τοπικού συστήματος και να επιτρέψει την ταχύτερη λήψη αποφάσεων. Τέτοια περιβάλλοντα εκτείνονται από κινητά τηλέφωνα και αισθητήρες Internet of Things (IoT) έως edge gateways και ενσωματωμένα συστήματα χαμηλής ισχύος. Παρά την ποικιλία τους, οι συσκευές αυτές έχουν ένα κοινό χαρακτηριστικό: διαθέτουν συνήθως περιορισμένη υπολογιστική ισχύ, μνήμη και ενεργειακή αυτονομία.

Σε αυτές τις συσκευές, η τοπική εκτέλεση ενός μεγάλου νευρωνικού δικτύου μπορεί να οδηγήσει σε υψηλό latency, αυξημένη κατανάλωση μπαταρίας/ενέργειας, υπερβολική χρήση μνήμης ή ακόμη και αδυναμία εκτέλεσης του μοντέλου. Οι περιορισμοί αυτοί είναι ιδιαίτερα σημαντικοί σε εφαρμογές πραγματικού χρόνου, όπου η ταχύτητα απόκρισης αποτελεί βασική απαίτηση. Παραδείγματα τέτοιων εφαρμογών είναι η αυτόνομη οδήγηση, η βιομηχανική ρομποτική και η ζωντανή ανάλυση βίντεο σε κάμερες ασφαλείας.

Σε τέτοια σενάρια, ακόμη και μικρές καθυστερήσεις μπορούν να επηρεάσουν την ποιότητα της απόφασης ή τη συνολική απόδοση του συστήματος. Συνεπώς, η απλή μεταφορά ολοένα μεγαλύτερων μοντέλων σε edge συσκευές δεν αποτελεί από μόνη της βιώσιμη λύση.

1.1.3 Υφιστάμενες τεχνικές βελτιστοποίησης

Για την αντιμετώπιση των παραπάνω περιορισμών έχουν προταθεί διάφορες τεχνικές που επιδιώκουν να γεφυρώσουν το χάσμα μεταξύ ακρίβειας και αποδοτικότητας. Μία πρώτη κατηγορία αφορά τη συμπίεση μοντέλων. Τεχνικές όπως το pruning, το quantization (ποσοτικοποίηση) και το knowledge distillation (απόσταξη γνώσης) μειώνουν το μέγεθος και την πολυπλοκότητα ενός μοντέλου, με στόχο να διατηρηθεί όσο το δυνατόν μεγαλύτερο μέρος της αρχικής του ακρίβειας.

Μια δεύτερη κατεύθυνση είναι τα δίκτυα early-exit (Early-Exit networks). Σε αυτά, το νευρωνικό δίκτυο εμπλουτίζεται με ενδιάμεσους ταξινομητές σε διαφορετικά επίπεδα. Έτσι, για δείγματα που μπορούν να ταξινομηθούν με επαρκή βεβαιότητα σε πρώιμο στάδιο, η εκτέλεση τερματίζεται πριν διατρέξει ολόκληρο το μοντέλο [6], [7]. Με τον τρόπο αυτό μειώνονται ο μέσος χρόνος επεξεργασίας και το μέσο υπολογιστικό κόστος.

Παράλληλα, αρκετά σημαντικό ρόλο διαδραματίζουν οι τεχνικές κατανομής εκτέλεσης, όπως το partitioning και το split computing ανάμεσα σε διαθέσιμους edge workers. Σε αυτήν την προσέγγιση, το νευρωνικό δίκτυο διαχωρίζεται σε τμήματα (head and tail): κάθε τμήμα εκτελείται σε διαφορετικό κόμβο του τοπικού edge συστήματος. Ο διαχωρισμός αυτός επιτρέπει τον έλεγχο τόσο του υπολογιστικού φορτίου που αναλαμβάνει κάθε worker όσο και του όγκου των ενδιάμεσων δεδομένων που μεταδίδονται μέσω του δικτύου [8].

Με αυτόν τον τρόπο, το split computing προσφέρει έναν πιο ευέλικτο συμβιβασμό μεταξύ τοπικού υπολογιστικού κόστους, καθυστέρησης επικοινωνίας και φόρτου δικτύου.

1.1.4 Ανάγκη για ολοκληρωμένες λύσεις

Παρά τη σημαντική πρόοδο, πολλές προσεγγίσεις εξετάζουν το πρόβλημα αποσπασματικά. Ορισμένες επικεντρώνονται στον σχεδιασμό αποδοτικότερων early-exit αρχιτεκτονικών, άλλες στη συμπίεση ενός μεμονωμένου μοντέλου, ενώ άλλες στο να βελτιστοποιηθεί η τοποθέτηση ενός partitioned μοντέλου σε συγκεκριμένους κόμβους. Αν και οι προσεγγίσεις αυτές είναι χρήσιμες, δεν καλύπτουν πλήρως τις ανάγκες σύγχρονων edge συστημάτων, όπου πολλαπλές συσκευές, ετερογενείς υπολογιστικοί πόροι, μοντέλα με τη δυνατότητα early-exit και κοινόχρηστες δικτυακές υποδομές συνυπάρχουν στο ίδιο περιβάλλον.

Η πρόκληση γίνεται ακόμη πιο έντονη σε πολυ-μοντελικά (multi-model) σενάρια. Ένα edge cluster σπάνια εξυπηρετεί μία μόνο ροή inference. Συχνά καλείται να εκτελεί παράλληλα περισσότερες υπηρεσίες, διαφορετικά μοντέλα ή πολλαπλά logical instances του ίδιου μοντέλου. Σε αυτήν την περίπτωση, η απλή αναπαραγωγή ενός επιτυχημένου single-model pipeline δεν εγγυάται αποδοτική λειτουργία σε επίπεδο συστήματος. Τα συχνά ενεργά partitions, ιδίως τα πρώτα partitions ενός early-exit μοντέλου που επεξεργάζονται όλα τα δείγματα πριν ληφθεί απόφαση εξόδου, μπορούν να συγκεντρωθούν στους ίδιους workers και να δημιουργήσουν bottlenecks σε υπολογισμό, μνήμη και επικοινωνία.

Το περιβάλλον αυτό εισάγει νέες προκλήσεις, όπως ο συντονισμός πόρων, η δυναμική λήψη αποφάσεων, η αβεβαιότητα στις προβλέψεις και η εξισορρόπηση μεταξύ ακρίβειας, latency, throughput και κατανάλωσης ενέργειας. Επομένως, απαιτούνται πιο ολοκληρωμένες προσεγγίσεις, οι οποίες να συνδυάζουν μηχανισμούς early-exit, τεχνικές split computing και resource-aware τοποθέτηση partitions, όχι μόνο για ένα μεμονωμένο μοντέλο αλλά και για πολλαπλές ταυτόχρονες ροές inference.

Η ανάγκη αυτή αποτελεί το βασικό κίνητρο της παρούσας διπλωματικής εργασίας. Στόχος είναι η μελέτη της εκτέλεσης early-exit νευρωνικών δικτύων σε edge cluster, τόσο σε single-model όσο και σε multi-model σενάρια. Η εργασία εξετάζει πώς η κατανεμημένη εκτέλεση και η τοποθέτηση των partitions επηρεάζουν την ακρίβεια, το latency, το throughput, την κατανάλωση ενέργειας, το επικοινωνιακό κόστος και την αξιοποίηση των διαθέσιμων workers.

1.2 Στόχοι της Διπλωματικής

Η παρούσα διπλωματική εργασία έχει ως στόχο τη μελέτη της εκτέλεσης early-exit βαθιών νευρωνικών δικτύων σε περιορισμένο edge cluster, από τη σκοπιά τόσο του μοντέλου όσο και του συστήματος εκτέλεσης. Η αξιολόγηση δεν περιορίζεται στην ακρίβεια

ή στη θεωρητική μείωση του υπολογιστικού κόστους, αλλά εξετάζεται πώς η δυναμική ροή των δειγμάτων μέσα στο δίκτυο επηρεάζει end-to-end μετρικές, όπως το latency, το throughput, η κατανάλωση ενέργειας, το επικοινωνιακό κόστος και η αξιοποίηση των διαθέσιμων workers.

Η εργασία δεν αποσκοπεί στην πρόταση νέας early-exit αρχιτεκτονικής ή νέου αλγορίθμου εκπαίδευσης. Αντίθετα, χρησιμοποιεί ResNet-18 και ResNet-34 μοντέλα, σε συμβατική και early-exit μορφή, και εξετάζει πώς μεταβάλλεται η συμπεριφορά τους όταν εκτελούνται σε έναν κόμβο, σε partitioned distributed pipeline και σε multi-model σενάρια όπου δύο logical model instances μοιράζονται τους ίδιους edge workers. Κεντρική υπόθεση είναι ότι η αποδοτικότητα τέτοιων συστημάτων δεν καθορίζεται μόνο από το ίδιο το νευρωνικό μοντέλο, αλλά και από την τοποθέτηση των partitions και τη resource-aware διαχείριση της υποδομής [9]. Οι επιμέρους στόχοι της εργασίας συνοψίζονται ως εξής:

- Να αξιολογηθεί η επίδραση των early exits στη single-node εκτέλεση, συγκρίνοντας baseline ResNet μοντέλα με τις αντίστοιχες early-exit εκδοχές τους ως προς την ακρίβεια, το latency, το throughput και την ενεργειακή συμπεριφορά.
- Να μελετηθεί η κατανεμημένη εκτέλεση ενός early-exit μοντέλου σε διαδοχικά partitions πάνω σε πολλούς edge workers, ώστε να αποτιμηθεί πώς το split computing επηρεάζει το end-to-end latency, το communication/runtime overhead και την αξιοποίηση κάθε worker.
- Να εξεταστεί η πολυ-μοντελική εκτέλεση ως πρόβλημα κατανομής πόρων, μελετώντας την περίπτωση δύο logical instances του ίδιου early-exit μοντέλου που εκτελούνται ταυτόχρονα πάνω στην ίδια pipeline διαδρομή.
- Να διερευνηθεί αν η resource-aware τοποθέτηση των partitions μπορεί να βελτιώσει τη multi-model εκτέλεση χωρίς αλλαγή στο εκπαιδευμένο μοντέλο, στα weights ή στην πολιτική early-exit.

1.3 Δομή της Εργασίας

Στο Κεφάλαιο 2 παρουσιάζεται το θεωρητικό υπόβαθρο της εργασίας. Αρχικά εισάγονται τα βαθιά νευρωνικά και συνελικτικά δίκτυα, με έμφαση στην ιεραρχική εξαγωγή χαρακτηριστικών, στο κόστος του inference και στον συμβιβασμό ανάμεσα σε ακρίβεια και πόρους. Στη συνέχεια αναλύονται τα early-exit νευρωνικά δίκτυα, η αρχιτεκτονική των ενδιάμεσων εξόδων, η εκπαίδευση multi-exit μοντέλων και τα κριτήρια τερματισμού. Το κεφάλαιο συνεχίζει με το edge computing, τους περιορισμούς των edge συσκευών και τα πολυεπίπεδα ή πολυμοντελικά edge συστήματα. Ολοκληρώνεται με τις τεχνικές partitioning και split computing, το confidence-based routing, τον συνδυασμό split computing και early-exit εκτέλεσης, καθώς και τη resource-aware τοποθέτηση μοντέλων σε edge workers και την προοδευτική εκτέλεση.

Στο Κεφάλαιο 3 εξετάζεται η σχετική βιβλιογραφία. Παρουσιάζονται εργασίες που

αφορούν early-exit αρχιτεκτονικές και adaptive inference, μεθόδους split computing για edge inference, καθώς και προσεγγίσεις resource-aware deployment για dynamic DNNs. Ιδιαίτερη έμφαση δίνεται στο πώς οι υπάρχουσες μέθοδοι αντιμετωπίζουν τη σχέση ανάμεσα στο μοντέλο, το split point, το κόστος ενδιάμεσων αναπαραστάσεων και τους διαθέσιμους πόρους του συστήματος. Το κεφάλαιο καταλήγει σε σύνθεση της βιβλιογραφίας που παρουσιάστηκε και τοποθετεί την παρούσα εργασία στο κενό μεταξύ της μοντελοκεντρικής βελτιστοποίησης και της συστημικής αξιολόγησης multi-model edge inference.

Στο Κεφάλαιο 4 περιγράφεται η μεθοδολογία και το πειραματικό πλαίσιο. Αρχικά διατυπώνονται ο σκοπός της πειραματικής μελέτης και η μεθοδολογική προσέγγιση. Στη συνέχεια παρουσιάζονται το λογισμικό εκτέλεσης, η πειραματική υποδομή Raspberry Pi edge cluster, τα σύνολα δεδομένων CIFAR-10 και CIFAR-10.1, καθώς και η προεπεξεργασία εισόδου. Περιγράφονται επίσης τα ResNet-18 και ResNet-34 μοντέλα, η early-exit επέκτασή τους, η εκπαίδευση, τα αποθηκευμένα weights και η entropy-based πολιτική εξόδου. Το κεφάλαιο αναλύει τη διάσπαση του early-exit pipeline σε τρία partitions, το μοντέλο ροής φόρτου και τη λογική της πολυ-μοντελικής τοποθέτησης. Οι πειραματικές τοπολογίες οργανώνονται σε τρεις ομάδες: single-model εκτέλεση, αφελή multi-model εκτέλεση δύο logical model instances και memory-aware placement στρατηγικές. Τέλος, περιγράφονται το distributed runtime, το πρωτόκολλο επικοινωνίας, οι μετρικές αξιολόγησης, οι ενεργειακές μετρήσεις και οι περιορισμοί της πειραματικής μελέτης.

Στο Κεφάλαιο 5 παρουσιάζονται τα πειραματικά αποτελέσματα. Η αξιολόγηση οργανώνεται με αυξανόμενη πολυπλοκότητα, ξεκινώντας από το Experiment 1, όπου συγκρίνονται τα baseline ResNet-18 και ResNet-34 με τις αντίστοιχες early-exit εκδοχές τους σε single-node εκτέλεση και στη συνέχεια σε distributed εκτέλεση τριών workers. Ακολουθεί το Experiment 2, το οποίο χρησιμοποιεί δύο logical instances του ίδιου early-exit μοντέλου στην ίδια pipeline διαδρομή ως αφελές multi-model baseline. Στο Experiment 3 εξετάζονται memory-aware στρατηγικές placement που αλλάζουν την τοποθέτηση των partitions χωρίς να αλλάζουν το μοντέλο ή την πολιτική early-exit. Το κεφάλαιο ολοκληρώνεται με ξεχωριστή ανάλυση της ενεργειακής αποδοτικότητας, του Energy-Delay Product και συμπληρωματική αξιολόγηση στο CIFAR-10.1.

Στο Κεφάλαιο 6 συζητούνται συνολικά τα ευρήματα της πειραματικής μελέτης. Η συζήτηση ερμηνεύει το early-exit ως μηχανισμό προσαρμοστικού υπολογισμού και αναλύει γιατί η κατανομημένη και πολυ-μοντελική εκτέλεση δεν οδηγεί αυτόματα σε καλύτερη αξιοποίηση του cluster. Στη συνέχεια εξετάζει το placement ως πολυκριτήριο πρόβλημα βελτιστοποίησης, όπου η απόδοση εξαρτάται από την κατανομή των exits, το data distribution, τα thresholds, το communication/runtime overhead, το utilization και την ενέργεια. Το κεφάλαιο ολοκληρώνεται με τις σχεδιαστικές αρχές που προκύπτουν για distributed early-exit inference σε edge περιβάλλοντα.

Στο Κεφάλαιο 7 παρουσιάζονται κατευθύνσεις μελλοντικής εργασίας και διατυπώνονται οι τελικές παρατηρήσεις. Οι κατευθύνσεις αυτές αφορούν, μεταξύ άλλων, την αξιολόγηση περισσότερων datasets και αρχιτεκτονικών, τη μελέτη διαφορετικών πολιτικών early-exit, την ανάπτυξη δυναμικών placement μηχανισμών, τη μελέτη διαφορετικών partition boundaries, την επέκταση σε πιο ετερογενείς edge υποδομές και τη βελτίωση της παρατηρησιμότητας, της ενεργειακής μέτρησης και του communication/runtime overhead.

Κεφάλαιο 2

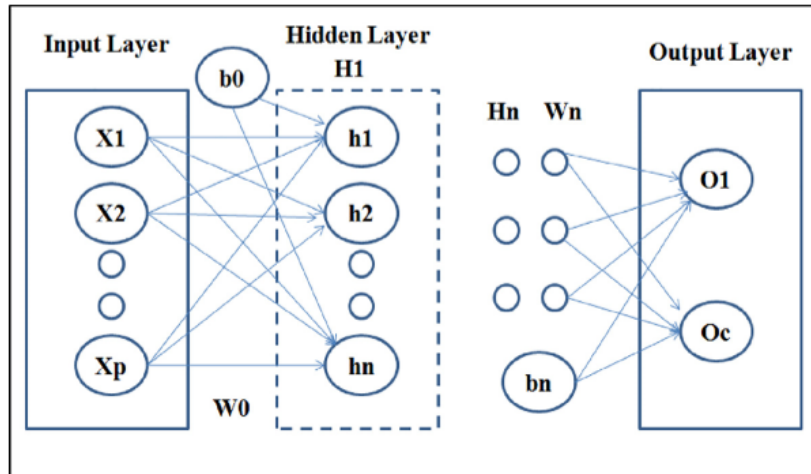
Θεωρητικό Υπόβαθρο

Το παρόν κεφάλαιο εισάγει τις βασικές έννοιες πάνω στις οποίες βασίζεται η διπλωματική εργασία. Η κεντρική θεματική είναι η αποδοτική εκτέλεση inference σε βαθιά νευρωνικά δίκτυα που αναπτύσσονται σε καταναμημένα edge περιβάλλοντα, όπου οι διαθέσιμοι υπολογιστικοί πόροι, η ενέργεια, το latency και το επικοινωνιακό κόστος αποτελούν βασικούς περιορισμούς [8]. Για τον λόγο αυτό, το κεφάλαιο ξεκινά αρχικά από τη λειτουργία των βαθιών νευρωνικών δικτύων, συνεχίζει με τα early-exit μοντέλα, παρουσιάζει το edge computing ως υπολογιστικό παράδειγμα και καταλήγει στις τεχνικές partitioning και split computing.

2.1 Βαθιά Νευρωνικά Δίκτυα

Τα βαθιά νευρωνικά δίκτυα (Deep Neural Networks, DNNs) είναι τεχνητά νευρωνικά δίκτυα με πολλαπλά κρυφά επίπεδα ανάμεσα στην είσοδο και την έξοδο. Σε ένα DNN, τα επίπεδα εφαρμόζονται διαδοχικά: η έξοδος κάθε επιπέδου αποτελεί την είσοδο του επόμενου. Με αυτόν τον τρόπο, η αρχική είσοδος μετατρέπεται σταδιακά σε όλο και πιο αφηρημένες αναπαραστάσεις. Τα πρώτα επίπεδα έχουν την τάση να συλλαμβάνουν απλούστερα χαρακτηριστικά, ενώ τα βαθύτερα επίπεδα συνδυάζουν αυτά τα χαρακτηριστικά σε πιο σύνθετες αναπαραστάσεις. Αυτή η ιεραρχική οργάνωση επιτρέπει στα DNNs να μοντελοποιούν μη γραμμικές σχέσεις υψηλής πολυπλοκότητας και να αξιοποιούν δεδομένα μεγάλης διάστασης, όπως εικόνες, σήματα, κείμενο ή ακολουθίες γεγονότων [10], [11]. Η γενική αυτή αλληλουχία εισόδου, κρυφών επιπέδων και εξόδου αποτυπώνεται στην Εικόνα 2.1.

Η επιτυχία των DNNs οφείλεται σε μεγάλο βαθμό στη δυνατότητά τους να μαθαίνουν χαρακτηριστικά απευθείας από τα δεδομένα, χωρίς να απαιτείται ρητή χειροκίνητη εξαγωγή ή κατασκευή χαρακτηριστικών από τον σχεδιαστή του συστήματος. Αυτό είναι ιδιαίτερα σημαντικό σε προβλήματα όπου τα δεδομένα δεν είναι δομημένα, όπως εικόνες από ιατρικές εξετάσεις, δεδομένα αισθητήρων για αναγνώριση δραστηριότητας και δείγματα κακόβουλου λογισμικού [10], [12], [11]. Η ίδια ιδιότητα, ωστόσο, συνοδεύεται από αυξημένες απαιτήσεις σε υπολογιστική δύναμη, μνήμη και δεδομένα εκπαίδευσης. Όσο αυξάνονται τα στρώματα ενός μοντέλου και ο αριθμός των επιμέρους παραμέτρων σε αυτά, τόσο αυξάνεται και ο αριθμός των πράξεων και των ενδιάμεσων ενεργοποιήσεων που πρέπει να αποθηκευτούν ή να μεταφερθούν κατά την εκτέλεση.



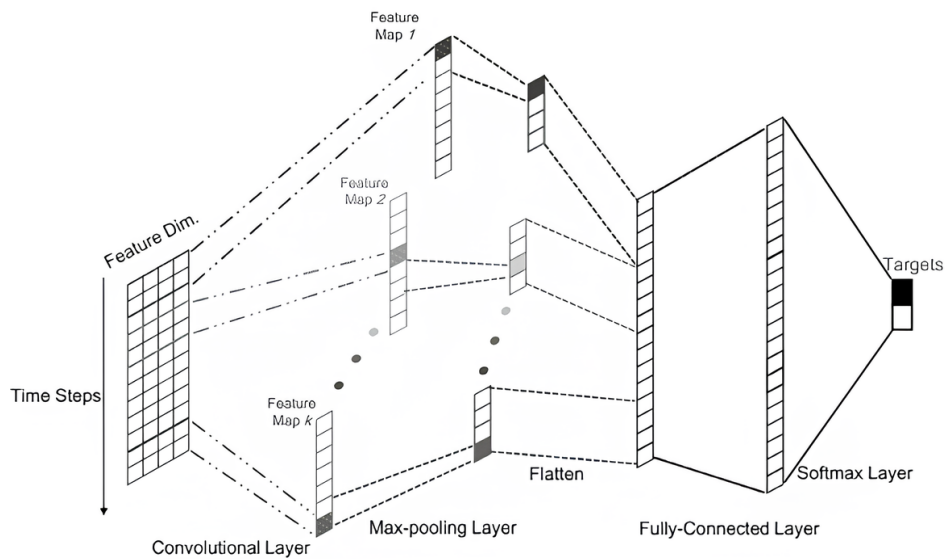
Εικόνα 2.1: Ενδεικτική αρχιτεκτονική βαθιού νευρωνικού δικτύου με επίπεδο εισόδου, πολλαπλά κρυφά επίπεδα και επίπεδο εξόδου [10].

2.1.1 Συνελικτικά Δίκτυα

Τα συνελικτικά νευρωνικά δίκτυα (Convolutional Neural Networks, CNNs) αποτελούν μία από τις πιο διαδεδομένες κατηγορίες DNNs, ιδίως για δεδομένα με χωρική ή χρονική δομή. Αρχικά έγιναν γνωστά μέσω της υπολογιστικής όρασης, επειδή μπορούν να εκμεταλλευτούν τη γειτονικότητα των εικονοστοιχείων και να αναγνωρίσουν τοπικά μοτίβα σε εικόνες. Η ίδια αρχή, όμως, έχει χρησιμοποιηθεί και σε άλλα είδη δεδομένων, όπως σήματα αισθητήρων, βιοϊατρικές εικόνες και δεδομένα λογισμικού που μετατρέπονται σε εικόνες ή ακολουθίες bytes/εντολών [13], [12], [14], [11].

Η τυπική αρχιτεκτονική ενός CNN αποτελείται από συνελικτικά επίπεδα (convolutional layer), επίπεδα υποδειγματοληψίας ή συγκέντρωσης (pooling layer) και, συνήθως, πλήρως συνδεδεμένα επίπεδα (fully connected layer) στο τέλος του δικτύου [13], [15]. Η συνολική αυτή ροή επεξεργασίας φαίνεται στην Εικόνα 2.2. Το συνελικτικό επίπεδο είναι το βασικό δομικό στοιχείο αυτής της αρχιτεκτονικής. Σε αυτό, ένα σύνολο από φίλτρα ή πυρήνες (filters or kernels) μικρής διάστασης ολισθαίνει πάνω στην είσοδο, για παράδειγμα πάνω στις περιοχές μιας εικόνας, και υπολογίζει τοπικές αποκρίσεις. Η πράξη αυτή αντιστοιχεί στη συνέλιξη της εισόδου με το εκάστοτε φίλτρο. Κάθε φίλτρο μαθαίνει να ανιχνεύει ένα συγκεκριμένο μοτίβο, όπως ακμές, γωνίες, υφές ή πιο σύνθετους σχηματισμούς στα βαθύτερα επίπεδα. Η έξοδος κάθε φίλτρου είναι ένας χάρτης χαρακτηριστικών (feature map), ο οποίος δείχνει σε ποιες περιοχές της εισόδου εμφανίζεται το αντίστοιχο μοτίβο. Ο τρόπος αυτός οδηγεί σε σημαντική μείωση των παραμέτρων σε σχέση με ένα πλήρως συνδεδεμένο επίπεδο, κυρίως λόγω δύο ιδιοτήτων. Η πρώτη είναι η τοπική συνδεσιμότητα: κάθε φίλτρο βλέπει κάθε φορά μόνο μια μικρή γειτονιά της εισόδου. Η δεύτερη είναι η επαναχρησιμοποίηση παραμέτρων: το ίδιο φίλτρο εφαρμόζεται σε πολλές διαφορετικές θέσεις της εισόδου, ώστε οι ίδιες παράμετροι να ανιχνεύουν το ίδιο μοτίβο σε διαφορετικές περιοχές. Έτσι, το συνελικτικό επίπεδο αποφεύγει να συνδέσει κάθε έξοδο με όλα τα στοιχεία της εισόδου και παραμένει πολύ πιο οικονομικό σε πλήθος παραμέτρων.

Μετά το συνελικτικό επίπεδο εφαρμόζεται συνήθως μία μη γραμμική συνάρτηση εν-



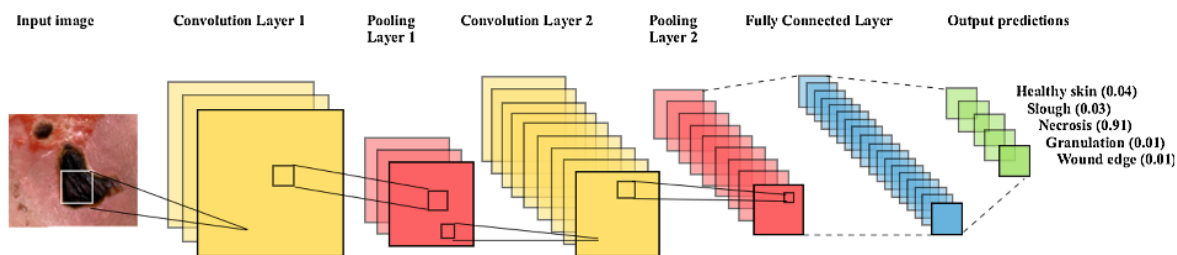
Εικόνα 2.2: Γενική ροή επεξεργασίας σε συνελκτικό νευρωνικό δίκτυο, από την είσοδο και τους χάρτες χαρακτηριστικών μέχρι το πλήρως συνδεδεμένο επίπεδο και την τελική πρόβλεψη [12].

εργοποίησης, όπως η ReLU, ώστε το δίκτυο να μπορεί να μάθει μη γραμμικές σχέσεις και όχι απλώς διαδοχικούς γραμμικούς μετασχηματισμούς. Στη συνέχεια, τα επίπεδα pooling συνοψίζουν γειτονικές τιμές των feature maps, συχνά κρατώντας τη μέγιστη (max pooling) ή τη μέση τιμή σε μικρές περιοχές. Με αυτόν τον τρόπο μειώνεται η χωρική διάσταση της αναπαράστασης, περιορίζεται το υπολογιστικό κόστος και το μοντέλο γίνεται πιο ανθεκτικό σε μικρές μετατοπίσεις ή παραμορφώσεις της εισόδου [13], [14]. Στα τελευταία επίπεδα, τα feature maps συνήθως μετατρέπονται σε ένα διάνυσμα (flattening) και τροφοδοτούν ένα ή περισσότερα fully connected layers, τα οποία συνδυάζουν τα χαρακτηριστικά υψηλού επιπέδου και παράγουν την τελική πρόβλεψη του μοντέλου.

Η στοίβαξη πολλών συνελκτικών και pooling επιπέδων οδηγεί σε ιεραρχική εξαγωγή χαρακτηριστικών. Σε μία εικόνα, για παράδειγμα, τα πρώτα επίπεδα μπορούν να αναγνωρίζουν ακμές ή απλές υφές, ενώ τα βαθύτερα επίπεδα συνδυάζουν αυτά τα στοιχεία σε πιο σύνθετα μοτίβα και αντικείμενα. Στα δεδομένα αισθητήρων, αντίστοιχα, τα CNNs μπορούν να εξάγουν τοπικά χρονικά ή χωροχρονικά χαρακτηριστικά από ακατέργαστες μετρήσεις, μειώνοντας την ανάγκη για χειροκίνητη προεπεξεργασία [12]. Για τον λόγο αυτό τα CNNs αποτελούν αντιπροσωπευτικό παράδειγμα της βασικής ιδέας του deep learning. Ένα ενδεικτικό παράδειγμα τέτοιας ροής για ανάλυση εικόνας φαίνεται στην Εικόνα 2.3.

2.1.2 Inference σε DNNs

Με τον όρο inference, ή εξαγωγή συμπεράσματος, περιγράφεται η φάση κατά την οποία ένα ήδη εκπαιδευμένο DNN δέχεται μία νέα είσοδο και υπολογίζει την αντίστοιχη πρόβλεψη. Σε αυτή τη φάση τα βάρη του μοντέλου είναι σταθερά και η εκτέλεση αντιστοιχεί σε μία διαδοχή προκαθορισμένων πράξεων: πολλαπλασιασμούς πινάκων, συνελίξεις, μη γραμμικές συναρτήσεις ενεργοποίησης, κανονικοποιήσεις και, στο τέλος, παραγωγή



Εικόνα 2.3: Παράδειγμα CNN για ανάλυση εικόνας, όπου διαδοχικά συνελκτικά και pooling επίπεδα μετατρέπουν την είσοδο σε χαρακτηριστικά που τροφοδοτούν το τελικό πλήρως συνδεδεμένο επίπεδο ταξινόμησης [14].

κατανομής πιθανοτήτων ή άλλης μορφής εξόδου. Παρότι το inference δεν περιλαμβάνει backpropagation και ενημέρωση παραμέτρων, εξακολουθεί να απαιτεί τη διάδοση κάθε εισόδου μέσα από τα διαδοχικά επίπεδα του δικτύου. Επομένως, όσο αυξάνονται το βάθος του μοντέλου, το μέγεθος των ενδιάμεσων ενεργοποιήσεων και ο ρυθμός άφιξης των εισόδων, τόσο μεγαλύτερες γίνονται οι απαιτήσεις σε χρόνο εκτέλεσης, μνήμη και υπολογιστικούς πόρους.

Το κόστος του inference δεν καθορίζεται μόνο από τον αριθμό των παραμέτρων του μοντέλου. Εξαρτάται ακόμη από τον αριθμό των αριθμητικών πράξεων, το μέγεθος των ενδιάμεσων ενεργοποιήσεων, την πρόσβαση στη μνήμη και τη δυνατότητα παραλληλοποίησης. Γι' αυτό η χρήση εξειδικευμένου υλικού, όπως GPUs, επιταχυντές νευρωνικών δικτύων ή ισχυροί edge κόμβοι, μπορεί να μειώσει σημαντικά τον χρόνο εκτέλεσης, ειδικά σε μοντέλα μεγάλης κλίμακας [13], [10]. Σε περιβάλλοντα περιορισμένων πόρων, όμως, η πλήρης τοπική εκτέλεση ενός DNN δεν είναι πάντα εφικτή λόγω περιορισμών σε μνήμη, ενέργεια και θερμικό προϋπολογισμό.

2.1.3 Συμβιβασμός Ακρίβειας και Πόρων

Η αύξηση του βάθους και της χωρητικότητας ενός DNN κατά κανόνα βελτιώνει την ικανότητα μάθησης αναπαραστάσεων και ενισχύει την προβλεπτική ικανότητα του μοντέλου, αλλά συνοδεύεται από αύξηση τόσο του συνολικού χρόνου εκτέλεσης όσο και του υπολογιστικού φόρτου. Έτσι δημιουργείται ένας θεμελιώδης συμβιβασμός ανάμεσα στην ακρίβεια και στους πόρους που απαιτούνται για την επίτευξή της. Ένα μεγαλύτερο μοντέλο μπορεί να προσφέρει καλύτερη επίδοση σε δύσκολα δείγματα, αλλά χρειάζεται περισσότερη μνήμη για τις παραμέτρους, περισσότερες πράξεις ανά είσοδο και συχνά μεγαλύτερο latency.

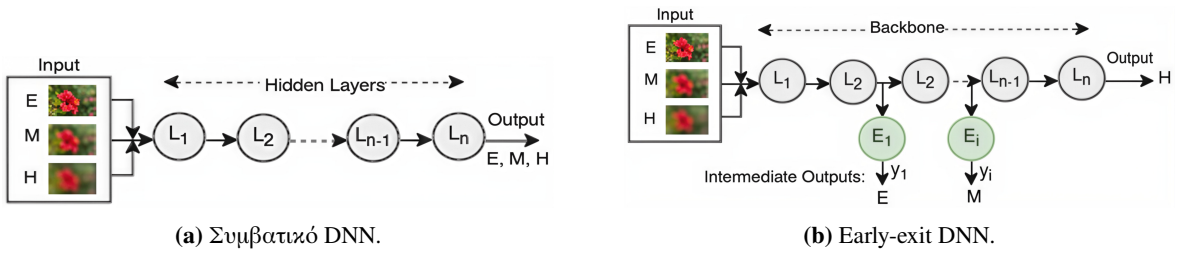
Ο συμβιβασμός αυτός είναι κεντρικός για την εκτέλεση DNNs σε edge συστήματα. Σε μία φορητή ή ενσωματωμένη συσκευή, η τοπική εκτέλεση μειώνει την ανάγκη επικοινωνίας, αλλά επιβαρύνει τον περιορισμένο υπολογιστικό και ενεργειακό προϋπολογισμό. Αντίθετα, η κατανομή του υπολογισμού σε περισσότερους ή ισχυρότερους edge κόμβους μπορεί να επιταχύνει την επεξεργασία του μοντέλου, αλλά εισάγει καθυστέρηση δικτύου και κόστος μετάδοσης ενδιάμεσων δεδομένων. Οι τεχνικές που παρουσιάζονται στις επόμενες ενότητες, όπως τα early-exit μοντέλα, το partitioning και το split computing, επιχειρούν να εκμεταλλευτούν ακριβώς αυτόν τον χώρο σχεδιασμού, προσαρμόζοντας

την εκτέλεση στη δυσκολία του δείγματος και στη διαθεσιμότητα των πόρων.

2.2 Early-Exit Νευρωνικά Δίκτυα

Τα early-exit νευρωνικά δίκτυα αποτελούν μία οικογένεια δυναμικών DNNs που μπορούν να παράγουν πρόβλεψη πριν από το τελικό επίπεδο του μοντέλου. Η βασική ιδέα είναι ότι δεν έχουν όλα τα δείγματα την ίδια δυσκολία. Ορισμένες εισόδους μπορούν να ταξινομηθούν με υψηλή βεβαιότητα με βάση σχετικά ρηχές αναπαραστάσεις, ενώ άλλες χρειάζονται το πλήρες βάθος του δικτύου. Αντί, λοιπόν, κάθε δείγμα να διατρέχει υποχρεωτικά όλα τα επίπεδα, το δίκτυο αποκτά ενδιάμεσους ταξινομητές, οι οποίοι μπορούν να τερματίσουν νωρίτερα την εκτέλεση όταν η πρόβλεψη θεωρηθεί επαρκώς αξιόπιστη [6], [7], [16].

Η προσέγγιση αυτή είναι ιδιαίτερα χρήσιμη σε περιβάλλοντα με αυστηρούς περιορισμούς latency, κατανάλωσης ενέργειας ή διαθέσιμης υπολογιστικής ισχύος. Σε ένα συμβατικό DNN, όπως φαίνεται στο Εικόνα 2.4a, η ίδια ακολουθία επιπέδων εκτελείται για κάθε είσοδο, ανεξάρτητα από το αν η είσοδος είναι εύκολη ή δύσκολη. Αντίθετα, όπως φαίνεται στην Εικόνα 2.4b, ένα early-exit DNN επιτρέπει προσαρμοστική εκτέλεση: εύκολα δείγματα μπορούν να εξέλθουν σε πρώιμα σημεία, ενώ δυσκολότερα δείγματα συνεχίζουν προς βαθύτερα επίπεδα.



Εικόνα 2.4: Σύγκριση συμβατικής και early-exit εκτέλεσης. Στο συμβατικό DNN όλα τα δείγματα διατρέχουν ολόκληρο το δίκτυο, ενώ στο early-exit DNN οι ενδιάμεσες έξοδοι επιτρέπουν πρόβλεψη σε διαφορετικά βάθη, ανάλογα με τη δυσκολία της εισόδου [16].

2.2.1 Αρχιτεκτονική early-exit

Ένα early-exit DNN βασίζεται συνήθως σε ένα υπάρχον backbone μοντέλο, όπως ένα CNN τύπου ResNet, VGG ή MobileNet, στο οποίο προστίθενται πλευρικοί ταξινομητές σε επιλεγμένα ενδιάμεσα βάθη [6], [16]. Το backbone εξακολουθεί να υπολογίζει διαδοχικά τις αναπαραστάσεις της εισόδου, αλλά σε ορισμένα σημεία οι ενδιάμεσες ενεργοποιήσεις τροφοδοτούν μικρότερες κεφαλές ταξινόμησης (classifier heads). Κάθε τέτοια κεφαλή παράγει μία πρόβλεψη, όπως ακριβώς και η τελική έξοδος του μοντέλου, αλλά με μικρότερο υπολογιστικό κόστος, επειδή αξιοποιεί μόνο το τμήμα του δικτύου που έχει προηγηθεί. Αν συμβολίσουμε με $L_1, L_2, \dots, L_n$ τα διαδοχικά επίπεδα του backbone, ένας ενδιάμεσος ταξινομητής E_i τοποθετείται μετά από κάποιο επίπεδο L_i . Η έξοδος του E_i είναι μία μερική πρόβλεψη y_i , ενώ η τελική έξοδος παράγεται μετά το τελευταίο

επίπεδο. Κατά το inference, το δείγμα μπορεί να σταματήσει σε έναν ενδιαμέσο ταξινομητή αν η ενδιαμέση πρόβλεψη ικανοποιεί ένα κριτήριο βεβαιότητας, ή να συνεχίσει προς τα επόμενα επίπεδα αν η πρόβλεψη παραμένει αβέβαιη. Έτσι, το βάθος εκτέλεσης γίνεται χαρακτηριστικό του ίδιου του δείγματος και όχι σταθερή ιδιότητα του μοντέλου.

Οι ενδιαμέσοι ταξινομητές πρέπει να είναι αρκετά ελαφριοί ώστε το κόστος τους να μην ακυρώνει το όφελος του early-exit. Συνήθως αποτελούνται από λίγα συνελικτικά ή πλήρως συνδεδεμένα επίπεδα, ενδεχομένως με pooling ή flattening ανάλογα με τη μορφή της ενδιαμέσης αναπαράστασης. Παράλληλα, δεν πρέπει να είναι υπερβολικά απλοί, διότι τότε οι πρώιμες προβλέψεις θα έχουν χαμηλή ακρίβεια. Η επιλογή των σημείων εξόδου (exit points) είναι επομένως κρίσιμη: πολύ ρηχές έξοδοι μπορεί να βασίζονται σε ανεπαρκή χαρακτηριστικά, ενώ πολύ βαθιές έξοδοι μειώνουν λιγότερο το υπολογιστικό κόστος. Ο σχεδιασμός κάθε κλάδου δεν είναι ανεξάρτητος από το υπόλοιπο δίκτυο: οι θέσεις των κλάδων (branches), το βάθος και το μέγεθός τους, καθώς και το κριτήριο εξόδου καθορίζουν από κοινού τον συμβιβασμό ανάμεσα σε ακρίβεια και ταχύτητα [6].

2.2.2 Εκπαίδευση Multi-Exit Μοντέλων

Η εκπαίδευση ενός multi-exit μοντέλου είναι πιο σύνθετη από την εκπαίδευση ενός συμβατικού single-exit DNN, επειδή το δίκτυο πρέπει να μάθει χρήσιμες αναπαραστάσεις για πολλαπλά βάθη ταυτόχρονα. Η τελική έξοδος χρειάζεται βαθιά, αφηρημένα χαρακτηριστικά, ενώ οι ενδιαμέσες έξοδοι πρέπει να μπορούν να ταξινομήσουν δείγματα με λιγότερη πληροφορία. Επομένως, η επιλογή στρατηγικής εκπαίδευσης επηρεάζει άμεσα τόσο την ακρίβεια κάθε εξόδου όσο και τη συνολική σταθερότητα του μοντέλου [16].

Η πιο βασική προσέγγιση είναι το joint training. Σε αυτή την προσέγγιση, το backbone και όλοι οι ενδιαμέσοι ταξινομητές εκπαιδεύονται ταυτόχρονα. Κάθε έξοδος παράγει τη δική της απώλεια, και η συνολική απώλεια του μοντέλου υπολογίζεται ως σταθμισμένο άθροισμα των επιμέρους απωλειών. Σε προβλήματα ταξινόμησης, η απώλεια κάθε εξόδου υπολογίζεται συνήθως με τη συνάρτηση cross-entropy, η οποία συγκρίνει την κατανομή πιθανοτήτων που παράγει ο ταξινομητής με την πραγματική ετικέτα του δείγματος. Για την έξοδο i , αν $p_{i,c}$ είναι η πιθανότητα που αποδίδει ο ταξινομητής στην κλάση c , και y_c η αντίστοιχη one-hot ετικέτα, η επιμέρους απώλεια γράφεται:

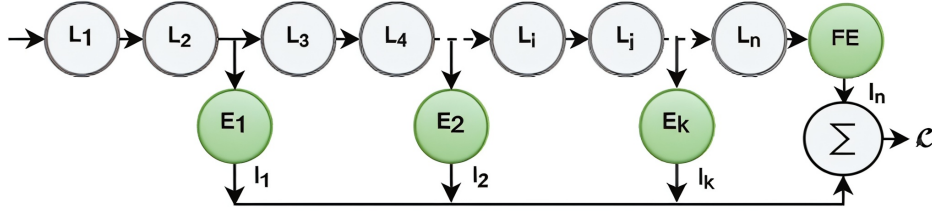
$$\mathcal{L}_i = - \sum_{c=1}^C y_c \log p_{i,c}, \quad (2.1)$$

όπου C είναι ο αριθμός των κλάσεων. Με βάση τις επιμέρους απώλειες όλων των εξόδων, η συνολική απώλεια του μοντέλου γράφεται:

$$\mathcal{L} = \sum_{i=1}^{N+1} w_i \mathcal{L}_i, \quad (2.2)$$

όπου N είναι ο αριθμός των ενδιαμέσων εξόδων, η έξοδος $N + 1$ αντιστοιχεί στην τελική έξοδο, $\mathcal{L}_i$ είναι η απώλεια cross-entropy της εξόδου i , και w_i είναι ο συντελεστής

που ρυθμίζει τη σχετική σημασία της συγκεκριμένης εξόδου κατά την εκπαίδευση. Η διαδικασία αυτή, όπως απεικονίζεται στην Εικόνα 2.5, επιτρέπει την από κοινού εκπαίδευση των early-exit κεφαλών και της τελικής εξόδου, αλλά εισάγει επιπλέον υπερπαραμέτρους στάθμισης, καθώς τα βάρη w_i καθορίζουν τη σχετική επίδραση κάθε απώλειας εξόδου στη συνολική βελτιστοποίηση [6], [16]. Αν η τελική έξοδος κυριαρχεί στην απώλεια, οι early-exit κεφαλές μπορεί να υποεκπαιδευτούν. Αντίθετα, υπερβολική έμφαση στις early-exit κεφαλές μπορεί να επηρεάσει αρνητικά την ποιότητα των βαθύτερων αναπαραστάσεων.



Εικόνα 2.5: Joint training σε early-exit DNN. Οι ενδιάμεσες εξοδοί και η τελική έξοδος παράγουν ξεχωριστές απώλειες, οι οποίες συνδυάζονται σε μία συνολική απώλεια για την ενημέρωση του backbone και των κλάδων εξόδου [16].

Η βιβλιογραφία εξετάζει και άλλες στρατηγικές εκπαίδευσης. Ωστόσο, στο πλαίσιο της παρούσας εργασίας, η έμφαση δίνεται στο joint training, καθώς είναι η πιο διαδεδομένη προσέγγιση και έχει αποδειχθεί αποτελεσματική σε πολλές περιπτώσεις.

2.2.3 Κριτήρια Τερματισμού

Κατά το inference, κάθε ενδιάμεση έξοδος πρέπει να συνοδεύεται από έναν κανόνα απόφασης. Ο κανόνας αυτός καθορίζει αν η πρόβλεψη της τρέχουσας εξόδου είναι αρκετά αξιόπιστη ώστε η εκτέλεση να σταματήσει, ή αν το δείγμα πρέπει να συνεχίσει προς βαθύτερα επίπεδα. Τα κριτήρια τερματισμού βασίζονται συνήθως σε κάποια εκτίμηση εμπιστοσύνης ή αβεβαιότητας της ενδιάμεσης πρόβλεψης. Οι πιο απλές τεχνικές χρησιμοποιούν απευθείας την κατανομή πιθανοτήτων της εξόδου, όπως τη μέγιστη softmax πιθανότητα, το margin ανάμεσα στις δύο πιθανότερες κλάσεις ή την εντροπία της κατανομής. Πιο σύνθετες προσεγγίσεις μπορούν να αξιοποιούν βαθμονόμηση εμπιστοσύνης, πρόσθετους εκτιμητές αβεβαιότητας ή κανόνες που λαμβάνουν υπόψη περιορισμούς latency και υπολογιστικού κόστους [6], [7]. Για τα στατικά κριτήρια τερματισμού, η απόφαση βασίζεται συνήθως στην κατανομή πιθανοτήτων που παράγει η ενδιάμεση έξοδος. Έστω ότι η έξοδος E_i παράγει logits $z_i \in \mathbb{R}^C$, όπου C είναι ο αριθμός των κλάσεων. Η softmax μετατρέπει τα logits σε μια κανονικοποιημένη κατανομή πιθανοτήτων y_i , με μη αρνητικές τιμές που αθροίζουν στη μονάδα:

$$y_{i,c} = \frac{\exp(z_{i,c})}{\sum_{k=1}^C \exp(z_{i,k})}, \quad c = 1, \dots, C. \quad (2.3)$$

Με βάση αυτή την κατανομή, μπορούν να οριστούν απλά μέτρα εμπιστοσύνης και αβεβαιότητας. Η μέγιστη softmax πιθανότητα εκφράζει πόσο έντονα το μοντέλο προ-

τιμά μία κλάση, το margin μετρά την απόσταση ανάμεσα στην πρώτη και τη δεύτερη πιθανότερη κλάση, ενώ η εντροπία μετρά πόσο διασκορπισμένη είναι η κατανομή:

$$p_i^{\max} = \max_c y_{i,c}, \quad m_i = y_{i,(1)} - y_{i,(2)}, \quad H(y_i) = - \sum_{c=1}^C y_{i,c} \log y_{i,c}. \quad (2.4)$$

Στον παραπάνω τύπο, $y_{i,(1)}$ και $y_{i,(2)}$ είναι η μεγαλύτερη και η δεύτερη μεγαλύτερη πιθανότητα της κατανομής. Υψηλή τιμή της μέγιστης πιθανότητας ή μεγάλο margin υποδηλώνουν ότι η πρόβλεψη είναι λιγότερο αμφίσημη. Αντίστοιχα, χαμηλή εντροπία σημαίνει ότι η κατανομή είναι συγκεντρωμένη σε λίγες κλάσεις, συνήθως σε μία κυρίαρχη κλάση, άρα η πρόβλεψη θεωρείται πιο βέβαιη. Στην περίπτωση κριτηρίου βασισμένου στην εντροπία, αν η εντροπία είναι μικρότερη από ένα threshold T_i για την έξοδο E_i , το δείγμα εξέρχεται από το δίκτυο· διαφορετικά συνεχίζει στο επόμενο τμήμα του μοντέλου [6]. Μία γενική μορφή του κανόνα μπορεί να γραφεί ως:

$$\text{exit at } E_i \quad \text{if} \quad H(y_i) < T_i, \quad (2.5)$$

όπου y_i είναι η κατανομή πιθανοτήτων της εξόδου E_i , $H(\cdot)$ είναι η εντροπία και T_i το αντίστοιχο threshold. Το threshold ρυθμίζεται με βάση τις απαιτήσεις της εφαρμογής. Ένα αυστηρό threshold επιτρέπει έξοδο μόνο όταν η αβεβαιότητα είναι πολύ χαμηλή, διατηρώντας υψηλότερη ακρίβεια αλλά οδηγώντας περισσότερα δείγματα προς τα βαθύτερα επίπεδα. Ένα πιο χαλαρό threshold αυξάνει το ποσοστό των πρόωρων εξόδων και μειώνει το latency, αλλά μπορεί να επιτρέψει λανθασμένες προβλέψεις από ρηχούς ταξινομητές, μειώνοντας τελικά την ακρίβεια.

Σε κάθε περίπτωση, το κρίσιμο ζήτημα είναι η αξιοπιστία της εκτίμησης αβεβαιότητας. Πολλά DNNs μπορεί να εμφανίζονται υπερβολικά βέβαια στις προβλέψεις τους, οπότε οι softmax πιθανότητες δεν αντιστοιχούν πάντα σε πραγματικές πιθανότητες ορθότητας. Για τον λόγο αυτό, η βαθμονόμηση της εμπιστοσύνης και η προσεκτική επιλογή κατωφλίων είναι σημαντικές για πρακτικά συστήματα early-exit και confidence-based routing [7].

2.2.4 Οφέλη και Περιορισμοί

Το βασικό όφελος των early-exit DNNs είναι η μείωση του μέσου κόστους εκτέλεσης. Επειδή σημαντικό ποσοστό των δειγμάτων μπορεί να ταξινομηθεί σε ενδιάμεσα βάθη, το σύστημα αποφεύγει την εκτέλεση των βαθύτερων και ακριβότερων επιπέδων για κάθε είσοδο.

Επιπλέον, οι ενδιάμεσες εξοδοί μπορούν να επηρεάσουν θετικά τη διαδικασία εκπαίδευσης. Επειδή κάθε έξοδος συνεισφέρει τη δική της απώλεια, τα ρηχότερα επίπεδα δεν εκπαιδεύονται μόνο μέσω του σήματος που επιστρέφει από την τελική έξοδο, αλλά λαμβάνουν και πιο άμεσα σήματα gradient από τις ενδιάμεσες κεφαλές. Αυτό μπορεί να διευκολύνει τη μάθηση χρήσιμων αναπαραστάσεων σε πρώιμα βάθη και να λειτουργήσει ως μορφή κανονικοποίησης [6]. Ωστόσο, το τελικό αποτέλεσμα εξαρτάται έντονα

από τη στρατηγική εκπαίδευσης, τη στάθμιση των απωλειών και τη σχέση ανάμεσα στις early-exit κεφαλές και την τελική έξοδο [16].

Οι περιορισμοί είναι εξίσου σημαντικοί. Πρώτον, η προσθήκη κλάδων αυξάνει το μέγεθος του μοντέλου και εισάγει επιπλέον υπολογιστικό κόστος σε κάθε σημείο εξόδου. Αν οι κλάδοι έχουν υψηλή υπολογιστική πολυπλοκότητα ή τοποθετηθούν σε ακατάλληλα σημεία, το κόστος τους μπορεί να μειώσει το όφελος της πρόωρης εξόδου. Δεύτερον, τα thresholds δημιουργούν άμεσο συμβιβασμό ανάμεσα σε ακρίβεια και latency και συχνά χρειάζονται ρύθμιση ανά σύνολο δεδομένων, μοντέλο και περιβάλλον εκτέλεσης. Τρίτον, οι ενδιάμεσες εξόδους δεν βλέπουν τις βαθύτερες αναπαραστάσεις του δικτύου, άρα είναι πιο ευάλωτες σε δύσκολα ή αμφίσημα δείγματα. Τέλος, αν η εκπαίδευση δεν συντονίσει σωστά τις εξόδους, μπορεί να εμφανιστεί αστάθεια, υποεκπαίδευση ρηχών εξόδων ή υποβάθμιση της τελικής εξόδου [7], [16].

2.3 Edge Computing και Κατανεμημένα Edge Συστήματα

Τα early-exit νευρωνικά δίκτυα που παρουσιάστηκαν στην προηγούμενη ενότητα μειώνουν το μέσο υπολογιστικό κόστος προσαρμόζοντας το βάθος εκτέλεσης στη δυσκολία κάθε δείγματος. Ωστόσο, η πρακτική αξία μιας τέτοιας τεχνικής εξαρτάται και από το περιβάλλον στο οποίο εκτελείται το μοντέλο. Σε εφαρμογές όπως το Internet of Things (IoT), η ανάλυση βίντεο και οι διαδραστικές υπηρεσίες, η επεξεργασία συχνά πρέπει να γίνεται κοντά στην πηγή των δεδομένων, ώστε να μειώνεται το latency και να αποφεύγεται η συνεχής μεταφορά μεγάλου ή ευαίσθητου όγκου πληροφορίας προς απομακρυσμένα κέντρα δεδομένων.

Το edge computing απαντά σε αυτή την ανάγκη τοποθετώντας τους υπολογιστικούς πόρους κοντά στις τελικές συσκευές. Η βασική ιδέα είναι ότι μέρος της επεξεργασίας μπορεί να πραγματοποιείται στο ίδιο το τοπικό περιβάλλον: σε κινητά, αισθητήρες, κάμερες, ενσωματωμένες πλατφόρμες, gateways, single-board computers ή μικρά edge clusters. Με αυτόν τον τρόπο, η εκτέλεση μοντέλων τεχνητής νοημοσύνης μεταφέρεται από το αποκλειστικά cloud-based σχήμα σε ένα πιο κατανεμημένο υπολογιστικό παράδειγμα, γνωστό και ως edge intelligence [17].

2.3.1 Πόροι και Περιορισμοί στο Edge

Παρά τα πλεονεκτήματα της εγγύτητας στην πηγή των δεδομένων, οι edge κόμβοι λειτουργούν υπό σημαντικούς περιορισμούς. Οι τελικές συσκευές και οι κοντινοί workers διαθέτουν πεπερασμένη υπολογιστική ισχύ, μνήμη, ενεργειακό προϋπολογισμό και θερμική ανοχή. Επιπλέον, οι διαθέσιμοι πόροι συχνά μοιράζονται ανάμεσα σε περισσότερες εφαρμογές, ενώ το διαθέσιμο bandwidth και η καθυστέρηση επικοινωνίας μπορεί να μεταβάλλονται στον χρόνο.

Στο πλαίσιο του inference, οι περιορισμοί αυτοί μεταφράζονται σε συγκεκριμένες μετρικές συστήματος: latency ανά δείγμα, throughput, κατανάλωση ενέργειας, χρήση

μνήμης, φόρτο ανά worker και όγκο δεδομένων που μεταφέρεται μεταξύ κόμβων. Ένα μοντέλο που είναι αποδοτικό σε έναν ισχυρό edge server μπορεί να είναι ακατάλληλο για μια μικρότερη συσκευή, ενώ ένα πολύ ελαφρύ μοντέλο μπορεί να μην προσφέρει την επιθυμητή ακρίβεια. Επομένως, η αξιολόγηση ενός edge inference συστήματος πρέπει να εξετάζει από κοινού την ακρίβεια του μοντέλου και τη συμπεριφορά της υποδομής [9].

Η παρατήρηση αυτή είναι ιδιαίτερα σημαντική για δυναμικά μοντέλα, όπως τα early-exit DNNs. Σε αυτά, διαφορετικά δείγματα μπορεί να τερματίζουν σε διαφορετικά βάθη, άρα το φορτίο που φτάνει σε κάθε τμήμα του μοντέλου δεν είναι σταθερό. Το μέσο latency ή το μέσο κόστος μιας πλήρους εκτέλεσης του μοντέλου δεν επαρκούν για να περιγράψουν τη συμπεριφορά του συστήματος. Χρειάζεται να λαμβάνονται υπόψη η κατανομή των exits, ο χρόνος εκτέλεσης κάθε partition, το κόστος επικοινωνίας και η αξιοποίηση των διαθέσιμων workers.

2.3.2 Τοπική, Κατανεμημένη και Πολυμοντελική Εκτέλεση

Η πιο απλή στρατηγική είναι η τοπική εκτέλεση, όπου ολόκληρο το μοντέλο εκτελείται στη συσκευή που παράγει τα δεδομένα. Η προσέγγιση αυτή μειώνει τη δικτυακή εξάρτηση και μπορεί να περιορίσει τη μεταφορά ευαίσθητων δεδομένων εκτός συσκευής. Ωστόσο, όταν το μοντέλο είναι μεγάλο ή ο ρυθμός εισόδων υψηλός, η τοπική εκτέλεση μπορεί να οδηγήσει σε αυξημένο latency, κατανάλωση ενέργειας ή αδυναμία εκτέλεσης λόγω περιορισμένης μνήμης.

Στην κατανεμημένη edge εκτέλεση, το inference αξιοποιεί περισσότερους κόμβους του ίδιου τοπικού ή κοντινού περιβάλλοντος. Ένα μοντέλο μπορεί να εκτελείται εξ ολοκλήρου σε έναν ισχυρότερο worker ή να διασπάται σε διαδοχικά τμήματα που εκτελούνται σε διαφορετικούς workers. Έτσι δημιουργείται ένας χώρος σχεδιασμού στον οποίο η τοποθέτηση των partitions, η δρομολόγηση των ενδιάμεσων αναπαραστάσεων και η κατανομή φόρτου επηρεάζουν άμεσα το latency, το throughput, την κατανάλωση ενέργειας και το κόστος επικοινωνίας.

Στα πραγματικά edge περιβάλλοντα, η υποδομή σπάνια εξυπηρετεί μία μόνο ροή inference. Συχνά συνυπάρχουν πολλαπλές υπηρεσίες, διαφορετικά μοντέλα ή περισσότερα logical instances του ίδιου μοντέλου, τα οποία μοιράζονται τους ίδιους workers και ανταγωνίζονται για πόρους. Σε τέτοια πολυμοντελικά σενάρια, η απόδοση δεν εξαρτάται μόνο από την αρχιτεκτονική του DNN, αλλά και από το πώς κατανέμονται τα ενεργά partitions πάνω στην υποδομή. Η επανάληψη μιας διαδρομής που είναι αποδοτική για ένα μεμονωμένο μοντέλο μπορεί να οδηγήσει σε συμφόρηση όταν πολλές ροές χρησιμοποιούν τα ίδια αρχικά partitions.

Για τον λόγο αυτό, η εκτέλεση ενός early-exit μοντέλου σε edge υποδομή αποτελεί ταυτόχρονα πρόβλημα μηχανικής μάθησης και πρόβλημα κατανεμημένων συστημάτων. Η ακρίβεια καθορίζεται από το μοντέλο και τα κριτήρια εξόδου, ενώ η πραγματική αξία του συστήματος εξαρτάται από το πού εκτελείται κάθε τμήμα, πόσο κοστίζει η μεταφορά ενδιάμεσων δεδομένων και πώς κατανέμεται το φορτίο ανάμεσα στους διαθέσιμους workers. Αυτή η οπτική οδηγεί στο επόμενο τμήμα, όπου παρουσιάζονται οι

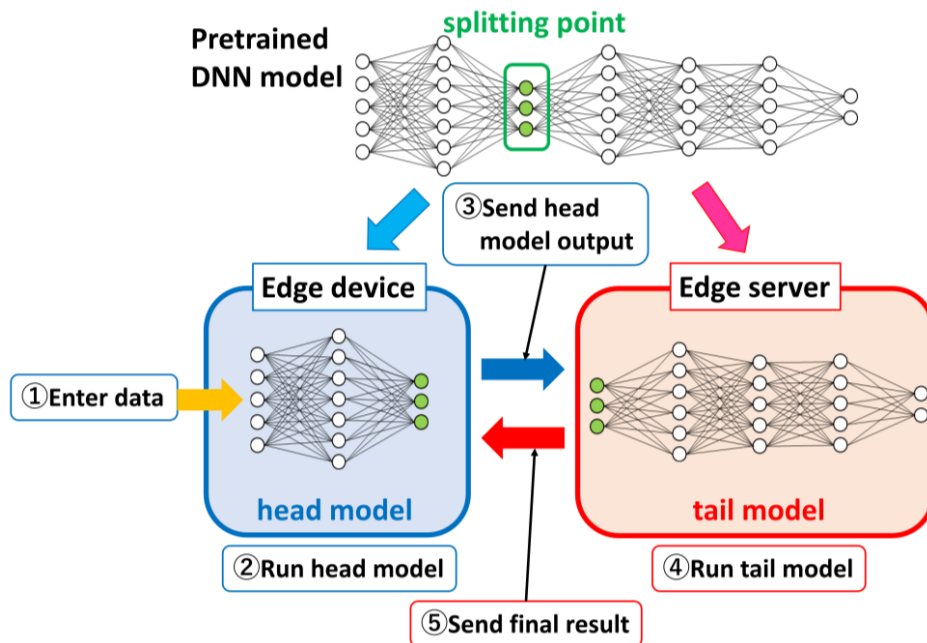
τεχνικές partitioning και split computing.

2.4 Partitioning και Split Computing σε Edge Workers

Το προηγούμενο section περιέγραψε την edge υποδομή ως σύνολο ετερογενών και περιορισμένων πόρων. Το επόμενο ερώτημα είναι πώς μπορεί ένα DNN να αξιοποιήσει πρακτικά αυτή την υποδομή. Η απλή επιλογή ενός μόνο σημείου εκτέλεσης δεν είναι πάντα επαρκής, επειδή διαφορετικά layers του μοντέλου, διαφορετικά δείγματα και διαφορετικές συνθήκες δικτύου έχουν διαφορετικό κόστος. Για τον λόγο αυτό, το partitioning και το split computing αφορούν το πώς ο υπολογισμός ενός μοντέλου μπορεί να καταναμηθεί ανάμεσα σε διαθέσιμους edge workers [8].

2.4.1 Βασικό Μοντέλο Split Computing

Στο split computing, ένα DNN χωρίζεται σε δύο ή περισσότερα διαδοχικά τμήματα. Το πρώτο τμήμα ονομάζεται συχνά *head* και περιλαμβάνει τα αρχικά layers του μοντέλου, ενώ το δεύτερο τμήμα ονομάζεται *tail* και περιλαμβάνει τα βαθύτερα layers και τον τελικό ταξινομητή. Αν το μοντέλο χωριστεί μετά από κάποιο layer L_k , τότε το head εκτελεί τα layers $L_1, \dots, L_k$, παράγει μια ενδιάμεση ενεργοποίηση h_k και η ενεργοποίηση αυτή μεταδίδεται στον worker που εκτελεί το tail. Η βασική διάσπαση head/tail φαίνεται στην Εικόνα 2.6.



Εικόνα 2.6: Παράδειγμα split computing, όπου ένα προεκπαιδευμένο DNN διαχωρίζεται σε head model και tail model. Το head εκτελείται στην edge device, η ενδιάμεση έξοδος μεταδίδεται στον edge server και το tail παράγει το τελικό αποτέλεσμα. Πηγή: [18].

Η επιλογή του split point είναι σημαντική, επειδή καθορίζει τον συμβιβασμό ανάμεσα σε υπολογισμό και επικοινωνία. Αν το split γίνει πολύ νωρίς, ο πρώτος worker εκτελεί μικρό μέρος του μοντέλου, αλλά μπορεί να χρειαστεί να μεταδώσει μεγάλο tensor. Αν γίνει πολύ αργά, μειώνεται η επικοινωνία προς τον επόμενο worker, αλλά αυξάνεται ο υπολογισμός και η κατανάλωση ενέργειας στο head. Επομένως, το split computing δεν είναι απλώς διαχωρισμός κώδικα σε στάδια, αλλά μηχανισμός επιλογής του σημείου στο οποίο συμφέρει να ανταλλαγεί τοπικός υπολογισμός με μετάδοση ενδιάμεσων χαρακτηριστικών [8].

Σε ένα πιο γενικό edge σύστημα, η εκτέλεση μπορεί να περιλαμβάνει περισσότερα από δύο τμήματα. Ένα μοντέλο μπορεί να χωριστεί σε ακολουθία από partitions, όπου κάθε partition εκτελείται σε διαφορετικό worker. Η βασική αρχή παραμένει ίδια: κάθε κόμβος λαμβάνει μια είσοδο, εκτελεί ένα συνεχόμενο τμήμα του DNN και προωθεί την ενδιάμεση αναπαράσταση στον επόμενο κόμβο.

2.4.2 Confidence-Based Routing

Στα early-exit DNNs, η απόφαση εκτέλεσης μετακινείται και στο επίπεδο του δείγματος. Αντί κάθε είσοδος να ακολουθεί πάντα την ίδια πλήρη διαδρομή, το σύστημα εκτελεί πρώτα ένα πρώιμο τμήμα και αποφασίζει αν η παραγόμενη πρόβλεψη είναι αρκετά αξιόπιστη. Αν ναι, η απάντηση επιστρέφεται από το τρέχον exit. Αν όχι, η ενδιάμεση αναπαράσταση προωθείται στο επόμενο partition.

Η απόφαση αυτή βασίζεται σε μετρικές αβεβαιότητας, όπως αυτές περιγράφηκαν στα κριτήρια τερματισμού των early-exit δικτύων. Η μέγιστη softmax πιθανότητα, το margin μεταξύ των δύο πιθανότερων κλάσεων και η εντροπία της εξόδου μπορούν να λειτουργήσουν ως ενδείξεις για το αν μια πρόβλεψη είναι αρκετά αξιόπιστη [6], [7]. Έτσι, η αβεβαιότητα δεν καθορίζει μόνο το βάθος εκτέλεσης του δείγματος, αλλά και τη ροή του μέσα στο κατανεμημένο pipeline.

Με αυτόν τον τρόπο, το confidence-based routing συνδέει την αβεβαιότητα του μοντέλου με το φορτίο της υποδομής. Μικρή αβεβαιότητα σημαίνει ότι το δείγμα μπορεί να ολοκληρωθεί νωρίς. Μεγάλη αβεβαιότητα σημαίνει ότι το σύστημα δαπανά επιπλέον υπολογισμό και επικοινωνία για να διατηρήσει την ακρίβεια. Ο συμβιβασμός αυτός είναι ανάλογος με το accuracy-latency trade-off των early exits, αλλά προσθέτει και μια τρίτη διάσταση: το επικοινωνιακό κόστος [7], [8].

2.4.3 Συνδυασμός Split Computing και Early-Exit Εκτέλεσης

Το split computing και τα early exits είναι συμπληρωματικές τεχνικές. Το split computing απαντά στο ερώτημα *πού* εκτελείται κάθε τμήμα του μοντέλου, ενώ η early-exit εκτέλεση απαντά στο ερώτημα *πόσο βαθιά* χρειάζεται να προχωρήσει κάθε δείγμα. Όταν συνδυάζονται, ένα εύκολο δείγμα μπορεί να τερματίσει στο head ή σε ένα πρώιμο partition, ενώ ένα δύσκολο δείγμα μπορεί να συνεχίσει προς το tail ή προς βαθύτερα partitions.

Η τοποθέτηση των exits επηρεάζει άμεσα τη συμπεριφορά του split συστήματος. Ένα exit πριν από το split point μπορεί να μειώσει τον αριθμό των δειγμάτων που μεταδίδονται στο tail. Ένα exit μετά από το split point δεν μειώνει την πρώτη μετάδοση, αλλά μπορεί να αποτρέψει την εκτέλεση ακόμη βαθύτερων layers ή την προώθηση σε επόμενο worker. Επομένως, τα exit points και τα split points δεν πρέπει να αντιμετωπίζονται ως ανεξάρτητες επιλογές.

Σε ένα τέτοιο σύστημα, το κόστος εκτέλεσης καθορίζεται από το πώς κατανέμονται τα δείγματα στα exits. Αν μεγάλο ποσοστό των εισόδων τερματίζει νωρίς, τότε μειώνεται ο υπολογισμός στα βαθύτερα partitions και περιορίζεται ο όγκος των δεδομένων που μεταφέρονται. Αν τα thresholds είναι αυστηρά και λίγα δείγματα τερματίζουν νωρίς, το σύστημα πλησιάζει τη συμπεριφορά ενός συμβατικού split DNN. Για τον λόγο αυτό, η αξιολόγηση τέτοιων συστημάτων πρέπει να περιλαμβάνει ακρίβεια, exit rates, latency και επικοινωνιακό φόρτο ανά worker.

2.4.4 Resource-Aware Τοποθέτηση

Σε πραγματικές edge υποδομές, η τοποθέτηση των partitions δεν μπορεί να θεωρείται ανεξάρτητη από τους διαθέσιμους πόρους. Κάθε worker έχει διαφορετική υπολογιστική ισχύ, μνήμη, ενεργειακό προφίλ και τρέχοντα φόρτο, ενώ κάθε σύνδεση έχει διαφορετικό latency και bandwidth. Επομένως, η απόφαση για το πού θα εκτελεστεί ένα τμήμα του DNN είναι πρόβλημα συστήματος και όχι μόνο αρχιτεκτονική επιλογή του μοντέλου.

Η resource-aware τοποθέτηση λαμβάνει υπόψη τόσο το υπολογιστικό κόστος όσο και το κόστος επικοινωνίας. Για κάθε πιθανό partition πρέπει να εκτιμηθεί ο χρόνος εκτέλεσης στον αντίστοιχο worker, το μέγεθος των ενδιάμεσων ενεργοποιήσεων που θα μεταδοθούν και η επίδραση στον φόρτο των επόμενων κόμβων. Στα multi-exit μοντέλα, η εικόνα γίνεται ακόμη πιο δυναμική, επειδή διαφορετικά δείγματα μπορούν να ακολουθήσουν διαφορετικά μονοπάτια μέσα στο ίδιο κατανεμημένο σύστημα [9].

Η resource-aware οπτική είναι ιδιαίτερα σημαντική για την παρούσα εργασία. Όταν ένα early-exit μοντέλο χωρίζεται σε δύο ή τρία partitions, η απόδοση δεν καθορίζεται μόνο από τον χρόνο εκτέλεσης κάθε τμήματος. Επηρεάζεται επίσης από το πόσα δείγματα σταματούν στο πρώτο partition, πόσα προωθούνται στο δεύτερο και πόσα τελικά φτάνουν στον τελικό classifier.

2.4.5 Προοδευτική Εκτέλεση

Οι παραπάνω τεχνικές οδηγούν σε μια πιο γενική ιδέα: η εκτέλεση ενός DNN σε edge περιβάλλον δεν χρειάζεται να είναι ίδια για κάθε δείγμα. Μπορεί να είναι προοδευτική, ξεκινώντας από φθηνότερα αρχικά στάδια και χρησιμοποιώντας βαθύτερα στάδια μόνο όταν η είσοδος το απαιτεί. Η προοδευτική αυτή εκτέλεση συνδυάζει τρία στοιχεία: ενδιάμεσες αναπαραστάσεις, κριτήρια εμπιστοσύνης και δυνατότητα συνέχισης σε επόμενο partition. Μια early-exit έξοδος ή μια routing decision είναι χρήσιμη μόνο όταν η

εκτίμηση εμπιστοσύνης συσχετίζεται επαρκώς με την πραγματική ορθότητα της πρόβλεψης.

Έτσι, το partitioning, το split computing και τα early exits σχηματίζουν ένα ενιαίο πλαίσιο προσαρμοστικής εκτέλεσης. Το μοντέλο δεν αντιμετωπίζεται ως ενιαίο υπολογιστικό βήμα που εκτελείται πάντα στον ίδιο κόμβο, αλλά ως ακολουθία σταδίων στα οποία η εκτέλεση μπορεί να τερματίζεται ή να συνεχίζεται ανάλογα με τη δυσκολία του δείγματος και την κατάσταση της υποδομής.

Η ίδια λογική γίνεται ακόμη πιο σημαντική σε πολυ-μοντελικά σενάρια, όπου περισσότερα μοντέλα ή logical instances μοιράζονται τους ίδιους edge workers. Σε αυτή την περίπτωση, δεν αρκεί να αποφασιστεί μόνο πού θα διασπαστεί κάθε μοντέλο, αλλά και σε ποιον worker θα τοποθετηθεί κάθε partition. Αν πολλά συχνά ενεργά partitions συγκεντρωθούν στον ίδιο κόμβο, μπορεί να δημιουργηθεί bottleneck σε υπολογισμό, μνήμη ή επικοινωνία, ακόμη και αν κάθε μεμονωμένο pipeline φαίνεται αποδοτικό. Άρα, το placement των partitions αποτελεί βασική παράμετρο για τη συνολική συμπεριφορά ενός multi-model edge inference συστήματος.

Αυτή η αρχή αποτελεί το άμεσο θεωρητικό υπόβαθρο για την πειραματική μελέτη της κατανομής και πολυ-μοντελικής εκτέλεσης early-exit μοντέλων.

Κεφάλαιο 3

Σχετική Βιβλιογραφία

Το προηγούμενο κεφάλαιο εισήγαγε τις έννοιες των DNNs, των early-exit μοντέλων, του edge computing και του split computing ως θεωρητικό υπόβαθρο. Στο κεφάλαιο αυτό δίνεται έμφαση στους ορισμούς της ερευνητικής βιβλιογραφίας: ποια προβλήματα προσπάθησαν να λύσουν οι υπάρχουσες εργασίες, ποιες υποθέσεις έκαναν για το μοντέλο και την υποδομή, ποιες μετρικές χρησιμοποίησαν και ποια κενά παραμένουν για την εκτέλεση early-exit μοντέλων σε καταναμημένα edge περιβάλλοντα.

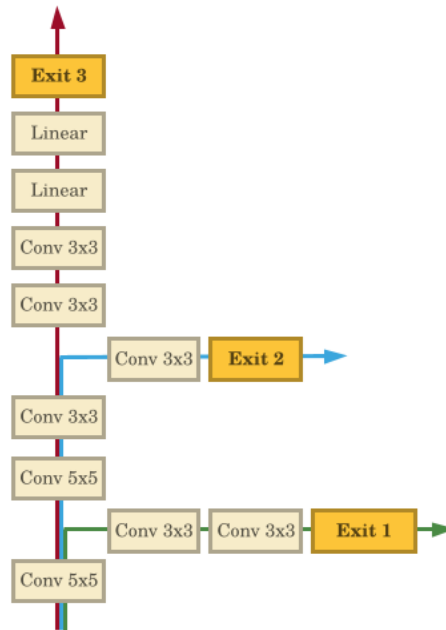
Η ανάγκη για αποδοτικό inference δεν προκύπτει μόνο από την αύξηση του μεγέθους των μοντέλων, αλλά και από την εξάπλωση της βαθιάς μάθησης σε εφαρμογές όπου τα δεδομένα παράγονται κοντά στον χρήστη ή σε φυσικά συστήματα: ιατρική απεικόνιση, ανίχνευση κακόβουλου λογισμικού, human activity recognition, περιβαλλοντική παρακολούθηση και ανάλυση βιοϊατρικών εικόνων [10], [11], [13], [12], [15], [14]. Σε τέτοιες εφαρμογές, η αποστολή όλων των δεδομένων εκτός του τοπικού edge περιβάλλοντος δεν είναι πάντα αποδεκτή, είτε λόγω καθυστέρησης και ενέργειας είτε λόγω ιδιωτικότητας και δικτυακού κόστους. Για αυτό, η σχετική βιβλιογραφία οργανώνεται εδώ γύρω από τέσσερα ερωτήματα: πότε μπορεί ένα δείγμα να τερματίσει νωρίς, πού πρέπει να εκτελεστεί κάθε partition, πώς συνδυάζονται early exits και run-time routing, και ποια κενά μένουν όταν η αξιολόγηση μεταφερθεί από το μοντέλο στο πραγματικό σύστημα.

3.1 Early-Exit Αρχιτεκτονικές και Adaptive Inference

Οι early-exit τεχνικές εξετάζουν πώς ένα νευρωνικό δίκτυο μπορεί να προσαρμόζει δυναμικά το βάθος εκτέλεσης ανάλογα με τη δυσκολία κάθε εισόδου. Αντί όλα τα δείγματα να διατρέχουν υποχρεωτικά ολόκληρο το μοντέλο, τοποθετούνται ενδιάμεσες έξοδοι που επιτρέπουν την έγκαιρη λήψη απόφασης όταν η πρόβλεψη είναι αρκετά αξιόπιστη. Με αυτόν τον τρόπο, η βιβλιογραφία των early-exit αρχιτεκτονικών συνδέει την ακρίβεια με το latency, την κατανάλωση ενέργειας και τη δυνατότητα εκτέλεσης σε περιορισμένους edge πόρους.

3.1.1 Multi-exit DNNs και δυναμικό βάθος εκτέλεσης

Η πιο χαρακτηριστική αφετηρία της σύγχρονης βιβλιογραφίας για early-exit DNNs είναι η εργασία που εισάγει το **BranchyNet** [6]. Η βασική ιδέα είναι η προσθήκη πλευρικών κλάδων ταξινόμησης σε υπάρχοντα δίκτυα, ώστε δείγματα με υψηλή εμπιστοσύνη να σταματούν πριν από την τελική έξοδο του μοντέλου. Με άλλα λόγια, τα εύκολα δείγματα δεν χρειάζονται πάντα τις βαθύτερες αναπαραστάσεις ενός DNN: αν μια ενδιάμεση έξοδος παράγει αρκετά σίγουρη πρόβλεψη, το δείγμα μπορεί να τερματίσει νωρίτερα και να αποφευχθεί η εκτέλεση των υπόλοιπων layers. Η αρχιτεκτονική αυτή φαίνεται στην Εικόνα 3.1.



Εικόνα 3.1: Παράδειγμα BranchyNet αρχιτεκτονικής με πλευρικούς κλάδους και πολλαπλές εξόδους. Οι ενδιάμεσοι classifiers επιτρέπουν σε εύκολα δείγματα να τερματίσουν πριν από την τελική έξοδο του δικτύου [6].

Το BranchyNet είναι σημαντικό επειδή δείχνει ότι το early-exit μπορεί να εφαρμοστεί σε γνωστές αρχιτεκτονικές, όπως LeNet, AlexNet και **ResNet**, χωρίς πλήρη επανασχεδιασμό του backbone. Η ιδέα αυτή είναι κεντρική και για την παρούσα εργασία, καθώς τα μοντέλα ResNet-18 και ResNet-34, που θα αξιοποιήσουμε, διατηρούν τη βασική residual αρχιτεκτονική τους και επεκτείνονται με πρόσθετες early-exit κεφαλές. Έτσι, το early-exit αντιμετωπίζεται ως επέκταση ενός υπάρχοντος backbone και όχι ως ανεξάρτητη αρχιτεκτονική από το μηδέν.

Παράλληλα, η εργασία εισάγει την από κοινού βελτιστοποίηση του backbone και των ενδιάμεσων exits, συνδυάζοντας τις επιμέρους απώλειες σε ένα σταθμισμένο άθροισμα (joint training). Ο μηχανισμός αυτός δεν χρησιμεύει μόνο για την εκπαίδευση των πλευρικών ταξινομητών. Επειδή κάθε ενδιάμεση έξοδος συνεισφέρει το δικό της training signal, τα ρηχότερα επίπεδα λαμβάνουν πιο άμεση πληροφορία gradient, ενώ η παρουσία πολλών εξόδων μπορεί να λειτουργήσει και ως μορφή regularization για το υπόλοιπο δίκτυο. Με αυτόν τον τρόπο, τα πρώιμα layers ενθαρρύνονται να παράγουν αναπαραστάσεις που βοηθούν καλύτερα στον διαχωρισμό των κλάσεων, γεγονός που

αυξάνει την πιθανότητα ένα δείγμα να μπορεί να ταξινομηθεί με βεβαιότητα πριν φτάσει στην τελική έξοδο. Η ίδια λογική ακολουθείται και στα multi-exit ResNet μοντέλα της παρούσας εργασίας, όπου κάθε ενδιάμεση κεφαλή και η τελική κεφαλή συνεισφέρουν στη συνολική cross-entropy loss με διαφορετικό βάρος.

Εξίσου σημαντική είναι η runtime πλευρά του BranchyNet. Κατά το inference, η απόφαση εξόδου λαμβάνεται σειριακά σε κάθε exit point με βάση την εντροπία της softmax πρόβλεψης: αν η εντροπία βρίσκεται κάτω από ένα threshold, το δείγμα τερματίζει, διαφορετικά συνεχίζει στα επόμενα layers. Συνεπώς, το threshold καθορίζει την κατανομή των εξόδων και άρα το πραγματικό μήκος εκτέλεσης κάθε δείγματος. Η ίδια entropy-based λογική χρησιμοποιείται και σε αυτή την εργασία.

Πειραματικά, η εργασία αναφέρει επιταχύνσεις περίπου $2\times$ έως $6\times$ σε CPU και GPU αντίστοιχα, δείχνοντας ότι η μείωση υπολογισμού μπορεί να μεταφραστεί σε πραγματικό χρόνο εκτέλεσης. Επιπλέον, τα αποτελέσματα δείχνουν ότι το όφελος εξαρτάται από το πόσα δείγματα εξέρχονται σε κάθε βήθος: για παράδειγμα, στο Branchy-ResNet ένα σημαντικό μέρος των δειγμάτων συνεχίζει μέχρι την τελική έξοδο, με αποτέλεσμα μικρότερη επιτάχυνση από ό,τι σε απλούστερα δίκτυα. Η παρατήρηση αυτή είναι κρίσιμη για edge περιβάλλοντα, επειδή το early-exit δεν μειώνει ομοιόμορφα τον φόρτο όλων των τμημάτων του δικτύου [6].

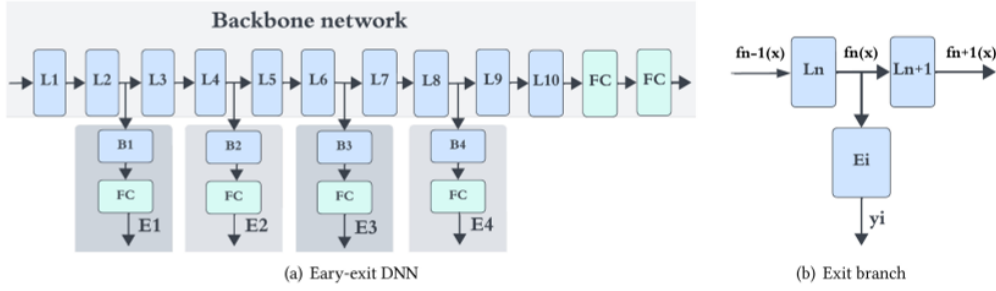
3.1.2 Από το BranchyNet στην ταξινόμηση των early-exit DNNs

Το BranchyNet άνοιξε τον δρόμο για μια ευρύτερη οικογένεια μοντέλων που στη βιβλιογραφία εμφανίζονται με συγγενείς όρους, όπως multi-exit networks, adaptive inference, dynamic inference, cascaded networks και early-termination networks. Οι πρόσφατες επισκοπήσεις ενοποιούν αυτές τις κατευθύνσεις κάτω από την ίδια βασική ιδέα: το DNN αποκτά ενδιάμεσες δυνατότητες πρόβλεψης και η εκτέλεση προσαρμόζεται στη δυσκολία της εκάστοτε εισόδου [7].

Μετά την αρχική διατύπωση του BranchyNet, το ερευνητικό ενδιαφέρον δεν περιορίζεται πλέον στο αν μπορούν να προστεθούν exits σε ένα δίκτυο, αλλά στο πώς σχεδιάζεται ένα αποτελεσματικό early-exit DNN. Η σχετική βιβλιογραφία οργανώνεται γύρω από τέσσερις βασικές αποφάσεις: την επιλογή του backbone, τη δομή των πλευρικών branches, τον αριθμό και την τοποθέτηση των exits, και την πολιτική τερματισμού στο inference [7]. Δεν υπάρχει μία καθολικά βέλτιστη επιλογή για τη θέση των πρώιμων εξόδων ή τη δομή των πλευρικών ταξινομητών. Η γενική μορφή ενός τέτοιου early-exit DNN φαίνεται στην Εικόνα 3.2.

3.1.3 Στρατηγικές εκπαίδευσης και αξιοπιστία ενδιάμεσων εξόδων

Η βιβλιογραφία διακρίνει έξι κύριες στρατηγικές εκπαίδευσης. Στο joint training, το backbone και όλες οι εξοδοί εκπαιδεύονται μαζί, μέσω σταθμισμένου αθροίσματος επιμέρους losses. Στο separate training, κάθε έξοδος αντιμετωπίζεται ως ξεχωριστό υπομοντέλο. Στο branch-wise training, τα exits εκπαιδεύονται σταδιακά, συχνά με πάγωμα



Εικόνα 3.2: Γενική μορφή early-exit DNN με ενδιαμέσους κλάδους πάνω σε backbone δίκτυο. Κάθε side branch προστίθεται σε διαφορετικό βάθος και επιτρέπει την παραγωγή πρόβλεψης πριν από την τελική έξοδο του μοντέλου [7].

προηγούμενων τμημάτων του δικτύου. Στο two-stage training, εκπαιδεύεται πρώτα το backbone και στη συνέχεια οι ενδιαμέσες κεφαλές πάνω σε παγωμένες αναπαραστάσεις. Οι distillation-based μέθοδοι αξιοποιούν τη γνώση βαθύτερων εξόδων ή teacher μοντέλων για να ενισχύσουν τις πρώιμες κεφαλές, ενώ οι hybrid στρατηγικές συνδυάζουν περισσότερες από μία από τις παραπάνω προσεγγίσεις [7], [16].

Η συγκριτική αξιολόγηση των στρατηγικών αυτών δείχνει ότι δεν υπάρχει καθολικά βέλτιστη επιλογή. Σε πειράματα με E-ResNet στο CIFAR-10, οι distillation-based μέθοδοι παρουσιάζουν τις ταχύτερες και υψηλότερες βελτιώσεις ακρίβειας στις ενδιαμέσες εξόδους, ενώ το joint training ακολουθεί κοντά, με ομαλή βελτίωση και περιορισμένο overfitting. Παράλληλα, το joint training και το two-stage training αναφέρονται ως οι πιο διαδεδομένες στρατηγικές στη βιβλιογραφία, ενώ οι branch-wise και distillation-based προσεγγίσεις παραμένουν λιγότερο διερευνημένες. Για τους σκοπούς της παρούσας εργασίας επιλέγεται το joint training, καθώς αποτελεί καθιερωμένη και σχετικά απλή διαδικασία εκπαίδευσης, ενώ αποφεύγει την πρόσθετη πολυπλοκότητα και το υπολογιστικό κόστος που εισάγουν οι distillation-based προσεγγίσεις [16].

Το κρίσιμο σημείο σε αυτή την εργασία δεν είναι η εξαντλητική σύγκριση των training strategies, αλλά η επίδρασή τους στη runtime συμπεριφορά. Μια early-exit έξοδος με χαμηλή ακρίβεια ή κακή εμπιστοσύνη θα τερματίζει λίγα δείγματα ή θα αυξάνει τον κίνδυνο λανθασμένων πρώιμων προβλέψεων. Αντίθετα, μια αξιόπιστη πρώιμη έξοδος μπορεί να μειώσει ουσιαστικά το πλήθος των δειγμάτων που συνεχίζουν στα βαθύτερα τμήματα του δικτύου. Έτσι, η εκπαίδευση των exits συνδέεται άμεσα με τον πραγματικό φόρτο εργασίας που θα παρατηρηθεί κατά το inference.

3.1.4 Πολιτικές εξόδου και adaptive inference

Αφού εκπαιδευτεί ένα early-exit DNN, η runtime συμπεριφορά του καθορίζεται από την πολιτική εξόδου. Η πολιτική αυτή αποφασίζει αν η πρόβλεψη μιας ενδιαμέσης κεφαλής είναι αρκετά αξιόπιστη ώστε το δείγμα να τερματίσει ή αν πρέπει να συνεχίσει προς βαθύτερα layers.

Οι πολιτικές εξόδου χωρίζονται συνήθως σε static και dynamic. Οι static ή rule-based πολιτικές χρησιμοποιούν προκαθορισμένα κριτήρια βεβαιότητας, όπως εντροπία,

μέγιστη softmax πιθανότητα, margin μεταξύ των επικρατέστερων κλάσεων ή κάποια custom scoring function. Για κάθε exit ορίζεται ένα threshold και το δείγμα τερματίζει όταν το αντίστοιχο κριτήριο ικανοποιείται. Οι dynamic ή learnable πολιτικές εισάγουν μηχανισμούς που μαθαίνουν την απόφαση εξόδου, όπως exit controllers, gating functions, reinforcement learning, multi-armed bandits ή κριτήρια συνέπειας μεταξύ διαδοχικών προβλέψεων [7].

Οι static πολιτικές έχουν το πλεονέκτημα ότι είναι απλές, ερμηνεύσιμες και έχουν μικρό runtime overhead. Για αυτό χρησιμοποιούνται συχνά όταν ο στόχος είναι να απομονωθεί η επίδραση του early-exit μηχανισμού από πιο σύνθετες αποφάσεις scheduling ή routing. Ωστόσο, η απλότητά τους συνοδεύεται από περιορισμούς: τα fixed thresholds μπορεί να μην προσαρμόζονται καλά σε αλλαγές της κατανομής των εισόδων, σε διαφορετικά datasets ή σε μεταβαλλόμενες συνθήκες εκτέλεσης. Οι dynamic πολιτικές μπορούν να αντιδράσουν καλύτερα σε τέτοιες μεταβολές, αλλά προσθέτουν επιπλέον πολυπλοκότητα στην εκπαίδευση και στο runtime.

Στην παρούσα εργασία χρησιμοποιείται entropy-based πολιτική εξόδου, ακολουθώντας την κλασική λογική του BranchyNet. Η επιλογή αυτή είναι σκόπιμη: επιτρέπει να διατηρηθεί σταθερός και απλός ο κανόνας τερματισμού, ώστε οι διαφορές ανάμεσα σε single-node και κατανεμημένη εκτέλεση να αποδοθούν κυρίως στον τρόπο εκτέλεσης του ίδιου early-exit pipeline και όχι σε διαφορετικές πολιτικές απόφασης.

3.2 Split Computing και Κατανεμημένη Edge Εκτέλεση DNNs

Η προηγούμενη ενότητα εστίασε στο πότε ένα δείγμα μπορεί να τερματίσει νωρίς μέσα σε ένα early-exit DNN. Το συμπληρωματικό ερώτημα είναι πού εκτελείται κάθε τμήμα του μοντέλου όταν το inference μοιράζεται σε περισσότερους από έναν κόμβους. Η βιβλιογραφία του edge intelligence τοποθετεί αυτό το πρόβλημα στο πλαίσιο της εκτέλεσης υπηρεσιών τεχνητής νοημοσύνης κοντά στο edge, με στόχο τη μείωση της καθυστέρησης και της περιττής μεταφοράς δεδομένων [17].

Για την παρούσα εργασία, το split computing είναι το συμπληρωματικό σκέλος των early exits: τα early exits καθορίζουν *πόσο βαθιά* εκτελείται κάθε δείγμα, ενώ το split computing εξετάζει *πού* εκτελούνται τα αντίστοιχα τμήματα του μοντέλου [8].

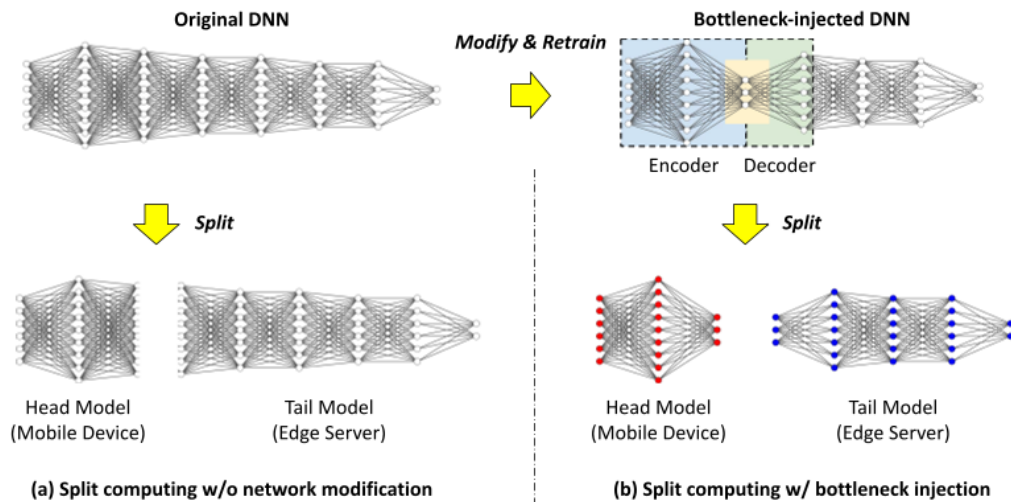
3.2.1 Split Inference και Κόστος Ενδιάμεσων Αναπαραστάσεων

Η βιβλιογραφία για split inference εξετάζει τον συμβιβασμό ανάμεσα στην πλήρη τοπική εκτέλεση και στην πλήρη αποστολή της εισόδου σε ισχυρότερο edge κόμβο. Το split computing εισάγει μια ενδιάμεση λύση: μέρος του υπολογισμού παραμένει κοντά στα δεδομένα, ενώ τα επόμενα τμήματα εκτελούνται σε άλλους workers. Το ενδιαφέρον, επομένως, δεν βρίσκεται στον απλό διαχωρισμό του DNN σε layers, αλλά στο αν το σημείο διαχωρισμού μειώνει πράγματι το συνολικό κόστος εκτέλεσης [8].

Κρίσιμος παράγοντας σε αυτή την απόφαση είναι το μέγεθος της ενδιάμεσης αναπαράστασης που μεταδίδεται ανάμεσα στους κόμβους. Ένα split point που φαίνεται ισορροπημένο ως προς τον αριθμό των layers μπορεί να παράγει μεγάλο intermediate tensor και να αυξάνει το communication/runtime overhead. Επιπλέον, οι ενδιάμεσες ενεργοποιήσεις δεν είναι πάντα μικρότερες από την αρχική είσοδο· σε αρκετές αρχιτεκτονικές, τα πρώτα convolutional layers αυξάνουν τον αριθμό των καναλιών και μπορούν να παράγουν tensors μεγαλύτερα από ένα ήδη συμπιεσμένο input. Για τον λόγο αυτό, η σχετική βιβλιογραφία αντιμετωπίζει την επιλογή split point ως κοινό πρόβλημα υπολογισμού, μνήμης και επικοινωνίας, και όχι ως καθαρά αρχιτεκτονική απόφαση [8].

3.2.2 Συμπιεσμένες Αναπαραστάσεις και Δυναμική Επιλογή Split Point

Μια βασική κατεύθυνση στη βιβλιογραφία είναι η μείωση του μεγέθους της ενδιάμεσης αναπαράστασης που μεταδίδεται στο δίκτυο. Ορισμένες προσεγγίσεις εισάγουν ρητά σημεία συμπίεσης ή μηχανισμούς compression, ώστε το tensor που αποστέλλεται στον επόμενο κόμβο να είναι πιο συμπαγές. Η προσέγγιση αυτή μπορεί να μειώσει σημαντικά το κόστος μετάδοσης, αλλά συχνά απαιτεί επανεκπαίδευση ή πρόσθετη βελτιστοποίηση, ενώ η τελική ακρίβεια μπορεί να επηρεαστεί από το πόσο επιθετική είναι η συμπίεση [8]. Η διαφορά ανάμεσα στον απλό διαχωρισμό και στην εισαγωγή bottleneck για συμπίεση της ενδιάμεσης αναπαράστασης φαίνεται στην Εικόνα 3.3.



Εικόνα 3.3: Δύο προσεγγίσεις split computing: απευθείας διαχωρισμός του αρχικού DNN και διαχωρισμός μετά από εισαγωγή bottleneck για τη συμπίεση της ενδιάμεσης αναπαράστασης που μεταδίδεται στο δίκτυο [8].

Μια παρεμφερής κατεύθυνση είναι η αξιοποίηση σημείων της αρχιτεκτονικής όπου οι ενδιάμεσες αναπαραστάσεις είναι ήδη συμπιεσμένες. Το Dynamic Split Computing δείχνει ότι ορισμένα layers παράγουν ενδιάμεσες αναπαραστάσεις μικρότερες από την αρχική είσοδο και μπορούν να χρησιμοποιηθούν ως υποψήφια split points χωρίς τροποποίηση του μοντέλου. Η επιλογή του split point μπορεί επίσης να γίνεται σε χρόνο εκτέλεσης, με βάση την κατάσταση του καναλιού επικοινωνίας και το batch size [19]. Έτσι, όταν το bandwidth είναι υψηλό, μπορεί να συμφέρει η μετάδοση ενός intermediate tensor σε edge server, ενώ σε χαμηλότερο bandwidth ή διαφορετικό batch size μπορεί να

προτιμάται άλλο σημείο διαχωρισμού ή ακόμη και η πλήρης τοπική εκτέλεση.

Παρότι η παρούσα εργασία δεν υλοποιεί δυναμική επιλογή split point, η βιβλιογραφία αυτή είναι χρήσιμη επειδή αναδεικνύει ότι η απόδοση ενός partitioned DNN είναι ιδιότητα ολόκληρου του συστήματος. Η ίδια τοποθέτηση μπορεί να είναι αποδοτική για μία κατάσταση δικτύου και λιγότερο αποδοτική για μια άλλη, γεγονός που ενισχύει την ανάγκη μέτρησης του πραγματικού runtime, της επικοινωνίας και της αξιοποίησης των workers.

3.2.3 Split Computing και Early Exits σε Edge Pipelines

Ο συνδυασμός split computing και early exits μετατρέπει το partitioned inference από στατική ακολουθία layers σε δυναμική ροή εργασίας. Το split computing καθορίζει σε ποιον worker εκτελείται κάθε partition, ενώ τα early exits καθορίζουν αν ένα δείγμα χρειάζεται να συνεχίσει σε επόμενα partitions. Έτσι, εύκολα δείγματα μπορούν να τερματίσουν νωρίς, μειώνοντας τον φόρτο των workers που εκτελούν τα επόμενα partitions, ενώ δυσκολότερα δείγματα συνεχίζουν βαθύτερα και καταναλώνουν περισσότερους υπολογιστικούς και δικτυακούς πόρους.

Η αλληλεπίδραση αυτή κάνει τις μετρικές συστήματος εξίσου σημαντικές με τις μετρικές μοντέλου. Ο αριθμός των αιτημάτων που φτάνει σε κάθε worker εξαρτάται από την κατανομή δυσκολίας των εισόδων, τα confidence thresholds και τα σημεία στα οποία έχει χωριστεί το backbone. Επομένως, η αξιολόγηση δεν μπορεί να περιοριστεί σε ακρίβεια ή θεωρητικά FLOPs, αλλά πρέπει να συνδυάζει exit distribution, worker-level compute time, communication/runtime overhead, throughput, latency και αξιοποίηση των διαθέσιμων workers.

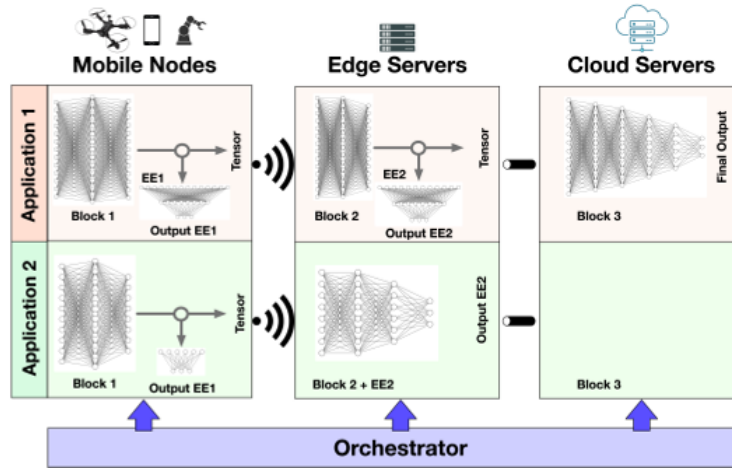
3.3 Resource-Aware Placement για Dynamic DNNs

Οι προηγούμενες ενότητες περιέγραψαν δύο βασικούς μηχανισμούς για αποδοτικό inference: τα early exits, που μειώνουν το βάθος εκτέλεσης ανά δείγμα, και το split computing, που μοιράζει τα layers ενός DNN σε περισσότερους κόμβους. Το επόμενο ερώτημα είναι πώς γίνεται η τοποθέτηση των τμημάτων του μοντέλου πάνω στην διαθέσιμη υποδομή. Σε ένα κατανεμημένο edge σύστημα, η καλύτερη λύση δεν προκύπτει μόνο από το ποιος κόμβος είναι υπολογιστικά ισχυρότερος. Εξαρτάται επίσης από το bandwidth, το communication/runtime overhead, τους περιορισμούς latency, τη διαθέσιμη ενέργεια και το ποσοστό των δειγμάτων που φτάνει σε κάθε partition.

3.3.1 To Deployment ως Constrained Optimization Πρόβλημα

Η εργασία [9] εξετάζει ακριβώς αυτό το πρόβλημα για dynamic DNNs με early exits. Το μοντέλο θεωρείται ως ακολουθία από blocks, όπου κάθε block περιλαμβάνει μέρος του

backbone και, ενδεχομένως, μία early-exit έξοδο. Ο orchestrator πρέπει να αποφασίσει ποια blocks θα εκτελεστούν και σε ποιους κόμβους θα τοποθετηθούν, λαμβάνοντας υπόψη απαιτήσεις εφαρμογής και περιορισμούς συστήματος. Η αντίστοιχη resource-aware ροή ανάθεσης φαίνεται στην Εικόνα 3.4.



Εικόνα 3.4: Resource-aware deployment δυναμικών DNNs με early exits σε multi-tier υποδομή. Ο orchestrator αναθέτει blocks του μοντέλου σε mobile, edge και cloud κόμβους, λαμβάνοντας υπόψη απαιτήσεις εφαρμογής και περιορισμούς συστήματος [9].

Η βασική διατύπωση της εργασίας είναι ενεργειακή: στόχος είναι η ελαχιστοποίηση της συνολικής ενέργειας inference. Η λύση όμως πρέπει ταυτόχρονα να ικανοποιεί στόχο ακρίβειας, άνω όριο latency, περιορισμούς υπολογιστικής ισχύος και περιορισμούς bandwidth. Έτσι, το deployment ενός early-exit DNN δεν αντιμετωπίζεται ως απλή επιλογή split point, αλλά ως constrained optimization πρόβλημα πάνω σε διασυνδεδεμένους κόμβους.

Το στοιχείο που κάνει τα early-exit μοντέλα ιδιαίτερα ενδιαφέροντα είναι ότι η ροή δειγμάτων δεν είναι σταθερή σε όλα τα blocks. Κάθε exit μπορεί να τερματίσει ένα ποσοστό εισόδων, μειώνοντας τον φόρτο των επόμενων τμημάτων. Επομένως, το κόστος ενός placement δεν εξαρτάται μόνο από το πόσο ακριβό είναι ένα block, αλλά και από το πόσο συχνά αναμένεται να εκτελεστεί.

3.3.2 FIN και Σύνδεση με την Παρούσα Εργασία

Για την επίλυση του παραπάνω προβλήματος, προτείνεται το Feasible Inference Graph (FIN) [9]. Η ιδέα του FIN είναι να μετατρέψει το deployment σε πρόβλημα γράφου. Οι κόμβοι του γράφου αναπαριστούν δυνατές αναθέσεις DNN blocks σε υπολογιστικούς κόμβους, ενώ τα μονοπάτια αντιστοιχούν σε εφικτές διαμορφώσεις εκτέλεσης. Οι περιορισμοί latency, accuracy, compute capacity και bandwidth ενσωματώνονται στον γράφο, έτσι ώστε η αναζήτηση ενός minimum-cost path να δίνει μια ενεργειακά αποδοτική ανάθεση. Η δυσκολία δεν βρίσκεται στο απλό minimum-cost path σε έναν στατικό γράφο, αλλά στη διαμόρφωση του εφικτού χώρου αναθέσεων υπό πολλαπλούς περιορισμούς. Στη γενική του μορφή, το resource-constrained deployment είναι NP-hard, καθώς συνδυάζει διακριτές αποφάσεις τοποθέτησης blocks με περιορισμούς πόρων, επικοινωνίας,

latency και ακρίβειας.

Τα αποτελέσματα της εργασίας δείχνουν ότι η resource-aware τοποθέτηση μπορεί να μειώσει σημαντικά την κατανάλωση ενέργειας, ενώ παράλληλα ικανοποιεί περιορισμούς ακρίβειας και latency. Ιδιαίτερα σημαντικό είναι ότι το FIN αξιοποιεί τα early exits ως μηχανισμό που μειώνει την ανάγκη εκτέλεσης βαθύτερων blocks και, συνεπώς, την εμπλοκή ακριβότερων κόμβων του συστήματος. Με αυτόν τον τρόπο, η εργασία δείχνει ότι τα dynamic DNNs πρέπει να μελετώνται ως μέρος ενός runtime συστήματος και όχι μόνο ως αρχιτεκτονικές μοντέλων.

Η παρούσα διπλωματική δεν προτείνει νέο αλγόριθμο βελτιστοποίησης placement τύπου FIN. Αντίθετα, αξιοποιεί τη συστημική αυτή οπτική για να μελετήσει πειραματικά πώς συμπεριφέρονται early-exit ResNet-18 και ResNet-34 μοντέλα όταν εκτελούνται σε περιορισμένο edge cluster, και πώς η τοποθέτηση των αντίστοιχων partitions επηρεάζει τη συμπεριφορά του συστήματος.

3.4 Σύνθεση Βιβλιογραφίας και Θέση της Παρούσας Εργασίας

Οι προηγούμενες ενότητες ανέδειξαν τρεις συμπληρωματικές κατευθύνσεις της σχετικής βιβλιογραφίας. Τα early-exit DNNs μειώνουν το μέσο βάθος εκτέλεσης, επιτρέποντας σε εύκολα δείγματα να τερματίζουν πριν από την τελική έξοδο. Το split computing εξετάζει πώς ένα DNN μπορεί να διαχωριστεί σε τμήματα και να εκτελεστεί σε περισσότερους από έναν κόμβους. Οι resource-aware μέθοδοι placement μελετούν πώς αυτά τα τμήματα μπορούν να τοποθετηθούν σε διαθέσιμη υποδομή, λαμβάνοντας υπόψη περιορισμούς σε latency, ενέργεια, bandwidth και υπολογιστική ισχύ [8], [19], [9].

Παρά τη σημαντική πρόοδο σε κάθε μία από αυτές τις κατευθύνσεις, παραμένει ένα κενό όταν το ενδιαφέρον μεταφέρεται από το ίδιο το μοντέλο στο πραγματικό σύστημα εκτέλεσης. Πολλές εργασίες αξιολογούν early-exit αρχιτεκτονικές κυρίως με μετρικές που εστιάζουν στο μοντέλο, όπως η ακρίβεια, τα exit rates ή η θεωρητική μείωση των FLOPs. Οι μετρικές αυτές είναι απαραίτητες, αλλά δεν περιγράφουν πλήρως τη συμπεριφορά ενός καταναμιμένου runtime. Όταν το μοντέλο εκτελείται ως pipeline πάνω σε edge workers, χρειάζεται να αξιολογούνται ταυτόχρονα latency, throughput, ενεργειακή κατανάλωση, communication/runtime overhead, worker-level compute time και αξιοποίηση των κόμβων.

Επιπλέον, αρκετές υπάρχουσες αξιολογήσεις εξετάζουν μία εφαρμογή ή ένα μοντέλο κάθε φορά. Σε πραγματικά edge περιβάλλοντα, όμως, οι ίδιοι workers μπορεί να εξυπηρετούν πολλαπλά μοντέλα ή πολλαπλές ροές αιτημάτων που ανταγωνίζονται για τους ίδιους πόρους. Η βιβλιογραφία αναγνωρίζει τη σημασία του multi-application και multi-tier deployment [9], ωστόσο η πειραματική μελέτη του πώς συμπεριφέρονται early-exit pipelines σε πολυ-μοντελικά edge σενάρια παραμένει περιορισμένη.

Η παρούσα διπλωματική τοποθετείται σε αυτό το σημείο. Δεν προτείνει νέα early-exit αρχιτεκτονική ούτε νέο θεωρητικό αλγόριθμο placement. Αντίθετα, μελετά την εκτέλεση early-exit ResNet-18 και ResNet-34 μοντέλων ως πραγματικά partitioned edge

pipelines. Η αξιολόγηση καλύπτει single-node εκτέλεση, κατανεμημένη εκτέλεση σε τρεις workers, αφελή multi-model εκτέλεση δύο logical model instances και memory-aware placement στρατηγικές. Με αυτόν τον τρόπο, η έμφαση μεταφέρεται από το αν τα early exits μειώνουν θεωρητικά τον υπολογισμό στο πώς αυτή η μείωση μεταφράζεται σε μετρούμενη συμπεριφορά συστήματος.

Κεφάλαιο 4

Μεθοδολογία και Πειραματικό Πλαίσιο

Το παρόν κεφάλαιο περιγράφει τη μεθοδολογία που ακολουθήθηκε για την πειραματική αξιολόγηση της συντονισμένης εκτέλεσης μοντέλων βαθιάς μάθησης με early-exit σε κατανεμημένα edge περιβάλλοντα. Σε αντίθεση με το προηγούμενο κεφάλαιο, το οποίο εστίασε στη βιβλιογραφία, εδώ η έμφαση μεταφέρεται στο πειραματικό πρωτόκολλο: στα CIFAR-10 και CIFAR-10.1 datasets, στα ResNet-18 και ResNet-34 μοντέλα που χρησιμοποιήθηκαν, στην υποδομή εκτέλεσης, στις τοπολογίες single-node, distributed και multi-model και στις μετρικές αξιολόγησης.

4.1 Σκοπός και Μεθοδολογική Προσέγγιση

Σκοπός της πειραματικής μελέτης είναι να αξιολογηθεί η εκτέλεση early-exit βαθιών νευρωνικών δικτύων ως πραγματικού φόρτου εργασίας σε περιορισμένο edge cluster. Η εργασία δεν προτείνει νέα early-exit αρχιτεκτονική ούτε νέο αλγόριθμο εκπαίδευσης. Αντίθετα, χρησιμοποιεί ResNet-18 και ResNet-34 μοντέλα, σε baseline και early-exit μορφή, και εξετάζει πώς μεταβάλλεται η συμπεριφορά τους όταν αλλάζει ο τρόπος εκτέλεσης πάνω στην ίδια υποδομή.

Η αξιολόγηση οργανώνεται κλιμακωτά. Αρχικά μελετάται η single-node εκτέλεση, ώστε να απομονωθεί η επίδραση των early exits σε σχέση με τα αντίστοιχα baseline μοντέλα. Στη συνέχεια, το ίδιο early-exit pipeline διασπάται σε τρία partitions και εκτελείται σε τρεις edge workers, ώστε να μετρηθεί το κόστος της κατανεμημένης εκτέλεσης. Τέλος, η μελέτη επεκτείνεται σε multi-model σενάρια, όπου δύο logical model instances μοιράζονται τους ίδιους workers. Σε αυτό το πλαίσιο συγκρίνεται μια αφελής τοποθέτηση, όπου και τα δύο instances ακολουθούν την ίδια pipeline διαδρομή, με memory-aware στρατηγικές που αλλάζουν την αντιστοίχιση των partitions στους workers.

Η βασική μεθοδολογία είναι να διατηρούνται σταθερά το εκπαιδευμένο μοντέλο, η πολιτική early-exit και το dataset μέσα σε κάθε ομάδα συγκρίσεων. Έτσι, οι διαφορές που παρατηρούνται στις μετρικές μπορούν να αποδοθούν κυρίως στην τοπολογία εκτέλεσης και στο partition placement, και όχι σε αλλαγές της αρχιτεκτονικής ή της εκπαίδευσης. Για τον λόγο αυτό, η αξιολόγηση δεν περιορίζεται στην ακρίβεια, αλλά περιλαμβάνει latency, throughput, exit distribution, communication runtime overhead, worker utilization και ενεργειακή κατανάλωση.

4.2 Λογισμικό και Βιβλιοθήκες Υλοποίησης

Η πειραματική υλοποίηση βασίζεται σε Python και οργανώνεται ως ενιαίο σύστημα εκτέλεσης για single-node και καταναμημένο inference. Σε κάθε πείραμα καθορίζονται με κοινό τρόπο τρεις ομάδες παραμέτρων: το σύνολο δεδομένων, η αρχιτεκτονική του μοντέλου και η τοπολογία εκτέλεσης.

Βιβλιοθήκη	Ρόλος στην υλοποίηση
PyTorch	Ορισμός, εκπαίδευση και εκτέλεση των ResNet και early-exit ResNet μοντέλων σε single-node και partitioned μορφή.
TorchVision / Pillow	Φόρτωση των CIFAR datasets, εφαρμογή της προεπεξεργασίας και συμβατή διαχείριση των εικόνων CIFAR-10.1.
FastAPI / Uvicorn	Υλοποίηση του HTTP-based distributed runtime στους workers και έκθεση των βασικών υπηρεσιών inference και monitoring.
Pydantic	Ορισμός και έλεγχος των δομών δεδομένων που ανταλλάσσονται στο distributed API.
python-multipart	Υποστήριξη μεταφοράς μεταδεδομένων και tensor payloads στο ίδιο inference request.
Requests	Αποστολή HTTP κλήσεων μεταξύ master και workers, καθώς και μεταξύ διαδοχικών workers.
NumPy	Αριθμητικοί υπολογισμοί, επεξεργασία tensor payloads και υπολογισμός latency και communication/runtime μετρικών.
Pandas	Οργάνωση των per-task αποτελεσμάτων και παραγωγή συγκεντρωτικών πινάκων για την ανάλυση.
Matplotlib	Παραγωγή των γραφημάτων αξιολόγησης από τα συγκεντρωτικά αποτελέσματα των πειραμάτων.
CodeCarbon	Εκτίμηση ενεργειακής κατανάλωσης και εκπομπών για single-node και distributed runs.
PyYAML	Ανάγνωση των configuration files και σύνθεση των ρυθμίσεων κάθε πειραματικού run.
tqdm	Παρακολούθηση της προόδου κατά την εκπαίδευση, το warmup και τα inference loops.

Πίνακας 4.2: Βασικές βιβλιοθήκες και ρόλος τους στην πειραματική υλοποίηση.

4.3 Πειραματική Υποδομή

Η πειραματική αξιολόγηση πραγματοποιήθηκε σε ένα τοπικό edge cluster τεσσάρων Raspberry Pi 4 Model B συσκευών. Η μία συσκευή λειτουργεί ως master node, ενώ οι υπόλοιπες τρεις λειτουργούν ως workers που εκτελούν τα layers του μοντέλου. Όλοι οι κόμβοι συνδέονται στο ίδιο τοπικό δίκτυο μέσω ενσύρματης σύνδεσης Ethernet. Με αυτόν τον τρόπο, η επικοινωνία master-worker και worker-worker πραγματοποιείται μέσα στο ίδιο LAN, χωρίς συμμετοχή απομακρυσμένης υποδομής.

Οι τέσσερις συσκευές είναι ομογενείς ως προς το υλικό: όλες διαθέτουν

Ρόλος	Hostname	IP	Χαρακτηριστικά
Master	raspberrypi-master	192.168.0.100	Raspberry Pi 4 Model B Rev 1.1, 4-core ARM Cortex-A72 έως 1.5 GHz, 3.7 GiB RAM, Debian 12, kernel 6.6.74.
Worker 1	raspberrypi-worker-1	192.168.0.102	Raspberry Pi 4 Model B Rev 1.1, 4-core ARM Cortex-A72 έως 1.5 GHz, 3.7 GiB RAM, Debian 12, kernel 6.6.74.
Worker 2	raspberrypi-worker-2	192.168.0.104	Raspberry Pi 4 Model B Rev 1.1, 4-core ARM Cortex-A72 έως 1.5 GHz, 3.7 GiB RAM, Debian 12, kernel 6.6.74.
Worker 3	raspberrypi-worker-3	192.168.0.103	Raspberry Pi 4 Model B Rev 1.1, 4-core ARM Cortex-A72 έως 1.5 GHz, 3.7 GiB RAM, Debian 11, kernel 6.1.21.

Πίνακας 4.4: Υλικό και βασικά χαρακτηριστικά των κόμβων της πειραματικής υποδομής.

τετραπύρρηγο επεξεργαστή ARM Cortex-A72, αρχιτεκτονική aarch64 και περίπου 4 GB μνήμης. Η βασική διαφορά εντοπίζεται στο λογισμικό του τρίτου worker, ο οποίος εκτελεί Debian 11 και παλαιότερο kernel σε σχέση με τους υπόλοιπους κόμβους. Η διαφορά αυτή δεν θεωρείται κεντρικός παράγοντας στην ερμηνεία των αποτελεσμάτων, επειδή το cluster είναι ομογενές ως προς το hardware και όλοι οι workers τρέχουν το ίδιο virtual environment και το ίδιο πειραματικό runtime.

4.4 Σύνολα Δεδομένων και Προεπεξεργασία

Τα αποθηκευμένα πειραματικά αποτελέσματα που χρησιμοποιούνται στην παρούσα αξιολόγηση αφορούν δύο συγγενικά σύνολα δεδομένων: το CIFAR-10 και το CIFAR-10.1. Το CIFAR-10 αποτελεί ελεγχόμενο benchmark ταξινόμησης εικόνων με 10 κλάσεις και test split 10.000 εικόνων [20]. Η επιλογή του επιτρέπει την εκτέλεση ολόκληρου του test split σε περιορισμένες Raspberry Pi συσκευές, διατηρώντας ταυτόχρονα αρκετή δυσκολία ώστε να εμφανίζονται διαφορετικές αποφάσεις early-exit.

Το CIFAR-10.1 είναι ανεξάρτητο test set για το CIFAR-10, με τις ίδιες 10 κλάσεις και το ίδιο είδος εικόνων χαμηλής ανάλυσης, το οποίο δημιουργήθηκε ώστε να είναι όσο γίνεται κοντά στην αρχική κατανομή του CIFAR-10 αλλά να περιέχει πραγματικά μη χρησιμοποιημένες εικόνες [21]. Τα αρχεία του συνόλου δεδομένων διατίθενται στο δημόσιο αποθετήριο CIFAR-10.1 [22]. Στην έκδοση που χρησιμοποιείται στην παρούσα εργασία περιλαμβάνει 2.000 εικόνες. Δεν χρησιμοποιείται για νέα εκπαίδευση ή fine-tuning των μοντέλων. Αντίθετα, τα ίδια εκπαιδευμένα βάρη, η ίδια πολιτική early-exit και οι ίδιες τοπολογίες εκτέλεσης εφαρμόζονται και στο CIFAR-10.1, ώστε να εξεταστεί αν τα συμπεράσματα του CIFAR-10 διατηρούνται και σε ένα ανεξάρτητο αλλά συμβατό test set.

Η χρήση του CIFAR-10.1 είναι σημαντική για δύο λόγους. Πρώτον, μειώνει τον κίνδυνο τα συμπεράσματα να εξαρτώνται υπερβολικά από το πολύ γνωστό και επαναχρησιμοποιημένο test split του CIFAR-10. Δεύτερον, επιτρέπει να μελετηθεί αν

μικρές φυσικές μεταβολές στην κατανομή των εικόνων αλλάζουν την ακρίβεια, την κατανομή των exits και, κατ' επέκταση, τον χρόνο εκτέλεσης και τον φόρτο των επόμενων partitions. Επομένως, το CIFAR-10 λειτουργεί ως βασικό benchmark, ενώ το CIFAR-10.1 ως έλεγχος γενίκευσης της ίδιας πειραματικής μεθοδολογίας.

4.4.1 Προεπεξεργασία Εισόδου

Για το inference χρησιμοποιείται σταθερή προεπεξεργασία σε όλα τα σενάρια και στα δύο datasets. Κάθε εικόνα αλλάζει μέγεθος σε 256×256 , μετατρέπεται σε tensor και κανονικοποιείται με τις τιμές του CIFAR-10:

$$\mu = [0.4914, 0.4822, 0.4465], \quad \sigma = [0.2023, 0.1994, 0.2010].$$

Η ίδια κανονικοποίηση χρησιμοποιείται και στο CIFAR-10.1, επειδή τα μοντέλα έχουν εκπαιδευτεί στο CIFAR-10 και το CIFAR-10.1 έχει σχεδιαστεί ως συμβατό test set για τις ίδιες κλάσεις. Με αυτόν τον τρόπο η αλλαγή dataset δεν εισάγει διαφορετική προεπεξεργασία και η σύγκριση απομονώνει τη μεταβολή των δειγμάτων εισόδου.

Η εκτέλεση του test split γίνεται με batch size ίσο με 1, χωρίς ανακάτεμα της σειράς των δειγμάτων και χωρίς παράλληλους dataloader workers. Η επιλογή batch size ίσου με 1 είναι κρίσιμη για τα early-exit και distributed experiments, επειδή η απόφαση εξόδου λαμβάνεται ανά δείγμα και το κατανεμημένο pipeline προωθεί μία ενδιάμεση αναπαράσταση κάθε φορά.

4.4.2 Warmup και Πειραματική Συνέπεια

Πριν από τη μετρούμενη εκτέλεση κάθε run χρησιμοποιεί 20 δείγματα προθέρμανσης. Τα δείγματα αυτά ακολουθούν το ίδιο inference path με τα κανονικά δείγματα, αλλά δεν προσμετρώνται στις τελικές μετρικές. Με αυτόν τον τρόπο μειώνεται η επίδραση αρχικών runtime effects, όπως lazy initialization, caching και πρώτη επικοινωνία με τους workers. Όλες οι συγκρίσεις γίνονται με την ίδια σειρά δεδομένων ανά dataset, την ίδια προεπεξεργασία και τα αντίστοιχα αποθηκευμένα βάρη κάθε μοντέλου.

4.5 Αρχιτεκτονικές ResNet και Early-Exit Επέκταση

Η πειραματική μελέτη χρησιμοποιεί δύο ResNet αρχιτεκτονικές: ResNet-18 και ResNet-34. Και οι δύο βασίζονται σε residual blocks και διατηρούν την ίδια μακροσκοπική δομή τεσσάρων residual layers [4]. Η διαφορά τους είναι το βάθος κάθε layer: το ResNet-18 χρησιμοποιεί διάταξη blocks $[2, 2, 2, 2]$, ενώ το ResNet-34 χρησιμοποιεί $[3, 4, 6, 3]$. Με αυτόν τον τρόπο, η εργασία μελετά τόσο ένα μικρότερο όσο και ένα βαθύτερο CNN μέσα στο ίδιο πειραματικό πλαίσιο.

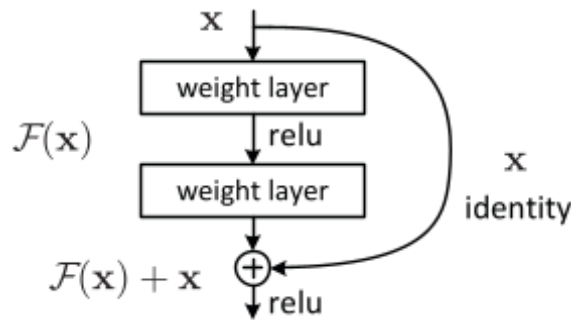
4.5.1 Residual Blocks

Η βασική ιδέα των ResNet αρχιτεκτονικών είναι ότι ένα βαθύ δίκτυο δεν χρειάζεται να μάθει από την αρχή ολόκληρο τον μετασχηματισμό $H(x)$ που αντιστοιχεί σε ένα σύνολο διαδοχικών επιπέδων. Αντί για αυτό, κάθε residual block μαθαίνει κυρίως τη διαφορά ανάμεσα στην είσοδο και στην επιθυμητή έξοδο. Η διαφορά αυτή γράφεται ως $F(x) = H(x) - x$, ενώ η έξοδος του block υπολογίζεται ως

$$y = F(x) + x.$$

Ο όρος x περνά από μια shortcut σύνδεση, παρακάμπτει τα συνελικτικά επίπεδα και προστίθεται στο αποτέλεσμα τους. Με αυτόν τον τρόπο, αν το block χρειάζεται να κάνει μόνο μια μικρή αλλαγή στην είσοδο, δεν είναι απαραίτητο να μάθει ξανά ολόκληρη την ταυτοτική απεικόνιση. Αρκεί να μάθει τη μικρή διόρθωση $F(x)$. Η shortcut σύνδεση δημιουργεί επίσης ένα πιο άμεσο μονοπάτι για τη διάδοση του gradient προς τα προηγούμενα επίπεδα. Έτσι, το σήμα του gradient μπορεί να επιστρέφει πιο εύκολα στα αρχικά layers, κάτι που διευκολύνει την εκπαίδευση δικτύων με πολύ μεγάλο αριθμό επιπέδων.

Στο ResNet paper, η residual διατύπωση παρουσιάζεται κυρίως ως τρόπος αντιμετώπισης του degradation problem: όταν αυξάνεται απλώς το βάθος ενός plain δικτύου, το training error μπορεί να χειροτερεύσει, χωρίς αυτό να οφείλεται απαραίτητα σε overfitting. Οι residual συνδέσεις κάνουν ευκολότερη την εκπαίδευση βαθύτερων μοντέλων, επειδή κάθε block μπορεί να μάθει μια διόρθωση πάνω στην είσοδό του αντί να μάθει από την αρχή ολόκληρο τον μετασχηματισμό [4]. Η δομή ενός τέτοιου residual block φαίνεται στην Εικόνα 4.1.



Εικόνα 4.1: Residual learning block με shortcut σύνδεση. Τα συνελικτικά επίπεδα υπολογίζουν το $F(x)$, ενώ η είσοδος x προστίθεται στην έξοδο του block μέσω identity mapping [4].

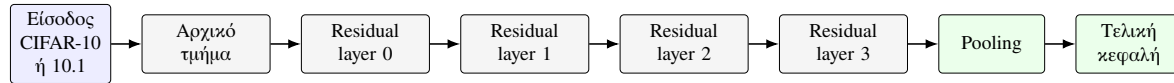
Στην απλούστερη περίπτωση η shortcut σύνδεση είναι identity mapping, δηλαδή μεταφέρει την είσοδο χωρίς αλλαγές. Επομένως, δεν προσθέτει νέες παραμέτρους ούτε σημαντικό υπολογιστικό κόστος. Όταν όμως το block αλλάζει τις διαστάσεις της αναπαράστασης, για παράδειγμα επειδή διπλασιάζει τα κανάλια ή μειώνει τη χωρική διάσταση, η shortcut διαδρομή πρέπει πρώτα να προσαρμόσει την είσοδο ώστε να μπορεί να προστεθεί στο $F(x)$. Στις ResNet υλοποιήσεις αυτό γίνεται συνήθως με projection shortcut, όπως ένα 1×1 convolution με κατάλληλο stride. Έτσι η ίδια residual λογική διατηρείται και όταν το μοντέλο περνά από ένα residual stage στο επόμενο.

Στο πλαίσιο της παρούσας εργασίας, τα δύο μοντέλα χρησιμοποιούν basic residual

blocks με δύο 3×3 συνελίξεις ανά block.

4.5.2 Baseline Μοντέλα

Τα baseline μοντέλα εκτελούν πάντα ολόκληρο το δίκτυο πριν παράγουν πρόβλεψη. Η ροή τους φαίνεται στην Εικόνα 4.2.

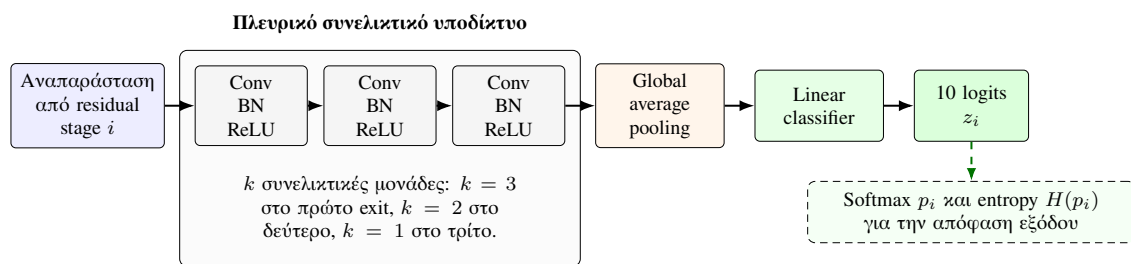


Εικόνα 4.2: Ροή inference των baseline ResNet μοντέλων χωρίς early-exit κεφαλές.

Η τελική γραμμική κεφαλή παράγει 10 logits για τις κοινές κλάσεις των CIFAR-10 και CIFAR-10.1. Τα baseline μοντέλα χρησιμοποιούνται ως σημείο αναφοράς για να αποτιμηθεί πόσο αλλάζει η ακρίβεια, το latency και το throughput όταν προστεθούν early exits.

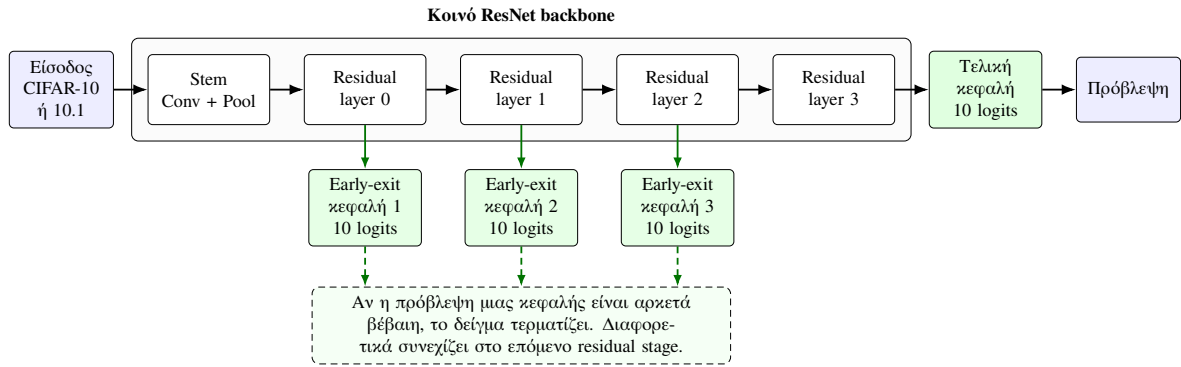
4.5.3 Early-Exit Μοντέλα

Οι early-exit εκδοχές των ResNet-18 και ResNet-34 διατηρούν το ίδιο backbone με τα αντίστοιχα baseline μοντέλα και προσθέτουν τρεις ενδιάμεσες ταξινομητικές κεφαλές. Κάθε ενδιάμεση έξοδος δέχεται την αναπαράσταση που έχει παραχθεί μέχρι το αντίστοιχο residual stage και την επεξεργάζεται με ένα μικρό πλευρικό συνελκτικό υποδίκτυο. Το υποδίκτυο αυτό αποτελείται από διαδοχικά συνελκτικά βήματα με κανονικοποίηση και μη γραμμική ενεργοποίηση, ακολουθούμενα από global average pooling και έναν τελικό γραμμικό ταξινομητή 10 κλάσεων. Η εσωτερική ροή μίας τέτοιας κεφαλής φαίνεται στο Εικόνα 4.3.



Εικόνα 4.3: Δομή μίας early-exit κεφαλής. Η αναπαράσταση που παράγεται από το αντίστοιχο residual stage περνά από ένα μικρό συνελκτικό υποδίκτυο, από global average pooling και από γραμμικό ταξινομητή που παράγει τα 10 logits της ενδιάμεσης πρόβλεψης.

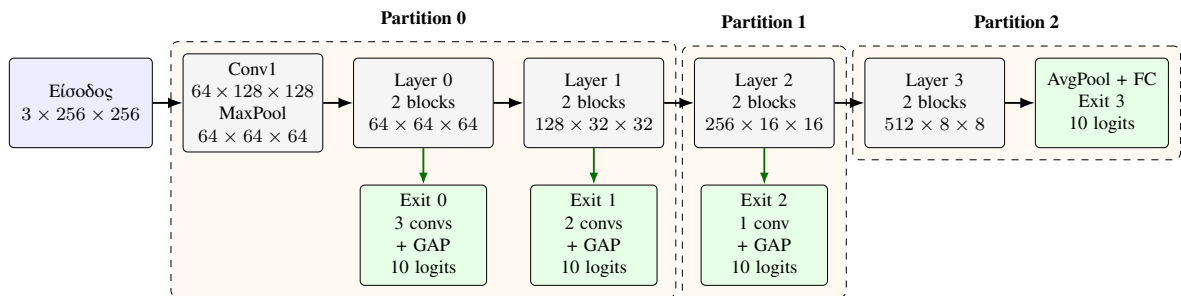
Οι τρεις κεφαλές δεν έχουν το ίδιο βάθος: η πρώτη, που συνδέεται με πιο ρηχή αναπαράσταση, χρησιμοποιεί τρεις συνελκτικές μονάδες, η δεύτερη δύο και η τρίτη μία. Με αυτόν τον τρόπο, οι πρώιμες εξόδους αποκτούν λίγο μεγαλύτερη ικανότητα επεξεργασίας, ενώ οι βαθύτερες εξόδους παραμένουν ελαφρύτερες. Οι θέσεις των exits είναι κοινές και για τις δύο αρχιτεκτονικές, όπως συνοψίζεται στο Εικόνα 4.4.



Εικόνα 4.4: Γενική αρχιτεκτονική των early-exit ResNet μοντέλων. Οι τρεις early-exit κεφαλές υλοποιούνται ως πλευρικές ταξινομητικές κεφαλές μετά τα τρία πρώτα residual layers, ενώ η τελική κεφαλή βρίσκεται μετά το τέταρτο layer.

Κάθε exit block παράγει ανεξάρτητη πρόβλεψη για τις 10 κλάσεις. Κατά το inference, οι έξοδοι ελέγχονται σειριακά με την πολιτική που περιγράφεται στην Ενότητα 4.6.3: ένα δείγμα είτε τερματίζει σε κάποια early-exit κεφαλή είτε συνεχίζει στο επόμενο layer.

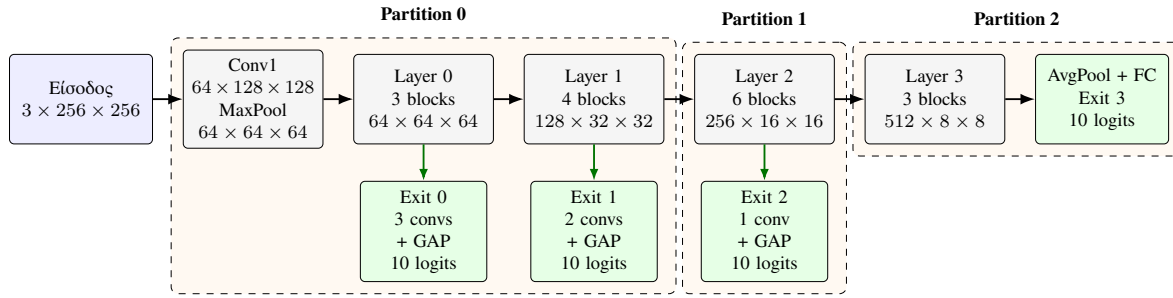
Για το ResNet-18 EE που χρησιμοποιείται στα πειράματα, η μεταβολή των διαστάσεων της εισόδου και οι θέσεις των εξόδων φαίνονται στο Εικόνα 4.5.



Εικόνα 4.5: Ροή ενεργοποιήσεων του ResNet-18 EE για είσοδο CIFAR-10 ή CIFAR-10.1. Τα blocks δείχνουν πώς αλλάζει το σχήμα του tensor μέσα στο backbone, ενώ οι πράσινες κεφαλές δείχνουν τις τρεις early-exit κεφαλές και την τελική έξοδο.

Το αντίστοιχο ResNet-34 EE ακολουθεί την ίδια ροή διαστάσεων, αλλά χρησιμοποιεί περισσότερα residual blocks σε κάθε layer, όπως φαίνεται στην Εικόνα 4.6.

Η διάσπαση αυτή είναι σκόπιμα ασύμμετρη: το Partition 0 περιλαμβάνει τα δύο πρώτα early exits και δέχεται όλες τις εργασίες inference στην αρχή της ροής, ενώ τα επόμενα partitions ενεργοποιούνται μόνο για όσα δείγματα δεν έχουν ήδη τερματίσει. Έτσι αναδεικνύεται ότι τα πρώτα exit branches μπορούν να συγκεντρώνουν δυσανάλογο φόρτο, και ότι η αντιστοίχιση των partitions στους workers μπορεί να αξιοποιήσει αυτή την ανισορροπία διασπείροντας τα συχνά ενεργά partitions διαφορετικών logical model instances σε περισσότερους workers. Η κατανομημένη διάσπαση αυτής της ροής ορίζεται στην Ενότητα 4.7.1.



Εικόνα 4.6: Ροή ενεργοποιήσεων του ResNet-34 EE για είσοδο CIFAR-10 ή CIFAR-10.1. Οι διαστάσεις των tensors παραμένουν ίδιες με το ResNet-18 EE, αλλά τα residual layers είναι βαθύτερα, με διάταξη blocks [3, 4, 6, 3].

4.6 Εκπαίδευση, Αποθηκευμένα Βάρη και Πολιτική early-exit

Τα πειράματα inference δεν εκπαιδεύουν μοντέλα κατά την εκτέλεση. Αντίθετα, χρησιμοποιούν αποθηκευμένα βάρη για τέσσερις εκδοχές: baseline ResNet-18, baseline ResNet-34, early-exit ResNet-18 και early-exit ResNet-34. Τα τέσσερα μοντέλα εκπαιδεύτηκαν σε Jupyter Notebook, εκτός του runtime των πειραμάτων inference, και τα βάρη που προέκυψαν αποθηκεύτηκαν στη συνέχεια ως checkpoints. Η εκπαίδευση έγινε στο CIFAR-10. Το CIFAR-10.1 χρησιμοποιείται μόνο στη φάση inference, χωρίς επανεκπαίδευση, fine-tuning ή αλλαγή threshold, ώστε να λειτουργεί ως ανεξάρτητος έλεγχος γενίκευσης των ίδιων μοντέλων. Στα κατανομημένα πειράματα τα ίδια βάρη είναι διαθέσιμα σε όλους τους workers. Κάθε worker κατασκευάζει την ίδια αρχιτεκτονική και εκτελεί μόνο το τμήμα που του έχει ανατεθεί, ώστε η κατανομημένη εκτέλεση να είναι ισοδύναμη με την αντίστοιχη single-node ροή ως προς τις αποφάσεις του μοντέλου.

4.6.1 Εκπαίδευση Baseline Μοντέλων

Τα baseline ResNet μοντέλα εκπαιδεύονται με cross-entropy loss:

$$\mathcal{L}_{base} = \text{CE}(y, \hat{y}).$$

Η training υλοποίηση χρησιμοποιεί Adam optimizer, learning rate 10^{-3} , batch size 64 και 40 epochs. Κατά την εκπαίδευση κρατείται το καλύτερο state dictionary με βάση την validation accuracy. Η φάση inference του κεφαλαίου δεν επαναλαμβάνει την εκπαίδευση, αλλά χρησιμοποιεί τα αποθηκευμένα weights ως σταθερή είσοδο για όλα τα σενάρια.

4.6.2 Εκπαίδευση Multi-Exit Μοντέλων

Στα early-exit μοντέλα, η εκπαίδευση παράγει logits και από τις τρεις ενδιάμεσες εξόδους και από την τελική κεφαλή. Η συνολική loss υπολογίζεται ως σταθμισμένο άθροισμα

των cross-entropy losses, κανονικοποιημένο με τον συνολικό αριθμό εξόδων:

$$\mathcal{L}_{EE} = \frac{\sum_{i=0}^{N-1} \lambda_i \text{CE}(y, \hat{y}_i)}{N}$$

Στην παραπάνω σχέση, $N = 4$ είναι ο συνολικός αριθμός εξόδων του μοντέλου: τρεις early-exit κεφαλές και μία τελική έξοδος. Η κανονικοποίηση γίνεται με το N , όπως στην υλοποίηση που χρησιμοποιήθηκε για τα checkpoints, και όχι με το άθροισμα των συντελεστών λ_i . Στην υλοποίηση χρησιμοποιούνται βάρη $\lambda = [1, 2, 3, 4]$. Έτσι, οι early-exit κεφαλές λαμβάνουν σήμα εκπαίδευσης, αλλά δίνεται μεγαλύτερη έμφαση στις βαθύτερες αναπαραστάσεις και στην τελική κεφαλή.

Η εκπαίδευση των μοντέλων πραγματοποιήθηκε με χρήση mixed-precision training. Η τεχνική αυτή μειώνει τις απαιτήσεις μνήμης και επιταχύνει τη διαδικασία εκπαίδευσης, διατηρώντας παράλληλα την αριθμητική σταθερότητα και την ακρίβεια των μοντέλων. Η χρήση mixed precision εφαρμόστηκε αποκλειστικά κατά την εκπαίδευση και δεν επηρεάζει τη διαδικασία inference, την πολιτική early-exit ή τη μεθοδολογία αξιολόγησης.

4.6.3 Entropy-Based Απόφαση Εξόδου

Κατά το inference, κάθε έξοδος μετατρέπει τα logits z_i σε πιθανότητες με softmax:

$$p_i(c) = \frac{\exp(z_{i,c})}{\sum_{j=1}^C \exp(z_{i,j})}.$$

Στη συνέχεια υπολογίζεται η εντροπία:

$$H(p_i) = - \sum_{c=1}^C p_i(c) \log(p_i(c)).$$

Το δείγμα τερματίζει στην πρώτη έξοδο που ικανοποιεί:

$$H(p_i) \leq \tau,$$

όπου σε όλα τα early-exit πειράματα χρησιμοποιείται $\tau = 0.9$. Αν καμία από τις τρεις early-exit κεφαλές δεν ικανοποιήσει το κριτήριο, το δείγμα φτάνει στην τελική έξοδο του δικτύου. Η ίδια πολιτική χρησιμοποιείται στη single-node και στην distributed εκτέλεση, ώστε οι διαφορές στις μετρικές να οφείλονται στον τρόπο εκτέλεσης και όχι σε διαφορετικό exit policy.

4.7 Partitioning και Κατανεμημένη Εκτέλεση

Η κατανεμημένη εκτέλεση βασίζεται στη διάσπαση του early-exit ResNet σε τρία διαδοχικά partitions. Κάθε worker φορτώνει το αντίστοιχο τμήμα του μοντέλου, εκτελεί τοπικά το partition που του έχει ανατεθεί και είτε επιστρέφει τελικό αποτέλεσμα, αν ενεργοποιηθεί κάποια έξοδος, είτε προωθεί την ενδιάμεση αναπαράσταση στον επόμενο worker.

4.7.1 Three-Partition Model Split

Η διάσπαση που χρησιμοποιείται στα distributed πειράματα συνοψίζεται στον Πίνακα 4.5. Η ίδια διάσπαση εφαρμόζεται και στα δύο early-exit μοντέλα.

Πίνακας 4.5: Διάσπαση του early-exit μοντέλου σε partitions και αντίστοιχες έξοδοι.

Partition	Τμήμα μοντέλου	Διαθέσιμη έξοδος
0	Αρχικό convolutional τμήμα και δύο πρώτα residual layers	Πρώτη ή δεύτερη early-exit κεφαλή.
1	Τρίτο residual layer	Τρίτη early-exit κεφαλή.
2	Τέταρτο residual layer, pooling και τελική ταξινομητική κεφαλή	Τελική έξοδος.

Με βάση την προεπεξεργασία της Ενότητας 4.4.1, η είσοδος κάθε δείγματος έχει σχήμα $3 \times 256 \times 256$, χωρίς να υπολογίζεται η διάσταση του batch. Μετά το αρχικό convolution και το max-pooling η αναπαράσταση έχει σχήμα $64 \times 64 \times 64$. Το Partition 0, όταν δεν τερματίζει σε κάποια από τις δύο πρώτες early-exit κεφαλές, προωθεί προς το Partition 1 την ενεργοποίηση μετά το δεύτερο residual layer, με σχήμα $128 \times 32 \times 32$. Αντίστοιχα, το Partition 1 προωθεί προς το Partition 2 την ενεργοποίηση μετά το τρίτο residual layer, με σχήμα $256 \times 16 \times 16$. Αυτές οι ενδιάμεσες αναπαραστάσεις είναι τα tensor payloads που μεταδίδονται μεταξύ workers και επομένως επηρεάζουν άμεσα το communication/runtime overhead.

Στην απλή distributed τοπολογία, το Partition 0 τοποθετείται στον πρώτο worker, το Partition 1 στον δεύτερο worker και το Partition 2 στον τρίτο worker. Η επιλογή αυτή δίνει ιδιαίτερο βάρος στο πρώτο partition, επειδή εκεί εκτελούνται δύο ενδιάμεσες έξοδοι και εκεί τερματίζει μεγάλο μέρος των εύκολων δειγμάτων. Τα επόμενα partitions δέχονται μόνο τα δείγματα που δεν τερμάτισαν νωρίτερα.

4.7.2 Πολυ-μοντελική Τοποθέτηση

Στην πολυ-μοντελική εκτέλεση, ένα logical model instance αντιστοιχεί σε μια ανεξάρτητη ροή inference του ίδιου early-exit μοντέλου. Κάθε instance έχει τη δική του διαδρομή

μέσα στο cluster, δηλαδή μια αντιστοίχιση ανάμεσα στα partitions του μοντέλου και στους διαθέσιμους workers. Η αντιστοίχιση αυτή ορίζει το placement των partitions.

Το placement επηρεάζει τόσο τον υπολογιστικό φόρτο όσο και τη μνήμη που απαιτείται σε κάθε worker. Για παράδειγμα, αν δύο instances τοποθετούν τα πρώτα partitions στον ίδιο worker, ο worker αυτός δέχεται όλα τα δείγματα και μπορεί να γίνει σημείο συμφόρησης. Αντίθετα, διαφορετικές διαδρομές μπορούν να μοιράσουν τα υψηλής κίνησης partitions σε περισσότερους workers. Οι συγκεκριμένες τοπολογίες που χρησιμοποιούνται στα πειράματα παρουσιάζονται στην Ενότητα 4.8.

4.8 Πειραματικές Τοπολογίες και Σενάρια

Η τρέχουσα δέσμη αποθηκευμένων αποτελεσμάτων περιλαμβάνει 28 runs: 14 runs στο CIFAR-10 και τα ίδια 14 runs στο CIFAR-10.1. Για κάθε βασικό σενάριο εκτελείται τόσο ResNet-18 όσο και ResNet-34, ώστε να αξιολογηθεί αν τα ίδια συμπεράσματα ισχύουν για δύο διαφορετικά βάθη μοντέλου. Η επανάληψη της ίδιας δέσμης στο CIFAR-10.1 διατηρεί αμετάβλητα τα μοντέλα, τα thresholds και τις τοπολογίες, αλλά αλλάζει μόνο το test set εισόδου.

Τα πειράματα δεν αποτελούν ανεξάρτητες μετρήσεις χωρίς μεταξύ τους σχέση, αλλά οργανώνονται κλιμακωτά. Αρχικά μετριέται η συμπεριφορά ενός πλήρους μοντέλου και της early-exit εκδοχής του σε έναν κόμβο. Στη συνέχεια το ίδιο early-exit μοντέλο διασπάται σε τρεις workers, ώστε να αποτιμηθεί το κόστος και το όφελος της κατανεμημένης εκτέλεσης. Τέλος, η ίδια υποδομή χρησιμοποιείται για πολυ-μοντελική εκτέλεση, όπου το ζητούμενο δεν είναι μόνο η επιτάχυνση ενός μεμονωμένου inference path, αλλά και η ικανότητα του edge cluster να εξυπηρετεί περισσότερες παράλληλες ροές inference.

4.8.1 Experiment 1: Single-Model Εκτέλεση

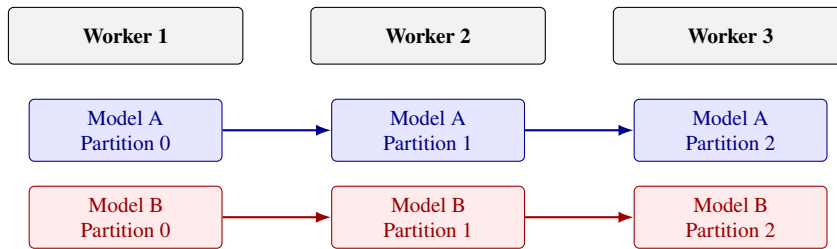
Το Experiment 1 μελετά ένα μοντέλο κάθε φορά. Περιλαμβάνει τρεις υποκατηγορίες:

- **Exp1.1:** baseline ResNet σε single-node CPU execution, χωρίς early exits. Σκοπός του είναι να ορίσει το σημείο αναφοράς για την ακρίβεια, το latency και το throughput όταν το δίκτυο εκτελείται ολόκληρο για κάθε δείγμα.
- **Exp1.2:** early-exit ResNet σε single-node CPU execution. Το πείραμα αυτό απομονώνει την επίδραση των early exits χωρίς δικτυακό κόστος ή κατανεμημένη εκτέλεση. Με αυτόν τον τρόπο μετριέται αν ο early-exit μηχανισμός μειώνει το latency και αυξάνει το throughput, καθώς και ποια είναι η αντίστοιχη επίπτωση στην ακρίβεια.
- **Exp1.3:** early-exit ResNet κατανεμημένο σε τρεις Raspberry Pi workers με τη διάσπαση Partition 0 – Partition 1 – Partition 2. Ο σκοπός του είναι να δείξει πώς το μοντέλο μπορεί να διαμοιραστεί σε πολλούς edge workers και τι επίπτωση έχει αυτή η

διάσπαση στο latency, στο communication/runtime overhead και στην αξιοποίηση των workers. Παράλληλα, λειτουργεί ως βάση για τα επόμενα πολυ-μοντελικά πειράματα, επειδή ορίζει την κοινή pipeline διαδρομή πάνω στην οποία θα τοποθετηθούν περισσότερα logical model instances.

4.8.2 Experiment 2: Multi-Model Εκτέλεση με Κοινή Διαδρομή

Το Experiment 2 εκτελεί δύο λογικά instances του ίδιου early-exit μοντέλου πάνω στους τρεις workers. Εφόσον το cluster έχει τρεις workers, η πολυ-μοντελική εκτέλεση μελετά δύο παράλληλες λογικές ροές inference. Και οι δύο ακολουθούν την ίδια διαδρομή, όπως φαίνεται στο Εικόνα 4.7.



Εικόνα 4.7: Κοινή διαδρομή των δύο logical model instances στο Experiment 2. Και τα δύο instances τοποθετούν το Partition 0 στον Worker 1, το Partition 1 στον Worker 2 και το Partition 2 στον Worker 3.

Οι δύο ροές εκτελούνται ταυτόχρονα. Για κάθε εικόνα του εκάστοτε dataset παράγονται δύο εργασίες inference, μία για κάθε λογικό instance του μοντέλου.

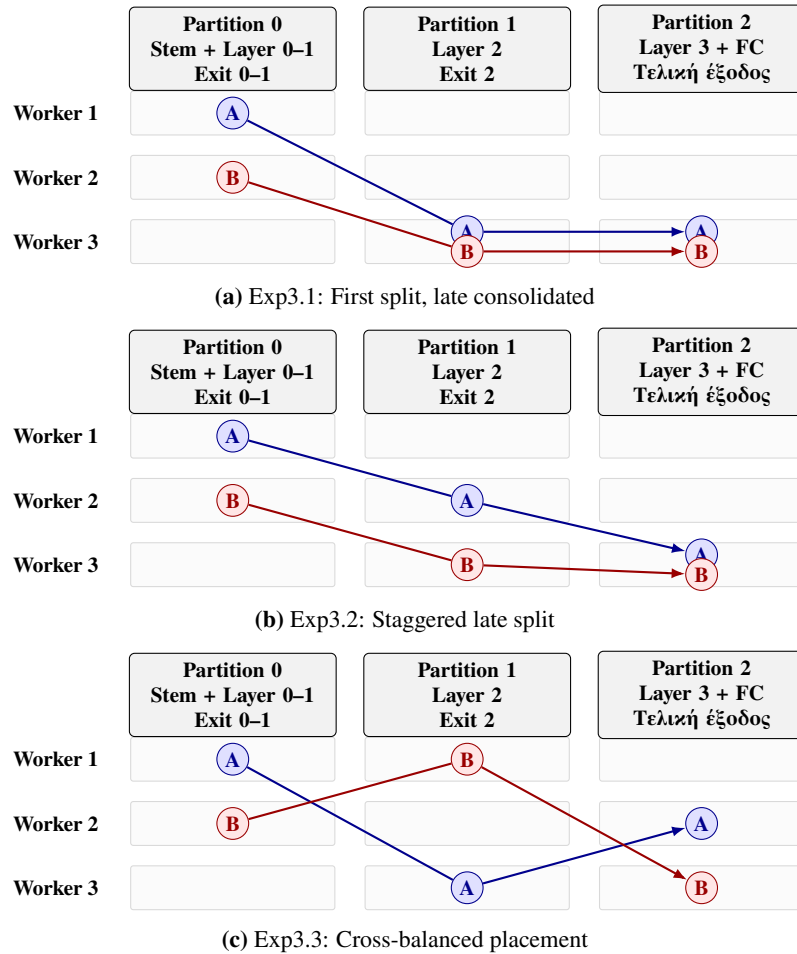
Ο σκοπός του Experiment 2 είναι να εξετάσει τι συμβαίνει όταν η ίδια κατανομή υποδομής δεν εξυπηρετεί ένα μόνο μοντέλο, αλλά δύο παράλληλες ροές inference. Επομένως, το Experiment 2 μετρά την επίδραση της πολυ-μοντελικής εκτέλεσης στο throughput, στο latency, στην αξιοποίηση των workers και στη συγκέντρωση φόρτου στα ίδια pipelines. Επειδή και τα δύο model instances ακολουθούν την ίδια διαδρομή, το πείραμα αποτελεί μια αφελή προσέγγιση πολυ-μοντελικής τοποθέτησης: τα partitions των δύο ροών αντιγράφονται στους ίδιους workers, χωρίς προσπάθεια ανακατανομής του φορτίου. Η επιλογή αυτή λειτουργεί ως σημείο αναφοράς για το Experiment 3, όπου εξετάζεται αν η διαφορετική τοποθέτηση των partitions μπορεί να μειώσει τη συμφόρηση και να βελτιώσει τη συνολική συμπεριφορά του συστήματος.

4.8.3 Experiment 3: Memory-Aware Placement

Το Experiment 3 διατηρεί δύο λογικά model instances, αλλά αλλάζει την τοποθέτηση των partitions. Η βασική ιδέα είναι ότι το Partition 0 δέχεται όλες τις εργασίες inference και περιέχει τα δύο πρώτα early exits, άρα αποτελεί υψηλής κίνησης partition. Για αυτό, οι στρατηγικές του Experiment 3 τοποθετούν το αρχικό partition των δύο model instances σε διαφορετικούς workers.

Ο σκοπός του Experiment 3 είναι να εξετάσει αν η τοποθέτηση των partitions μπορεί να βελτιώσει την πολυ-μοντελική εκτέλεση χωρίς να αλλάξει ούτε η αρχιτεκτονική του μοντέλου ούτε η early-exit πολιτική. Σε αντίθεση με το Experiment 2, όπου τα δύο μοντέλα ακολουθούν την ίδια worker διαδρομή, εδώ η τοποθέτηση αντιμετωπίζεται ως μεταβλητή σχεδιασμού. Το ζητούμενο είναι να μειωθεί η συγκέντρωση μνήμης και υπολογιστικού φόρτου σε έναν worker, ιδιαίτερα στα πρώτα partitions που δέχονται όλες τις εργασίες inference πριν ενεργοποιηθεί κάποια early-exit κεφαλή.

Η αντιστοίχιση partition–worker για τις τρεις στρατηγικές φαίνεται στο Εικόνα 4.8.



Εικόνα 4.8: Στρατηγικές τοποθέτησης του Experiment 3. Κάθε στήλη αντιστοιχεί σε partition του early-exit pipeline και κάθε γραμμή σε worker. Τα σημεία A και B δείχνουν σε ποιον worker εκτελείται το αντίστοιχο partition των δύο logical model instances.

Οι τρεις στρατηγικές επιτρέπουν σύγκριση ανάμεσα σε συγκέντρωση των μεταγενέστερων partitions στον ίδιο worker, μερική μετακίνηση του δεύτερου partition και πλήρη cross-balanced τοποθέτηση των αργότερων partitions.

Με αυτόν τον τρόπο, το Experiment 3 απαντά αν η απλή αναπαραγωγή της ίδιας pipeline διαδρομής είναι επαρκής για multi-model inference ή αν η ρητή τοποθέτηση των partitions μπορεί να οδηγήσει σε καλύτερη κατανομή πόρων και υψηλότερο συνολικό throughput.

4.8.4 Κατάλογος Runs

Πίνακας 4.7: Σύνοψη των αποθηκευμένων CIFAR-10 και CIFAR-10.1 πειραματικών runs.

Ομάδα	Μοντέλα	Inference tasks ανά dataset	Περιγραφή
Exp1.1	ResNet-18, ResNet-34 baseline	CIFAR-10: 10.000, CIFAR-10.1: 2.000 ανά run	Single-node baseline inference.
Exp1.2	ResNet-18 EE, ResNet-34 EE	CIFAR-10: 10.000, CIFAR-10.1: 2.000 ανά run	Single-node early-exit inference.
Exp1.3	ResNet-18 EE, ResNet-34 EE	CIFAR-10: 10.000, CIFAR-10.1: 2.000 ανά run	Distributed inference σε τρεις workers.
Exp2	2x ResNet-18 EE, 2x ResNet-34 EE	CIFAR-10: 20.000, CIFAR-10.1: 4.000 ανά run	Δύο logical models στην κανονική τριών-worker διαδρομή.
Exp3.1–3.3	2x ResNet-18 EE, 2x ResNet-34 EE	CIFAR-10: 20.000, CIFAR-10.1: 4.000 ανά run	Τρεις placement στρατηγικές για δύο logical models.

4.9 Distributed Runtime και Επικοινωνία

Η κατανεμημένη εκτέλεση υλοποιείται με αρχιτεκτονική master-worker. Ο master διαβάζει τα δείγματα του test split, ξεκινά κάθε αίτημα inference στον πρώτο worker της αντίστοιχης διαδρομής και καταγράφει την τελική απόκριση. Οι workers εκτελούν τα partitions που τους έχουν ανατεθεί και επικοινωνούν μεταξύ τους όταν ένα δείγμα πρέπει να συνεχίσει σε επόμενο στάδιο.

4.9.1 Ποή Inference

Η βασική ροή ενός distributed δείγματος είναι:

1. ο master διαβάζει ένα δείγμα από το test split,
2. το δείγμα αποστέλλεται στον worker που εκτελεί το πρώτο partition,
3. ο worker υπολογίζει το τοπικό partition και ελέγχει αν κάποια early-exit κεφαλή ικανοποιεί το κριτήριο τερματισμού,
4. αν το δείγμα τερματίσει, η πρόβλεψη επιστρέφεται στον master,
5. διαφορετικά, η ενδιάμεση αναπαράσταση μεταδίδεται στον worker του επόμενου partition,

6. η τελική απόκριση περιλαμβάνει την έξοδο στην οποία τερμάτισε το δείγμα, την πρόβλεψη, την εμπιστοσύνη, τη διαδρομή που ακολουθήθηκε και τον χρόνο υπολογισμού ανά worker.

Στα multi-model πειράματα, δύο λογικές ροές inference εκτελούνται παράλληλα πάνω στο ίδιο cluster. Κάθε αίτημα συνοδεύεται από τα απαραίτητα μεταδεδομένα ώστε οι workers να γνωρίζουν σε ποιο λογικό instance ανήκει και ποιο partition πρέπει να εκτελέσουν.

4.9.2 Πρωτόκολλο Επικοινωνίας

Κάθε distributed αίτημα μεταφέρει δύο είδη πληροφορίας: μεταδεδομένα που περιγράφουν το δείγμα και το τρέχον partition, και το tensor payload που αντιστοιχεί είτε στην αρχική εικόνα είτε σε ενδιάμεση αναπαράσταση. Το tensor σειριοποιείται πριν από τη μετάδοση και ανακατασκευάζεται στον παραλήπτη με βάση το σχήμα και τον τύπο των δεδομένων.

Η ίδια λογική υποστηρίζει τόσο την απλή ακολουθιακή διαδρομή των τριών workers όσο και τις εναλλακτικές τοποθετήσεις του Experiment 3. Αν δύο διαδοχικά partitions έχουν ανατεθεί στον ίδιο worker, η προώθηση γίνεται τοπικά χωρίς δικτυακή μετάδοση. Αν βρίσκονται σε διαφορετικούς workers, η ενδιάμεση αναπαράσταση μεταφέρεται μέσω του τοπικού δικτύου.

4.9.3 Έλεγχος Συμβατότητας Workers

Πριν από κάθε distributed run ελέγχεται ότι οι workers έχουν φορτώσει το σωστό μοντέλο, το σωστό dataset και το partition που αντιστοιχεί στο τρέχον πείραμα. Ο έλεγχος αυτός είναι απαραίτητος επειδή η αλλαγή από ResNet-18 σε ResNet-34 ή από μία στρατηγική placement σε άλλη αλλάζει τον ρόλο που πρέπει να αναλάβει κάθε worker.

4.10 Μετρικές Αξιολόγησης

Η αξιολόγηση κάθε πειραματικής εκτέλεσης βασίζεται σε δύο επίπεδα καταγραφής. Το πρώτο είναι η συγκεντρωτική περιγραφή του πειράματος, η οποία περιλαμβάνει τις συνολικές μετρικές του σεναρίου. Το δεύτερο είναι η αναλυτική καταγραφή ανά εργασία inference. Για κάθε εργασία καταγράφονται η πρόβλεψη, η πραγματική ετικέτα, ο χρόνος εκτέλεσης, το exit που χρησιμοποιήθηκε και, στα κατανεμημένα πειράματα, οι χρόνοι compute και η διαδρομή στους workers.

Στα multi-model πειράματα, η βασική μονάδα μέτρησης δεν είναι μόνο η μοναδική εικόνα εισόδου, αλλά το ζεύγος εικόνας και λογικού model instance. Επομένως, όταν δύο logical model instances εκτελούνται πάνω στο ίδιο test set, κάθε εικόνα παράγει δύο

εργασίες inference. Το N_{tasks} που χρησιμοποιείται στις συγκεντρωτικές μετρικές αντιστοιχεί σε αυτές τις εργασίες.

Ο Πίνακας 4.9 συνοψίζει τις μετρικές που χρησιμοποιούνται, τον τρόπο υπολογισμού τους και τον ρόλο τους στην ερμηνεία των πειραμάτων.

Πίνακας 4.9: Συνοπτική αντιστοίχιση μετρικών, υπολογισμού και ρόλου.

Μετρική	Υπολογισμός ή πηγή	Ρόλος
Correct tasks και total tasks	N_{correct} και N_{tasks} από τις αναλυτικές εγγραφές ανά εργασία.	Ορίζουν το μέγεθος της πειραματικής εκτέλεσης και τη βάση για την ακρίβεια.
Accuracy	$100 \cdot N_{\text{correct}}/N_{\text{tasks}}$.	Μετρά την ποιότητα ταξινόμησης.
Exit counts & ratios	$N_{\text{exit } i}$ και $N_{\text{exit } i}/N_{\text{tasks}}$, για exits 0–3.	Δείχνουν πόσες εργασίες inference τερματίζουν σε κάθε βάθος του early-exit μοντέλου.
Latency ανά εργασία	Χρόνος ανά εργασία inference. Σε single-node πειράματα είναι ο χρόνος τοπικής εκτέλεσης του μοντέλου. Σε distributed πειράματα είναι ο end-to-end χρόνος που παρατηρεί ο master.	Βασική μετρική απόκρισης ανά εργασία inference.
Latency statistics	Μέση τιμή, τυπική απόκλιση, ελάχιστο και μέγιστο.	Συνοψίζουν την τυπική και την ακραία συμπεριφορά του χρόνου απόκρισης.
Total inference time	Πραγματική χρονική διάρκεια του κύριου μέρους της πειραματικής εκτέλεσης, χωρίς το warmup: T_{exp} .	Κοινός παρονομαστής για throughput και utilization.
Throughput	$N_{\text{tasks}}/T_{\text{exp}}$, όπου T_{exp} είναι η πραγματική διάρκεια της πειραματικής εκτέλεσης.	Ρυθμός εξυπηρέτησης εργασιών inference ανά δευτερόλεπτο.
Busy time	$T_{\text{busy}} = \sum_j t_j$, όπου t_j είναι ο χρόνος της εργασίας j .	Συνολικός αθροιστικός χρόνος που παρατηρείται στις εργασίες inference.
Utilization	$T_{\text{busy}}/T_{\text{exp}}$ για single-node/master και $T_{\text{compute},w}/T_{\text{exp}}$ για κάθε worker.	Εκτίμηση αξιοποίησης της μονάδας εκτέλεσης από τη συγκεκριμένη πειραματική ροή.
Remote compute time	Άθροισμα των compute χρόνων που αναφέρουν τα partitions των workers για κάθε distributed εργασία.	Απομονώνει τον χρόνο εκτέλεσης των partitions από το end-to-end latency.
Communication/runtime overhead	$t_j - c_j$, όπου t_j είναι το end-to-end latency και c_j ο remote compute χρόνος της ίδιας εργασίας.	Προσεγγίζει το κόστος επικοινωνίας, serialization, HTTP handling, forwarding και αναμονής.
Communication/runtime overhead ratios	Μέση τιμή των λόγων ανά εργασία $(t_j - c_j)/t_j$ και συνολικός λόγος $\sum_j (t_j - c_j) / \sum_j t_j$.	Δείχνουν ποιο μέρος του latency δεν αντιστοιχεί σε compute των partitions.
Worker compute metrics	Συνολικός και μέσος compute χρόνος ανά worker, καθώς και worker utilization.	Αποκαλύπτουν αν ο φόρτος συγκεντρώνεται σε συγκεκριμένους workers.
Energy και carbon	Εκτιμήσεις ενεργειακής κατανάλωσης σε kWh και εκπομπών σε kgCO ₂ eq, συνολικά ή χωριστά για master και workers στα distributed πειράματα.	Περιγράφουν το ενεργειακό και περιβαλλοντικό κόστος της εκτέλεσης.

Μετρική	Υπολογισμός ή πηγή	Ρόλος
Energy per task και EDP	$E_{\text{task}} = E_{\text{system}} \cdot 3.6 \cdot 10^6 / N_{\text{tasks}}$ και $EDP = E_{\text{task}} \cdot \bar{t}$.	Συσχετίζουν την κατανάλωση ενέργειας με το πλήθος εργασιών και τον χρόνο απόκρισης.

4.10.1 Ακρίβεια και Exit Distribution

Η ακρίβεια υπολογίζεται από το πλήθος σωστών προβλέψεων και το πλήθος εργασιών inference:

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{tasks}}} \cdot 100.$$

Η ακρίβεια χρησιμοποιείται ως βασική μετρική ποιότητας ταξινόμησης, αλλά στην παρούσα αξιολόγηση έχει και ελεγκτικό ρόλο. Περιμένουμε οι single-node, distributed και multi-model εκτελέσεις του ίδιου checkpoint, να έχουν την ίδια ή πρακτικά ίδια ακρίβεια, καθώς δεν αλλάζει η συμπεριφορά ταξινόμησης του μοντέλου ούτε η πολιτική εξόδου. Επομένως, οι διαφορές που παρατηρούνται σε latency, throughput, utilization και ενέργεια μπορούν να αποδοθούν στον τρόπο εκτέλεσης και στην τοπολογία του συστήματος, όχι σε διαφορετική ποιότητα προβλέψεων. Αντίθετα, στη σύγκριση baseline και early-exit μοντέλων, η ακρίβεια ποσοτικοποιεί τον συμβιβασμό ανάμεσα στην ποιότητα ταξινόμησης και στη μείωση του μέσου υπολογισμού.

Στα single-model πειράματα, το N_{tasks} ταυτίζεται με το πλήθος εικόνων του test set. Στα multi-model πειράματα, το ίδιο test set εκτελείται για κάθε logical model instance, άρα το N_{tasks} είναι το άθροισμα όλων των εργασιών inference. Για παράδειγμα, δύο instances πάνω σε 10.000 εικόνες δημιουργούν 20.000 εργασίες inference.

Για τα early-exit μοντέλα καταγράφονται τα πλήθη $N_{\text{exit } i}$ και τα αντίστοιχα ratios για τα exits 0, 1, 2 και 3. Τα exits 0–2 αντιστοιχούν στις ενδιάμεσες κεφαλές, ενώ το exit 3 αντιστοιχεί στην τελική έξοδο του μοντέλου. Στα baseline πειράματα, όπου δεν υπάρχουν early-exit κεφαλές, δεν ορίζεται αντίστοιχη κατανομή εξόδων.

Η κατανομή των exits λειτουργεί ως γέφυρα ανάμεσα στη συμπεριφορά του μοντέλου και στη συμπεριφορά του συστήματος. Σε επίπεδο μοντέλου δείχνει πόσο συχνά η early-exit πολιτική επιτρέπει τον πρόωρο τερματισμό και, άρα, ποιο είναι το πραγματικό μέσο βάθος εκτέλεσης. Σε επίπεδο distributed εκτέλεσης δείχνει ποια partitions και ποιοι workers ενεργοποιούνται στην πράξη. Για τον λόγο αυτό χρησιμοποιείται για να ερμηνευθούν τόσο τα οφέλη του adaptive compute όσο και η πιθανή ανισορροπία φόρτου στους workers.

4.10.2 Latency, Throughput και Utilization

Για κάθε εργασία inference καταγράφεται latency t_j . Στα single-node πειράματα ο χρόνος αυτός μετρά την τοπική εκτέλεση του μοντέλου για το δείγμα. Στα distributed πειράματα, αντίθετα, ο χρόνος μετριέται από την πλευρά του master και περιλαμβάνει ολόκληρη τη διαδρομή μέχρι την τελική απόκριση: αποστολή request, εκτέλεση στους workers, ενδεχόμενη προώθηση ενεργοποιήσεων και επιστροφή αποτελέσματος.

Το latency χρησιμοποιείται ως η βασική μετρική απόκρισης ανά εργασία inference. Δείχνει πόσο κοστίζει χρονικά η επεξεργασία ενός δείγματος ή, στα multi-model πειράματα, ενός ζεύγους δείγματος και logical model instance. Επομένως, είναι η κατάλληλη μετρική για να φανεί αν ο early-exit μηχανισμός μειώνει τον χρόνο που απαιτείται για μία μεμονωμένη πρόβλεψη και αν η κατανομημένη εκτέλεση προσθέτει ή αφαιρεί καθυστέρηση σε σχέση με την τοπική εκτέλεση. Από τους χρόνους ανά εργασία υπολογίζονται στατιστικά η μέση τιμή, η οποία περιγράφει τον τυπικό χρόνο απόκρισης.

Το συνολικό inference time T_{exp} είναι η πραγματική χρονική διάρκεια του κύριου μέρους της πειραματικής εκτέλεσης. Το warmup εκτελείται πριν από τη χρονομέτρηση και δεν προσμετράται στις τελικές μετρίκες. Το throughput ορίζεται ως:

$$\text{Throughput} = \frac{N_{\text{tasks}}}{T_{\text{exp}}}.$$

Η μετρική αυτή εκφράζει πόσες εργασίες inference εξυπηρετούνται ανά δευτερόλεπτο. Στα multi-model πειράματα, επειδή οι εργασίες των δύο logical instances εκτελούνται παράλληλα, το throughput υπολογίζεται πάνω στο συνολικό πλήθος εργασιών και όχι μόνο στο πλήθος μοναδικών εικόνων.

Το throughput συμπληρώνει το latency, επειδή περιγράφει τη χωρητικότητα του συστήματος και όχι τον χρόνο μίας μεμονωμένης εργασίας. Δύο τοπολογίες μπορεί να έχουν παρόμοιο μέσο latency αλλά διαφορετικό throughput, αν η μία αξιοποιεί καλύτερα την παράλληλη εκτέλεση ή μειώνει τα σημεία συμφόρησης. Για τον λόγο αυτό, το throughput χρησιμοποιείται κυρίως για τη σύγκριση single-model και multi-model σεναρίων και για την αξιολόγηση των memory-aware τοποθετήσεων.

Το utilization εκφράζει πόσο έντονα αξιοποιείται μια μονάδα εκτέλεσης από τη συγκεκριμένη πειραματική ροή. Ο σκοπός της μετρικής είναι να δείξει αν ο χρόνος της πειραματικής εκτέλεσης καταναλώνεται κυρίως σε ενεργή επεξεργασία ή σε αναμονή, επικοινωνία και συγχρονισμό. Στα single-node πειράματα βοηθά να φανεί πόσο συνεχόμενα απασχολείται ο μοναδικός κόμβος από το inference. Στα distributed και multi-model πειράματα είναι ακόμη πιο σημαντική, επειδή επιτρέπει να εντοπιστεί αν κάποιος worker συγκεντρώνει δυσανάλογο φόρτο, ενώ άλλοι παραμένουν υποαξιοποιημένοι. Η μετρική υπολογίζεται ως λόγος αθροιστικού χρόνου απασχόλησης προς τη συνολική διάρκεια της πειραματικής εκτέλεσης:

$$U_N = \frac{T_{\text{busy}}}{T_{\text{exp}}}.$$

Το T_{busy} υπολογίζεται ως άθροισμα των χρόνων όλων των εργασιών inference:

$$T_{\text{busy}} = \sum_{j=1}^{N_{\text{tasks}}} t_j,$$

όπου t_j είναι ο χρόνος που μετράται για την εργασία j . Στα single-node πειράματα, ο χρόνος αυτός αντιστοιχεί στην τοπική εκτέλεση του μοντέλου για ένα δείγμα. Στα distributed πειράματα, η αντίστοιχη master-level ποσότητα προκύπτει από τους end-to-end χρόνους που παρατηρεί ο master, δηλαδή από τη στιγμή που ξεκινά η αποστολή της εργασίας μέχρι τη λήψη της τελικής απόκρισης.

Στα distributed πειράματα, για κάθε worker χρησιμοποιείται αντίστοιχα ο αθροιστικός compute χρόνος που χρεώνεται στον worker, $T_{\text{compute},w}$, και η worker-level αξιοποίηση υπολογίζεται ως $U_w = T_{\text{compute},w}/T_{\text{exp}}$. Επειδή στα multi-model πειράματα υπάρχουν παράλληλες εργασίες από διαφορετικά logical model instances, ο αθροιστικός χρόνος εργασιών ή compute χρόνος ενός worker μπορεί να είναι μεγαλύτερος από την πραγματική wall-clock διάρκεια της πειραματικής εκτέλεσης. Συνεπώς, η τιμή αυτή είναι application-level normalized load και όχι CPU utilization του λειτουργικού συστήματος. Δεν είναι απαραίτητα περιορισμένη στο $[0, 1]$: τιμή 1.5, ή 150% όταν παρουσιάζεται ως ποσοστό, σημαίνει ότι στο ίδιο wall-clock διάστημα αντιστοιχούν περίπου 1.5 worker-seconds compute ανά δευτερόλεπτο πειράματος για τον συγκεκριμένο worker, λόγω concurrency ή συγκέντρωσης φόρτου σε κοινό worker.

4.10.3 Remote Compute και Communication/Runtime Overhead

Στα distributed πειράματα, κάθε worker επιστρέφει τον τοπικό compute χρόνο του partition που εκτέλεσε. Για μία εργασία j , οι χρόνοι αυτοί αθροίζονται στον remote compute χρόνο:

$$c_j = \sum_{s \in \text{path}(j)} T_{\text{compute},s}.$$

Ο remote compute χρόνος χρησιμοποιείται για να απομονωθεί το τμήμα του latency που αντιστοιχεί σε πραγματική εκτέλεση των partitions. Έτσι, το end-to-end latency μπορεί να διαβαστεί ως συνδυασμός υπολογισμού και κόστους κατανεμημένης εκτέλεσης.

Το communication/runtime overhead υπολογίζεται ως η διαφορά ανάμεσα στο end-to-end latency και στον remote compute χρόνο:

$$T_{\text{overhead},j} = t_j - c_j.$$

Η ποσότητα αυτή δεν είναι packet-level μέτρηση δικτύου ούτε μετρά άμεσα μεταδιδόμενα bytes. Περιλαμβάνει το κόστος μεταφοράς των tensor payloads, serialization και deserialization, HTTP/FastAPI handling, forwarding μεταξύ workers, response handling και πιθανή αναμονή λόγω συμφόρησης ή παράλληλης εκτέλεσης. Επομένως, πρόκειται για εκτίμηση στο επίπεδο της εφαρμογής για το μη υπολογιστικό μέρος του distributed latency.

Για τον λόγο αυτό, το communication/runtime overhead χρησιμοποιείται ως βασική μετρική αποδοτικότητας του placement. Υψηλή τιμή δείχνει ότι το κέρδος από τη διαμοίραση του compute μπορεί να περιορίζεται από τη διαδρομή εκτέλεσης μέσα στο σύστημα.

Εκτός από τον απόλυτο χρόνο communication/runtime overhead, χρησιμοποιείται και ο συνολικός λόγος communication/runtime overhead:

$$R_{\text{overhead,total}} = \frac{\sum_{j=1}^{N_{\text{tasks}}} T_{\text{overhead},j}}{\sum_{j=1}^{N_{\text{tasks}}} t_j}.$$

Ο λόγος αυτός εκφράζει ποιο ποσοστό του συνολικού latency budget των εργασιών inference αντιστοιχεί σε μη υπολογιστικό χρόνο. Για τον λόγο αυτό χρησιμοποιείται στους συγκεντρωτικούς πίνακες και στα γραφήματα ως system-level μετρική σύγκρισης των placements. Σε αντίθεση με τον απλό μέσο όρο των ratios ανά εργασία, ο συνολικός λόγος ζυγίζει κάθε εργασία με βάση τον πραγματικό χρόνο της και δείχνει πόσο από το συνολικό άθροισμα των end-to-end latencies δεν είναι remote compute. Ο παρονομαστής είναι το άθροισμα των latencies ανά εργασία και όχι το wall-clock T_{exp} , κάτι που είναι σημαντικό στα multi-model πειράματα όπου διαφορετικές εργασίες μπορούν να εκτελούνται παράλληλα.

4.10.4 Ενέργεια και Εκπομπές

Οι ενεργειακές μετρικές προέρχονται από εκτιμήσεις λογισμικού για την κατανάλωση του συστήματος κατά τη διάρκεια της πειραματικής εκτέλεσης μέσω του CodeCarbon πακέτου. Στα single-node πειράματα καταγράφονται απευθείας η εκτιμώμενη ενέργεια kWh και οι εκπομπές kgCO₂eq. Στα distributed πειράματα καταγράφονται χωριστά οι τιμές του master και των workers, καθώς και αθροιστικές τιμές για τους workers και για το συνολικό σύστημα:

$$E_{\text{system}} = E_{\text{master}} + \sum_w E_w.$$

Αντίστοιχα υπολογίζεται και το συνολικό carbon footprint. Οι τιμές αυτές είναι εκτιμήσεις επιπέδου λογισμικού και δεν προέρχονται από εξωτερικό power meter. Για τον λόγο αυτό χρησιμοποιούνται κυρίως συγκριτικά, δηλαδή για να φανεί πώς αλλάζει το ενεργειακό κόστος ανάμεσα στα σενάρια.

Για τα γραφήματα και τους συγκριτικούς πίνακες παράγονται επίσης παράγωγες μετρικές. Η ενέργεια ανά εργασία inference υπολογίζεται ως:

$$E_{\text{task}} = \frac{E_{\text{system}} \cdot 3.6 \cdot 10^6}{N_{\text{tasks}}},$$

όπου η ενέργεια μετατρέπεται από kWh σε Joule. Τέλος, χρησιμοποιείται το energy-delay product:

$$\text{EDP} = E_{\text{task}} \cdot \bar{t},$$

όπου $\bar{t}$ είναι το μέσο latency της πειραματικής εκτέλεσης. Η μετρική αυτή συνδυάζει ενεργειακό κόστος και latency, ώστε να μην αξιολογείται μια τοπολογία μόνο ως προς την ταχύτητα ή μόνο ως προς την κατανάλωση.

4.11 Περιορισμοί

Τα αποτελέσματα πρέπει να ερμηνευθούν μέσα στο συγκεκριμένο πειραματικό πλαίσιο της εργασίας. Πρώτος περιορισμός είναι η χρήση της οικογένειας CIFAR-10 ως ελεγχόμενου benchmark, με το CIFAR-10.1 να λειτουργεί ως ανεξάρτητο test set με μικρή φυσική μεταβολή της κατανομής. Ωστόσο, ούτε το CIFAR-10 ούτε το CIFAR-10.1 αντιπροσωπεύουν πλήρως φόρτους εργασίας πραγματικού μεγέθους, όπου οι εικόνες, τα μοντέλα, οι ρυθμοί αιτημάτων και οι συνθήκες δικτύου μπορεί να είναι σημαντικά πιο απαιτητικά. Επομένως, τα συμπεράσματα αφορούν κυρίως τη συστημική συμπεριφορά των early-exit και distributed μηχανισμών κάτω από ελεγχόμενες συνθήκες, και όχι την απόδοση σε ένα πλήρες παραγωγικό περιβάλλον.

Ένας δεύτερος περιορισμός αφορά την πολιτική early-exit της Ενότητας 4.6.3. Σε όλα τα early-exit πειράματα χρησιμοποιείται η ίδια τιμή threshold, ώστε οι μετρικές να παραμένουν συγκρίσιμες ως προς το μοντέλο και την τοπολογία εκτέλεσης. Παράλληλα, όμως, δεν εξετάζεται η επίδραση διαφορετικών thresholds στον συμβιβασμό ανάμεσα σε ακρίβεια, latency και ποσοστά εξόδου.

Το distributed inference αξιολογείται με batch size ίσο με 1. Η επιλογή αυτή ταιριάζει με σενάρια online inference, όπου κάθε αίτημα εξυπηρετείται ξεχωριστά και η απόφαση early-exit λαμβάνεται ανά δείγμα. Ταυτόχρονα, περιορίζει την ερμηνεία των αποτελεσμάτων για περιπτώσεις όπου χρησιμοποιούνται μεγαλύτερα batches. Σε τέτοια σενάρια, η καλύτερη αξιοποίηση του υπολογιστικού υλικού μπορεί να αλλάξει το

throughput, ενώ η δυναμική φύση των exits δυσκολεύει τη συσκευασία πολλών δειγμάτων στην ίδια ροή εκτέλεσης, αφού κάθε δείγμα μπορεί να τερματίσει σε διαφορετικό βάθος.

Οι μετρικές επικοινωνίας και ενέργειας πρέπει επίσης να διαβαστούν ως μετρήσεις του συγκεκριμένου runtime και όχι ως πλήρης φυσική χαρακτηριστικοποίηση του δικτύου ή της κατανάλωσης. Το communication/runtime overhead μετριέται σε επίπεδο εφαρμογής και περιλαμβάνει το κόστος σειριοποίησης, αποστολής, λήψης και αναμονής μέσα στο υλοποιημένο pipeline. Δεν αποτελεί packet-level μέτρηση του δικτύου και δεν απομονώνει χαμηλού επιπέδου φαινόμενα, όπως retransmissions ή λεπτομέρειες του TCP stack. Αντίστοιχα, οι ενεργειακές μετρικές που παράγονται με το CodeCarbon βασίζονται σε εκτιμήσεις λογισμικού και επηρεάζονται από την πλατφόρμα, το τρέχον system load και την ακρίβεια του διαθέσιμου μοντέλου εκτίμησης. Συνεπώς, είναι χρήσιμες για συγκριτική ανάλυση μεταξύ πειραμάτων στο ίδιο setup, αλλά δεν πρέπει να ερμηνεύονται ως απόλυτες εργαστηριακές μετρήσεις ισχύος.

Τέλος, τα συμπεράσματα αφορούν τη συγκεκριμένη τετρακόμβη υποδομή Raspberry Pi που χρησιμοποιήθηκε. Το cluster αποτελεί ρεαλιστικό παράδειγμα περιορισμένης edge υποδομής, με μικρό κόστος, περιορισμένη μνήμη και CPU-only εκτέλεση. Επιπλέον, οι κόμβοι του cluster είναι ομογενείς, δηλαδή ταυτόσημοι ως προς το διαθέσιμο υλικό. Σε πιο ρεαλιστικές edge υποδομές, όμως, το cluster μπορεί να είναι ετερογενές, με κόμβους διαφορετικής υπολογιστικής ισχύος, μνήμης, ενεργειακής κατανάλωσης, διαθέσιμων επιταχυντών ή δικτυακής συνδεσιμότητας. Επομένως, η σχέση ανάμεσα σε υπολογιστικό κόστος και κόστος επικοινωνίας αποτυπώνεται μόνο για τη συγκεκριμένη σύνθεση υλικού και δικτύου, και η εργασία δεν ισχυρίζεται ότι οι αριθμητικές τιμές των μετρικών γενικεύονται αυτούσιες σε κάθε edge περιβάλλον.

Οι περιορισμοί αυτοί δεν αναιρούν τη χρησιμότητα της μελέτης, αλλά ορίζουν το πλαίσιο μέσα στο οποίο πρέπει να διαβαστούν τα πειραματικά αποτελέσματα του επόμενου κεφαλαίου.

Κεφάλαιο 5

Πειραματικά Αποτελέσματα

Το παρόν κεφάλαιο παρουσιάζει τα πειραματικά αποτελέσματα της εργασίας. Εδώ δίνεται έμφαση στα αποτελέσματα των μετρήσεων και στη σύγκριση των διαφορετικών τρόπων εκτέλεσης. Τα αποτελέσματα οργανώνονται με βάση τις τρεις ομάδες πειραμάτων που ορίστηκαν: single-model εκτέλεση, αφελή multi-model κατανομή και memory-aware multi-model τοποθέτηση. Πέρα από το αρχικό test set του CIFAR-10, παρουσιάζεται και συμπληρωματική αξιολόγηση στο CIFAR-10.1, ώστε να εξεταστεί αν οι τάσεις που προκύπτουν από τα ίδια εκπαιδευμένα μοντέλα διατηρούνται και σε ανεξάρτητο σύνολο γενίκευσης.

5.1 Οργάνωση της Πειραματικής Αξιολόγησης

Τα πειραματικά αποτελέσματα παρουσιάζονται με σειρά αυξανόμενης πολυπλοκότητας. Αρχικά εξετάζεται η συμπεριφορά ενός μόνο μοντέλου, ώστε να απομονωθεί η επίδραση των early exits και της κατανεμημένης εκτέλεσης. Στη συνέχεια μελετάται η περίπτωση δύο λογικών instances του ίδιου μοντέλου, όπου και τα δύο ακολουθούν την ίδια pipeline διαδρομή πάνω στους workers. Η διάταξη αυτή χρησιμοποιείται ως αφελής multi-model βάση σύγκρισης. Τέλος, αξιολογούνται τρεις memory-aware στρατηγικές τοποθέτησης, οι οποίες κρατούν σταθερή την αρχιτεκτονική του μοντέλου και αλλάζουν μόνο την αντιστοίχιση των partitions στους workers.

Οι πίνακες χρησιμοποιούνται για τις ακριβείς αριθμητικές τιμές, ενώ τα διαγράμματα χρησιμοποιούνται για την ανάδειξη των τάσεων. Συγκεκριμένα, τα barplots του χρόνου και του throughput δείχνουν τη συνολική συμπεριφορά κάθε τοπολογίας, τα utilization plots δείχνουν αν κάποιος worker γίνεται bottleneck, τα exit distribution plots εξηγούν ποιο μέρος του μοντέλου ενεργοποιείται στην πράξη, και τα ενεργειακά plots εξετάζουν αν οι επιταχύνσεις συνοδεύονται από μικρότερο κόστος ανά inference εργασία.

5.2 Experiment 1: Single-Model Εκτέλεση

Το Experiment 1 λειτουργεί ως βάση αναφοράς για την υπόλοιπη αξιολόγηση. Η ομάδα αυτή εξετάζει την εκτέλεση ενός μόνο μοντέλου σε τρία σενάρια: single-node baseline

χωρίς early exits, single-node early-exit εκτέλεση, και κατανεμημένη early-exit εκτέλεση σε τρεις workers. Με αυτόν τον τρόπο μπορούν να διαχωριστούν δύο ερωτήματα: πρώτον, αν ο early-exit μηχανισμός μειώνει τον χρόνο inference σε σχέση με το πλήρες ResNet, και δεύτερον, αν η κατανομή του ίδιου μοντέλου σε πολλούς workers διατηρεί ή χάνει αυτό το πλεονέκτημα όταν εισάγεται κόστος επικοινωνίας.

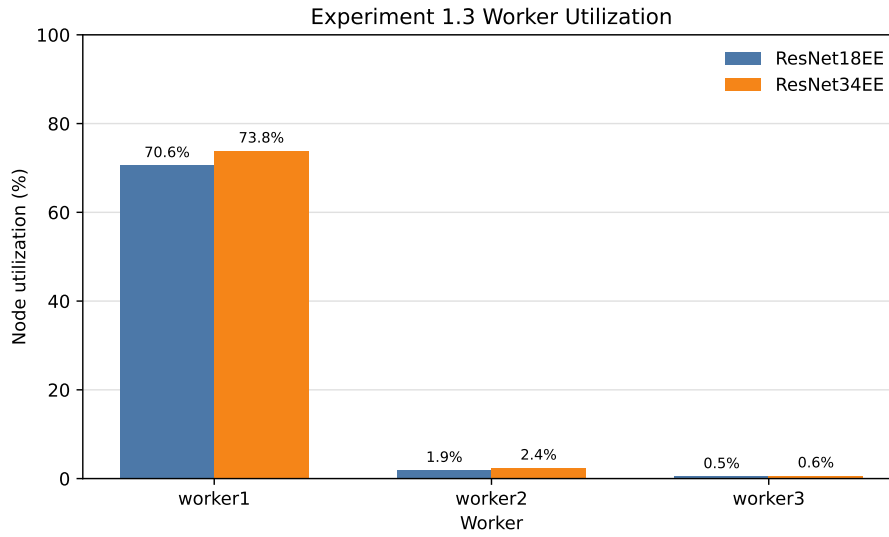
Πίνακας 5.1: Experiment 1 core metrics.

experiment	accuracy	avg latency	throughput	model	exit0	exit1	exit2	exit3
1.1	89.74	0.4466	2.191	ResNet18 Baseline	-	-	-	-
1.2	85.32	0.2659	3.630	ResNet18EE	68.18%	24.04%	6.15%	1.63%
1.3	85.32	0.3582	2.704	ResNet18EE	68.18%	24.04%	6.15%	1.63%
1.1	88.57	0.8637	1.145	ResNet34 Baseline	-	-	-	-
1.2	86.38	0.3906	2.490	ResNet34EE	78.02%	17.39%	2.99%	1.60%
1.3	86.38	0.4149	2.344	ResNet34EE	78.02%	17.39%	2.99%	1.60%

Ο Πίνακας 5.1 δείχνει ότι η μετάβαση από το baseline ResNet στο early-exit μοντέλο μειώνει ουσιαστικά τον μέσο χρόνο inference. Για το ResNet-18, ο χρόνος μειώνεται από 0.4466 s στο Exp1.1 σε 0.2659 s στο Exp1.2, ενώ το throughput αυξάνεται από 2.191 σε 3.630 tasks/s. Αντίστοιχα, για το ResNet-34, ο χρόνος μειώνεται από 0.8637 s σε 0.3906 s και το throughput αυξάνεται από 1.145 σε 2.490 tasks/s. Η βελτίωση αυτή συνοδεύεται από μείωση της ακρίβειας σε σχέση με το πλήρες baseline, αλλά η ακρίβεια των early-exit μοντέλων παραμένει σταθερή μεταξύ single-node και distributed εκτέλεσης. Αυτό είναι αναμενόμενο, αφού στις δύο αυτές περιπτώσεις αλλάζει μόνο ο τρόπος εκτέλεσης και όχι το εκπαιδευμένο μοντέλο ή η πολιτική εξόδου.

Η κατανεμημένη εκτέλεση του Exp1.3 έχει διαφορετική συμπεριφορά. Για το ResNet-18EE, το μέσο latency αυξάνεται από 0.2659 s σε 0.3582 s σε σχέση με το single-node early-exit run, ενώ το throughput μειώνεται από 3.630 σε 2.704 tasks/s. Για το ResNet-34EE η μεταβολή είναι μικρότερη, από 0.3906 s σε 0.4149 s, με throughput 2.344 tasks/s στο distributed run. Η διαφορά αυτή δείχνει ότι η κατανεμημένη εκτέλεση δεν είναι αυτόματα ταχύτερη. Όταν τα partitions είναι σχετικά ελαφριά και μεγάλο ποσοστό των δειγμάτων τερματίζει νωρίς, το κόστος αποστολής ενδιάμεσων ενεργοποιήσεων και συντονισμού μπορεί να αντισταθμίσει μέρος του κέρδους από την κατανομή του υπολογισμού.

Εστιάζοντας τώρα στην κατανομή των εξόδων έχουμε ότι η πλειονότητα των δειγμάτων τερματίζει στα πρώτα exits, όπως φαίνεται στον Πίνακα 5.1. Στο ResNet18EE, το 68.18% των δειγμάτων τερματίζει στο πρώτο exit και το 24.04% στο δεύτερο. Μόνο το 6.15% φτάνει στο τρίτο exit και το 1.63% στην τελική έξοδο. Στο ResNet34EE η συγκέντρωση στα πρώτα exits είναι ακόμη ισχυρότερη, με 78.02% στο πρώτο exit και 17.39% στο δεύτερο. Επομένως, η πλειονότητα των εικόνων δεν ενεργοποιεί τα βαθύτερα partitions του μοντέλου. Ως προς τον φόρτο των partitions, όμως, πρέπει να διακριθεί το πού τερματίζει ένα δείγμα από το ποια partitions εκτελούνται. Το Partition 0 εκτελείται για το 100% των εργασιών inference, επειδή προηγείται όλων των αποφάσεων εξόδου. Για το ResNet18EE, το 92.22% των εργασιών inference τερματίζει μέσα στο Partition 0, ενώ το 7.78% συνεχίζει στο Partition 1 και μόνο το 1.63% φτάνει τελικά στο Partition 2. Για το ResNet34EE, τα αντίστοιχα ποσοστά είναι 95.41% τερματισμός στο Partition 0, 4.59% εκτέλεση του Partition 1 και 1.60% εκτέλεση του Partition 2. Η ανισορροπία αυτή είναι σημαντική για την αξιολόγηση της κατανομής του φόρτου στους workers.



Εικόνα 5.1: Αξιοποίηση workers στο distributed single-model run του Experiment 1.3.

Η Εικόνα 5.1 επιβεβαιώνει αυτή την ανισορροπία. Ο worker1, που δέχεται όλες τις εργασίες inference και εκτελεί το αρχικό partition, εμφανίζει utilization 70.56% για το ResNet18EE και 73.84% για το ResNet34EE. Αντίθετα, οι worker2 και worker3 παραμένουν σχεδόν ανενεργοί σε επίπεδο application-level utilization. Η εικόνα αυτή είναι σημαντική για τη συνέχεια της αξιολόγησης: η early-exit αρχιτεκτονική μειώνει τον υπολογισμό, αλλά παράλληλα δημιουργεί έντονα μη ομοιόμορφη ροή μέσα στο distributed pipeline. Επομένως, η απλή διάσπαση του μοντέλου σε συνεχόμενα partitions δεν αρκεί από μόνη της για καλή αξιοποίηση ενός edge cluster.

5.3 Experiment 2: Αφελής Multi-Model Κατανομή

Το Experiment 2 επεκτείνει το distributed σενάριο σε δύο λογικά instances του ίδιου early-exit μοντέλου. Το πείραμα αυτό δεν εισάγει νέα αρχιτεκτονική ή διαφορετική πολιτική εξόδου. Αντίθετα, χρησιμοποιείται ως αφελές multi-model baseline: εξετάζει τι συμβαίνει όταν δύο παράλληλες ροές inference τοποθετούνται πάνω στην ίδια κατανομημένη διαδρομή χωρίς καμία προσπάθεια να εξισορροπίσουμε τα partitions.

Ο Πίνακας 5.2 συγκρίνει το Experiment 2 με το distributed single-model run του Experiment 1.3. Στο Experiment 2 η στήλη συνολικού throughput αφορά και τα δύο logical model instances μαζί.

Η ακρίβεια παραμένει ίδια με το Experiment 1.3, επειδή δεν αλλάζει ούτε το εκπαιδευμένο μοντέλο ούτε η πολιτική εξόδου. Τα exit ratios παραμένουν επίσης ίδια και δεν επαναλαμβάνονται στον πίνακα, αφού το Experiment 2 εκτελεί δύο αντίγραφα της ίδιας early-exit ροής. Η διαφορά εμφανίζεται στις runtime μετρικές: η προσθήκη δεύτερου logical instance αυξάνει το συνολικό throughput μόνο κατά 6.5% για το ResNet18EE και κατά 5.3% για το ResNet34EE, ενώ το throughput ανά instance μειώνεται περίπου στο μισό. Ταυτόχρονα, το μέσο latency σχεδόν διπλασιάζεται και στα δύο μοντέλα.

Πίνακας 5.2: Σύγκριση του Experiment 1.3 με το αφελές multi-model Experiment 2.

Experiment	Model	Accuracy (%)	Avg lat. (s)	Thr. total (tasks/s)	Thr./inst. (tasks/s)	Worker util. W1/W2/W3 (%)
1.3	ResNet18EE	85.32	0.3582	2.704	2.704	70.56/1.88/0.47
2	ResNet18EE	85.32	0.6940	2.881	1.440	79.81/1.97/0.49
1.3	ResNet34EE	86.38	0.4149	2.344	2.344	73.84/2.40/0.64
2	ResNet34EE	86.38	0.8101	2.468	1.234	82.48/2.33/0.62

Η αιτία φαίνεται από την αξιοποίηση των workers. Ο worker1 παραμένει ο κυρίαρχος ενεργός worker, ενώ οι worker2 και worker3 παραμένουν σχεδόν ανενεργοί, επειδή και τα δύο logical instances στέλνουν όλες τις εργασίες inference στον πρώτο worker πριν ενεργοποιηθεί οποιαδήποτε early-exit κεφαλή.

Επομένως, η multi-model εκτέλεση δεν κλιμακώνεται αποτελεσματικά με απλή επανάληψη της ίδιας distributed pipeline διαδρομής. Το Experiment 2 αποτελεί έτσι το κατάλληλο αφελές baseline για την αξιολόγηση των memory-aware τοποθετήσεων του Experiment 3.

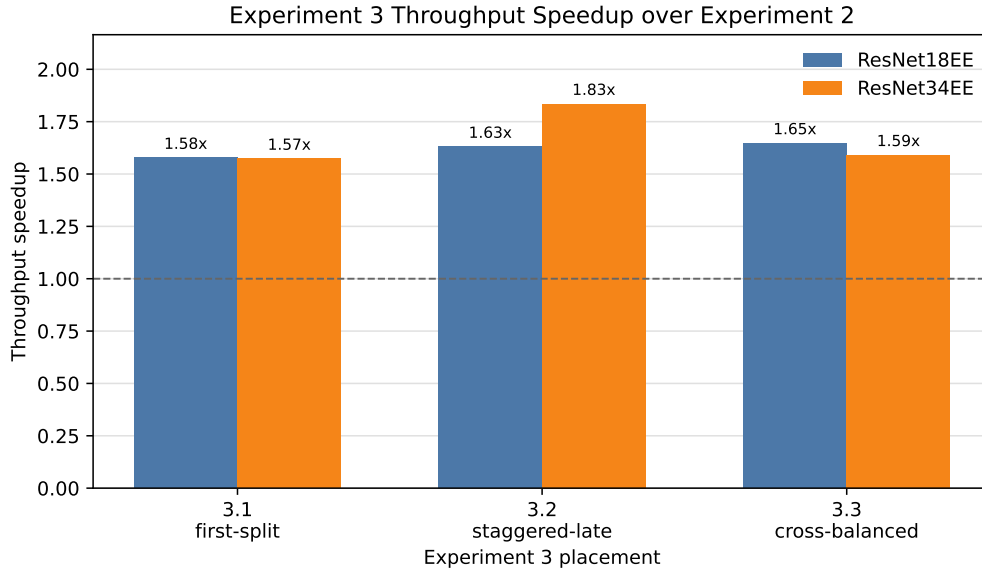
5.4 Experiment 3: Memory-Aware Placement

Το Experiment 3 εξετάζει αν η έξυπνη τοποθέτηση των partitions μπορεί να βελτιώσει τη multi-model εκτέλεση χωρίς αλλαγή στο μοντέλο ή στην early-exit πολιτική. Η σύγκριση με το Experiment 2 είναι άμεση: και στις δύο περιπτώσεις εκτελούνται δύο λογικά instances του ίδιου early-exit μοντέλου πάνω στους ίδιους τρεις workers. Η διαφορά είναι ότι στο Experiment 3 τα πρώτα partitions των δύο instances δεν συγκεντρώνονται στον ίδιο worker. Αντίθετα, οι τρεις στρατηγικές τοποθέτησης επιχειρούν να μοιράσουν τα συχνά ενεργά partitions και να μειώσουν τη συμφόρηση που παρατηρήθηκε στο αφελές baseline. Η τοπολογία φαίνεται στο Εικόνα 4.8.

Πίνακας 5.3: Experiment 3 latency και throughput σε σχέση με το Experiment 2.

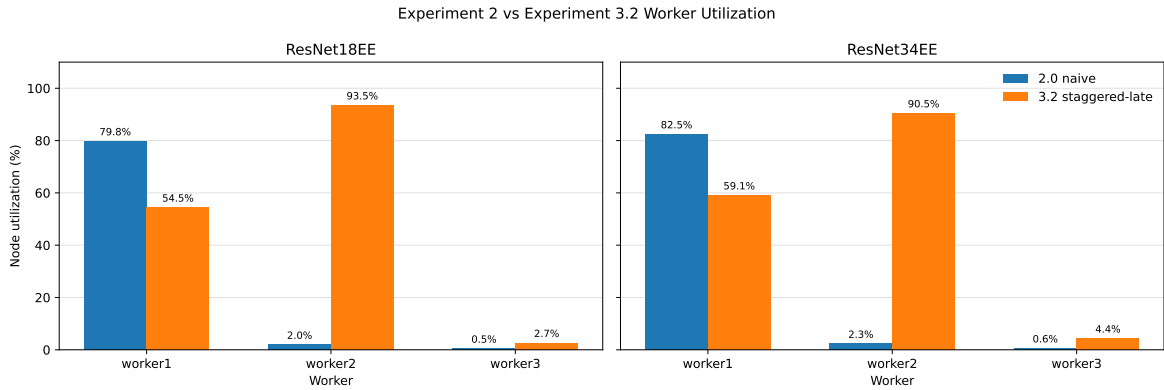
Experiment	Placement	Model	Avg Latency (s)	Throughput	Inference Time Reduction (%)
3.1	first-split	ResNet18EE	0.4398	4.544	36.62
3.2	staggered-late	ResNet18EE	0.4256	4.696	38.67
3.3	cross-balanced	ResNet18EE	0.4212	4.745	39.31
3.1	first-split	ResNet34EE	0.5149	3.882	36.44
3.2	staggered-late	ResNet34EE	0.4413	4.529	45.52
3.3	cross-balanced	ResNet34EE	0.5092	3.926	37.14

Ο Πίνακας 5.3 και η Εικόνα 5.2 δείχνουν ότι όλες οι memory-aware στρατηγικές βελτιώνουν σημαντικά το throughput σε σχέση με το αφελές Experiment 2. Αρχικά, για το ResNet18EE, τα speedups κυμαίνονται από 1.58x έως 1.65x. Η καλύτερη τιμή παρατηρείται στο Exp3.3, με throughput 4.745 tasks/s και μείωση μέσου χρόνου inference κατά 39.31%. Το Exp3.2 βρίσκεται πολύ κοντά, με speedup 1.63x και μείωση χρόνου 38.67%.



Εικόνα 5.2: Throughput των στρατηγικών του Experiment 3 ως speedup σε σχέση με το Experiment 2.

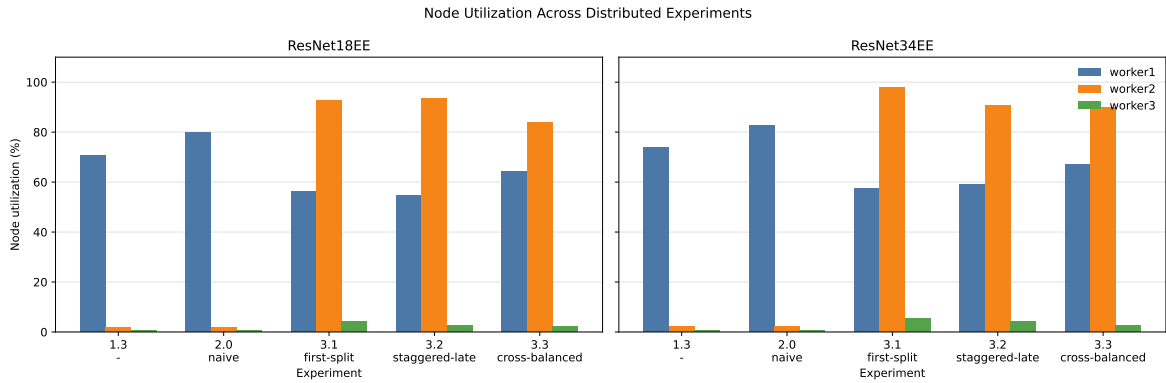
Για το ResNet34EE, ωστόσο, η εικόνα διαφοροποιείται πιο έντονα. Το Exp3.2 πετυχαίνει throughput 4.529 tasks/s, δηλαδή speedup 1.83x σε σχέση με το Experiment 2, και μείωση μέσου χρόνου inference κατά 45.52%. Τα Exp3.1 και Exp3.3 επίσης βελτιώνουν το baseline, αλλά μένουν σε speedups 1.57x και 1.59x αντίστοιχα. Επομένως, αν η επιλογή γίνει με βάση τη συνολική συμπεριφορά στα δύο βάθη μοντέλου, το Exp3.2 είναι η πιο ισχυρή στρατηγική. Δεν είναι απόλυτα καλύτερο σε κάθε μεμονωμένη μέτρηση, αλλά δίνει την καλύτερη συνολική ισορροπία, ειδικά επειδή βελτιώνει δραστικά το βαθύτερο ResNet34EE.



Εικόνα 5.3: Σύγκριση αξιοποίησης workers ανάμεσα στο αφελές Experiment 2 και στο καλύτερο συνολικά Exp3.2.

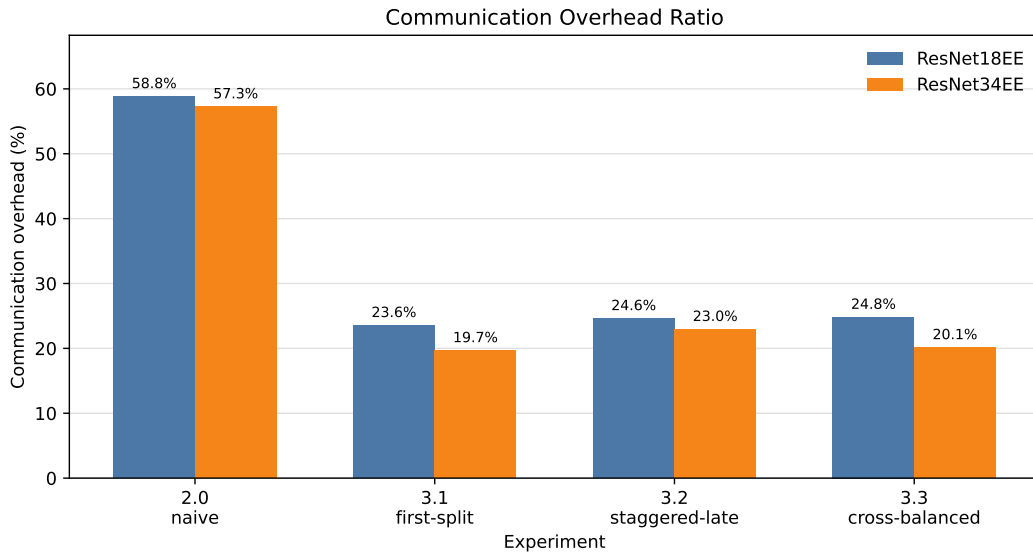
Η Εικόνα 5.3 εξηγεί γιατί η αλλαγή τοποθέτησης βελτιώνει το αποτέλεσμα. Στο Experiment 2, ο worker1 συγκεντρώνει σχεδόν όλο το φορτίο, επειδή και τα δύο logical models ξεκινούν από την ίδια pipeline θέση. Στο Exp3.2, το φορτίο μεταφέρεται σε δύο ενεργούς workers. Για το ResNet18EE, ο worker1 μειώνεται από 79.8% σε 54.5%, ενώ ο worker2 φτάνει στο 93.5%. Για το ResNet34EE, ο worker1 μειώνεται από 82.5% σε 59.1%, ενώ ο worker2 φτάνει στο 90.5%. Η κατανομή αυτή δεν είναι τέλεια ισορ-

ροπημένη, επειδή ο worker2 γίνεται πλέον ο πιο απασχολημένος worker, αλλά είναι λειτουργικά καλύτερη από την πλήρη συγκέντρωση του φόρτου στον worker1.



Εικόνα 5.4: Αξιοποίηση workers σε όλα τα distributed πειράματα.

Η συνολική εικόνα του utilization στην Εικόνα 5.4 δείχνει ότι το πρόβλημα δεν περιορίζεται στο Experiment 2. Ακόμη και στο Experiment 1.3, όπου υπάρχει μόνο ένα μοντέλο, οι βαθύτεροι workers έχουν χαμηλή αξιοποίηση λόγω της κατανομής των early exits. Στο Experiment 3, όμως, οι worker1 και worker2 συμμετέχουν πλέον πιο ενεργά στην εκτέλεση. Τα Exp3.1 και Exp3.2 οδηγούν σε πολύ υψηλό worker2 utilization, ενώ το Exp3.3 δίνει πιο ήπια κατανομή μεταξύ worker1 και worker2. Παρόλα αυτά, η πιο ομοιόμορφη κατανομή δεν οδηγεί πάντοτε στο καλύτερο throughput. Αυτό δείχνει ότι η επιλογή placement πρέπει να λαμβάνει υπόψη όχι μόνο το ποσοστό απασχόλησης κάθε worker, αλλά και τη σειρά των partitions, το κόστος επικοινωνίας και το πόσο συχνά ενεργοποιείται κάθε partition.



Εικόνα 5.5: Ποσοστό communication/runtime overhead στα Experiments 2 και 3.

Η Εικόνα 5.5 δείχνει το communication/runtime overhead ως ποσοστό του end-to-end latency. Στο Experiment 2, το συνολικό communication/runtime overhead ratio είναι 58.85% για το ResNet18EE και 57.27% για το ResNet34EE. Στο Experiment 3 οι αντίστοιχες τιμές πέφτουν περίπου στην περιοχή 20–25%. Παράλληλα, αν εξεταστεί ο

απόλυτος μέσος χρόνος communication/runtime overhead ανά εργασία, η μείωση είναι ακόμη πιο έντονη: για το ResNet18EE ο μέσος χρόνος communication/runtime overhead πέφτει από 0.4084 s σε περίπου 0.104–0.105 s, δηλαδή κατά περίπου 74%, ενώ για το ResNet34EE πέφτει από 0.4640 s σε περίπου 0.101–0.103 s, δηλαδή κατά περίπου 78%. Αυτό είναι κρίσιμο, επειδή δείχνει ότι οι βελτιώσεις του Experiment 3 δεν προέρχονται απλώς από περισσότερη παράλληλη εργασία, αλλά και από πιο αποδοτικές διαδρομές εκτέλεσης.

Συνολικά, το Experiment 3 επιβεβαιώνει ότι η τοποθέτηση των partitions αποτελεί κεντρική παράμετρο για multi-model edge inference. Το αφελές baseline του Experiment 2 διατηρεί τη σωστή λειτουργία του μοντέλου, αλλά οδηγεί σε συμφόρηση, χαμηλή αξιοποίηση μέρους του cluster και υψηλό communication/runtime overhead. Οι memory-aware τοποθετήσεις μειώνουν αυτά τα προβλήματα και αυξάνουν το throughput χωρίς νέα εκπαίδευση και χωρίς αλλαγή της early-exit πολιτικής. Με βάση το συνολικό throughput και τη συμπεριφορά στα δύο μοντέλα, το Exp3.2 αποτελεί το καλύτερο συνολικό placement για τη συγκεκριμένη πειραματική υποδομή.

5.5 Ενεργειακή Αποδοτικότητα

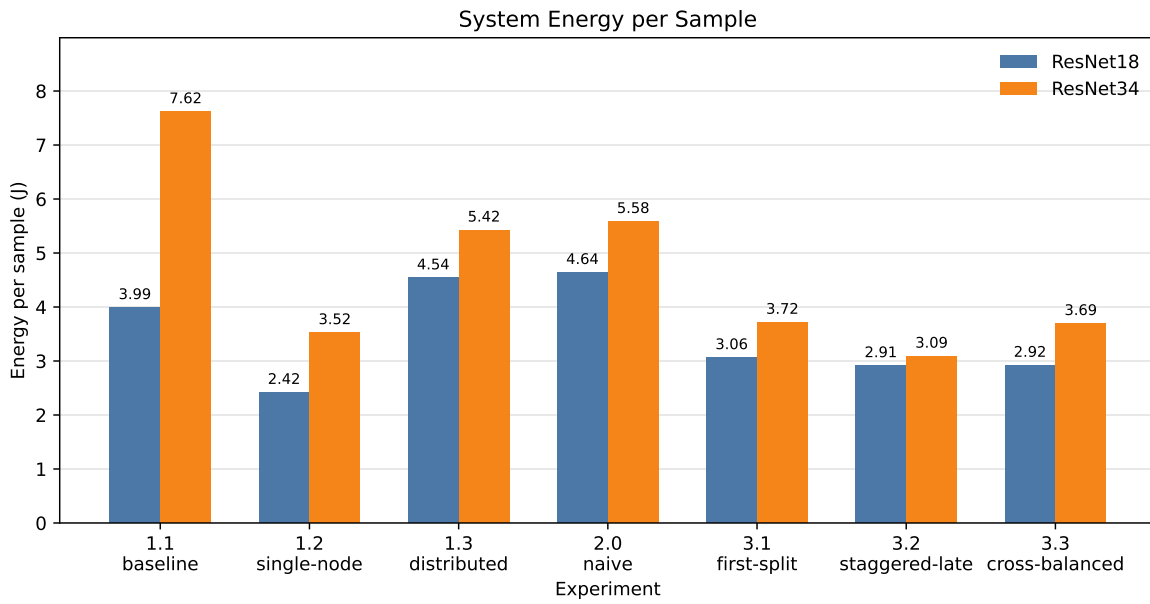
Η ενεργειακή αξιολόγηση συμπληρώνει την ανάλυση χρόνου και throughput. Σε περιορισμένα edge συστήματα, μια τοπολογία δεν είναι απαραίτητα επιθυμητή μόνο επειδή είναι ταχύτερη. Πρέπει επίσης να εξεταστεί αν το κέρδος σε χρόνο συνοδεύεται από μικρότερο ή μεγαλύτερο ενεργειακό κόστος. Για τον λόγο αυτό, η ανάλυση χρησιμοποιεί κυρίως την ενέργεια ανά inference εργασία, δηλαδή:

$$E_{\text{task}} = \frac{E_{\text{system}} \cdot 3.6 \cdot 10^6}{N_{\text{tasks}}}.$$

Στα πειράματα η ενέργεια καταγράφεται σε kWh από τους CodeCarbon trackers και μετατρέπεται σε Joule ανά inference εργασία. Για τα distributed runs, E_{system} είναι το άθροισμα της ενέργειας του master και των workers. Η κανονικοποίηση γίνεται με βάση το πλήθος των inference tasks και όχι μόνο το πλήθος μοναδικών εικόνων, επειδή τα single-model runs εκτελούν 10.000 εργασίες inference, ενώ τα multi-model runs εκτελούν 20.000.

Η Εικόνα 5.6 δείχνει ότι τα early-exit μοντέλα σε single-node εκτέλεση έχουν το μικρότερο ενεργειακό κόστος ανά inference εργασία. Για το ResNet18, το baseline Exp1.1 καταναλώνει 3.99 J ανά εργασία, ενώ το early-exit Exp1.2 μειώνει την τιμή σε 2.42 J. Για το ResNet34, η αντίστοιχη μείωση είναι από 7.62 J σε 3.52 J. Το αποτέλεσμα αυτό είναι συνεπές με τη μείωση του χρόνου inference που παρατηρήθηκε στο Experiment 1: όταν πολλά δείγματα τερματίζουν νωρίς, μειώνεται και ο υπολογισμός που απαιτείται ανά πρόβλεψη.

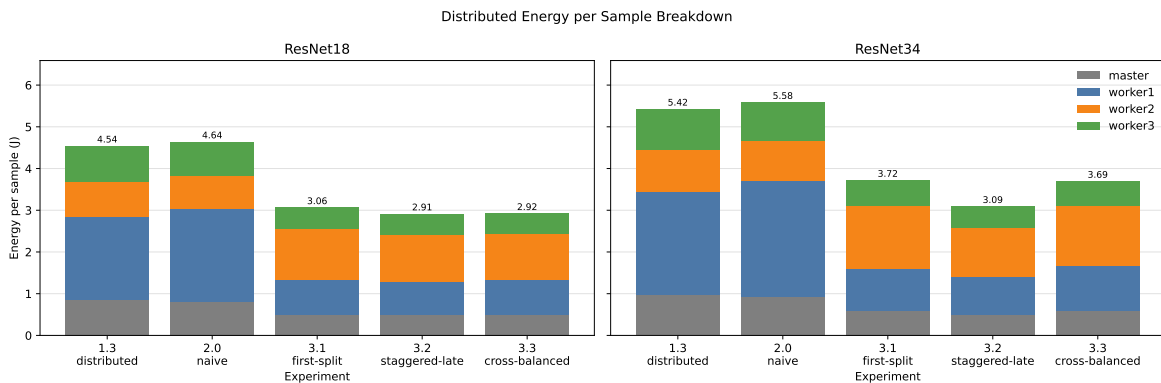
Η κατανεμημένη single-model εκτέλεση του Exp1.3 αυξάνει την ενέργεια ανά inference εργασία σε σχέση με το single-node early-exit run. Για το ResNet18EE η τιμή ανεβαίνει από 2.42 J σε 4.54 J, ενώ για το ResNet34EE από 3.52 J σε 5.42 J. Η αύξηση αυτή



Εικόνα 5.6: Ενέργεια συστήματος ανά inference εργασία για όλα τα πειράματα.

είναι αναμενόμενη, επειδή ενεργοποιείται ολόκληρο το distributed σύστημα, δηλαδή master και τρεις workers, ακόμη και όταν μεγάλο μέρος των εργασιών inference τερματίζει στα πρώτα exits. Επομένως, η κατανομή ενός μόνο μοντέλου σε πολλούς workers δεν είναι ενεργειακά αποδοτική όταν η ροή δειγμάτων δεν αξιοποιεί επαρκώς τους workers που φιλοξενούν τα βαθύτερα partitions.

Στα multi-model πειράματα, το αφελές Experiment 2 έχει υψηλή ενέργεια ανά inference εργασία: 4.64 J για το ResNet18EE και 5.58 J για το ResNet34EE. Οι memory-aware στρατηγικές του Experiment 3 μειώνουν αισθητά αυτές τις τιμές. Το Exp3.2 φτάνει στα 2.91 J για το ResNet18EE και στα 3.09 J για το ResNet34EE, δηλαδή μειώνει την ενέργεια ανά inference εργασία κατά περίπου 37% και 45% αντίστοιχα σε σχέση με το Exp2. Το Exp3.3 είναι σχεδόν ισοδύναμο με το Exp3.2 για το ResNet18EE, αλλά μένει υψηλότερα για το ResNet34EE. Αυτό ενισχύει την εικόνα ότι το Exp3.2 είναι το πιο σταθερό overall placement.



Εικόνα 5.7: Ανάλυση ενέργειας ανά inference εργασία σε master και workers για τα distributed πειράματα.

Η Εικόνα 5.7 αναλύει την ενέργεια ανά inference εργασία και ανά master/worker

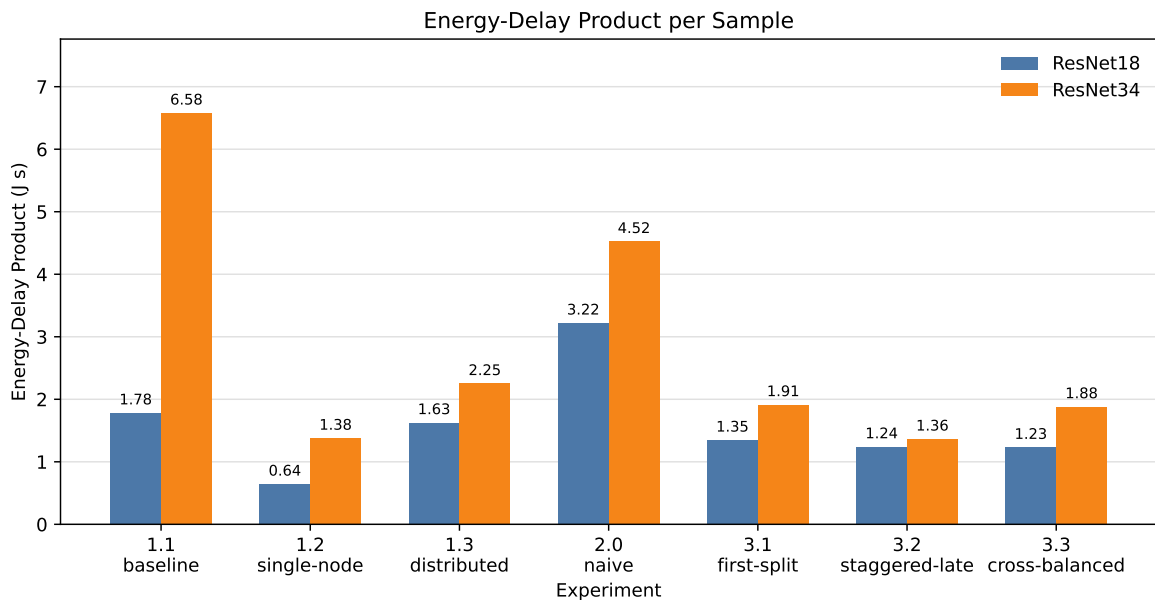
για τα distributed πειράματα. Στο Exp1.3 και στο Exp2, μεγάλο μέρος της ενέργειας αποδίδεται στον worker1, ο οποίος εκτελεί το αρχικό partition και δέχεται όλες τις εργασίες inference. Αυτό συμφωνεί με τα utilization plots, όπου ο worker1 είναι ο κύριος ενεργός worker. Παρότι οι worker2 και worker3 έχουν μικρή application-level αξιοποίηση, εξακολουθούν να συνεισφέρουν στη συνολική ενέργεια του συστήματος, επειδή το distributed setup παραμένει ενεργό καθ' όλη τη διάρκεια του run.

Στο Experiment 3 η εικόνα αλλάζει. Η ενέργεια μεταφέρεται περισσότερο προς τον worker2, ειδικά στα Exp3.1 και Exp3.2, όπου ο worker2 φιλοξενεί συχνά ενεργά partitions. Παρά τη μετατόπιση αυτή, το συνολικό ύψος των stacked bars είναι χαμηλότερο από το Exp2. Αυτό σημαίνει ότι η καλύτερη τοποθέτηση δεν μοιράζει απλώς την ίδια ποσότητα ενέργειας σε διαφορετικούς workers, αλλά μειώνει και το συνολικό ενεργειακό κόστος ανά inference εργασία. Η μείωση αυτή συνδέεται με τον μικρότερο συνολικό χρόνο run και με το χαμηλότερο communication/runtime overhead που παρουσιάστηκε στην προηγούμενη ενότητα.

Για να συνδυαστούν η ενέργεια και ο χρόνος σε μία μετρική, χρησιμοποιείται το Energy-Delay Product ανά inference εργασία:

$$EDP = E_{\text{task}} \cdot T_{\text{avg_latency}}.$$

Η μετρική αυτή ευνοεί τοπολογίες που είναι ταυτόχρονα ενεργειακά οικονομικές και γρήγορες. Μια εκτέλεση με χαμηλή ενέργεια αλλά μεγάλο χρόνο, ή αντίστροφα με μικρό χρόνο αλλά υψηλή ενέργεια, δεν θα εμφανιστεί απαραίτητα ως βέλτιστη.



Εικόνα 5.8: Energy-Delay Product ανά inference εργασία για όλα τα πειράματα.

Η Εικόνα 5.8 δείχνει ότι το αφελές Experiment 2 έχει το χειρότερο EDP ανάμεσα στα early-exit multi-model σενάρια. Για το ResNet18EE το EDP είναι 3.22 J s, ενώ για το ResNet34EE είναι 4.52 J s. Στο Exp3.2 οι τιμές μειώνονται σε 1.24 J s και 1.36 J s αν-

τίστοιχα. Η μείωση αυτή είναι ιδιαίτερα σημαντική για το ResNet34EE, όπου το Exp3.2 συνδυάζει τη μεγαλύτερη βελτίωση throughput με τη χαμηλότερη ενέργεια ανά inference εργασία μεταξύ των distributed multi-model στρατηγικών.

Το ResNet18EE παρουσιάζει μια μικρή διαφοροποίηση: το Exp3.3 έχει οριακά χαμηλότερο EDP από το Exp3.2, επειδή πετυχαίνει τον μικρότερο μέσο χρόνο inference για το συγκεκριμένο μοντέλο. Ωστόσο, η διαφορά είναι αμελητέα, ενώ στο ResNet34EE το Exp3.2 υπερिशύει ξεκάθαρα. Επομένως, και από την ενεργειακή σκοπιά, το Exp3.2 παραμένει η πιο ισορροπημένη επιλογή όταν λαμβάνονται υπόψη και τα δύο μοντέλα.

5.6 Συμπληρωματική Αξιολόγηση στο CIFAR-10.1

Το CIFAR-10.1 χρησιμοποιείται ως ανεξάρτητο σύνολο γενίκευσης για τα ίδια μοντέλα και τα ίδια checkpoints που αξιολογήθηκαν στο CIFAR-10. Δεν γίνεται καμία νέα εκπαίδευση, fine-tuning ή αλλαγή του entropy threshold. Επομένως, οι διαφορές που εμφανίζονται στην ακρίβεια αποδίδονται στη μετατόπιση του dataset, ενώ οι διαφορές σε latency, throughput και ενέργεια δείχνουν πώς συμπεριφέρεται η ίδια υποδομή εκτέλεσης σε πιο απαιτητική αλλά ανεξάρτητη ροή δειγμάτων.

Πίνακας 5.4: Experiment 1 core metrics (CIFAR-10.1).

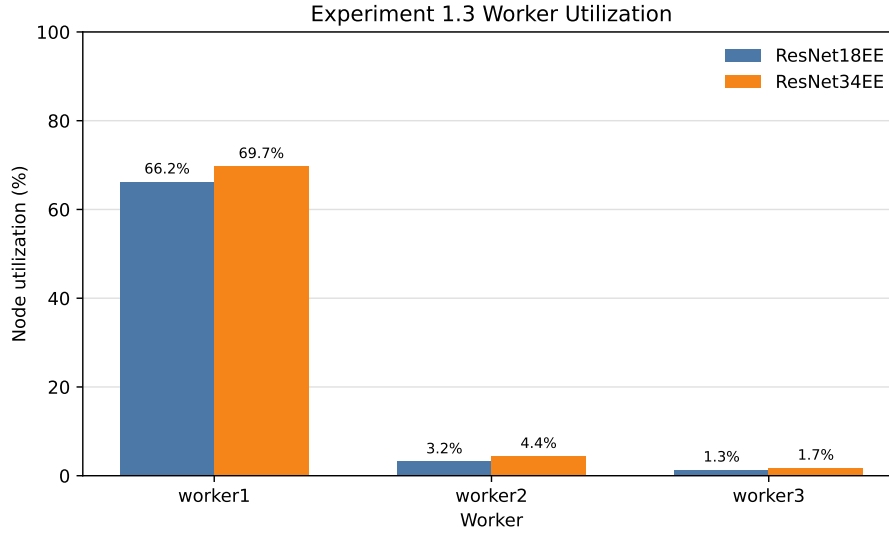
Experiment	Accuracy	Avg Latency (s)	Throughput	Model	Exit 0	Exit 1	Exit 2	Exit 3
1.1	81.95	0.4386	2.231	ResNet18 Baseline	-	-	-	-
1.2	75.45	0.3543	2.736	ResNet18EE	54.35%	32.55%	9.40%	3.70%
1.3	75.45	0.3440	2.804	ResNet18EE	54.35%	32.55%	9.40%	3.70%
1.1	78.95	1.0109	0.978	ResNet34 Baseline	-	-	-	-
1.2	77.35	0.4826	2.022	ResNet34EE	65.60%	25.50%	4.85%	4.05%
1.3	77.35	0.4192	2.316	ResNet34EE	65.60%	25.50%	4.85%	4.05%

Ο Πίνακας 5.4 δείχνει ότι η ακρίβεια στο CIFAR-10.1 είναι χαμηλότερη από την αντίστοιχη στο CIFAR-10, όπως είναι αναμενόμενο για ανεξάρτητο test set χωρίς επανεκπαίδευση. Τα baseline μοντέλα φτάνουν σε 81.95% για το ResNet-18 και 78.95% για το ResNet-34, ενώ τα early-exit μοντέλα φτάνουν σε 75.45% και 77.35% αντίστοιχα. Παρά τη μείωση της ακρίβειας, η λειτουργική ισοδυναμία μεταξύ single-node και distributed εκτέλεσης διατηρείται: το ίδιο early-exit μοντέλο δίνει την ίδια ακρίβεια στο Exp1.2 και στο Exp1.3.

Σε επίπεδο χρόνου, το CIFAR-10.1 διατηρεί το βασικό πλεονέκτημα του early-exit μηχανισμού. Για το ResNet18EE, ο μέσος χρόνος του single-node early-exit run είναι 0.3543 s, έναντι 0.4386 s για το baseline ResNet-18. Για το ResNet34EE, ο χρόνος είναι 0.4826 s, έναντι 1.0109 s για το baseline ResNet-34. Στο distributed Exp1.3, οι χρόνοι είναι 0.3440 s και 0.4192 s αντίστοιχα, δηλαδή στο συγκεκριμένο CIFAR-10.1 run η κατανομημένη εκτέλεση δεν επιβαρύνει το latency του single-model σε σχέση με το Exp1.2.

Στην κατανομή των exits παρατηρείται μετατόπιση προς βαθύτερες εξόδους σε σχέση με το CIFAR-10, χωρίς όμως να αλλάζει η βασική μορφή της ροής. Στο ResNet18EE, το ποσοστό του πρώτου exit μειώνεται από 68.18% σε 54.35%, ενώ αυξάνονται τα ποσοστά

στα επόμενα exits. Αντίστοιχα, στο ResNet34EE το πρώτο exit μειώνεται από 78.02% σε 65.60%. Αυτό δείχνει ότι το ανεξάρτητο test set περιέχει σχετικά περισσότερα δείγματα που απαιτούν βαθύτερη επεξεργασία. Παρόλα αυτά, η πλειονότητα των εργασιών εξακολουθεί να τερματίζεται στα δύο πρώτα exits: 86.90% για το ResNet18EE και 91.10% για το ResNet34EE. Άρα τα βαθύτερα partitions ενεργοποιούνται συχνότερα από ό,τι στο CIFAR-10, αλλά παραμένουν δευτερεύοντα σε σχέση με το αρχικό τμήμα του μοντέλου.



Εικόνα 5.9: Αξιοποίηση workers στο distributed single-model run του Experiment 1.3 για CIFAR-10.1.

Η Εικόνα 5.9 επιβεβαιώνει την ίδια ανισορροπία που εμφανίστηκε και στο CIFAR-10, με τη μικρή διαφορά ότι ο φόρτος κατανέμεται ελαφρώς πιο ισορροπημένα, καθώς παρατηρείται λίγο μεγαλύτερο utilization στους υπόλοιπους workers. Ο worker1 παραμένει ο κύριος ενεργός worker, με utilization 66.22% για το ResNet18EE και 69.70% για το ResNet34EE, ενώ οι worker2 και worker3 παραμένουν σε πολύ χαμηλότερα επίπεδα. Αυτό δείχνει ότι η μετάβαση στο CIFAR-10.1 αλλάζει την ακρίβεια και τα απόλυτα χρονικά μεγέθη, αλλά όχι τη βασική δομή της ροής μέσα στο distributed single-model pipeline.

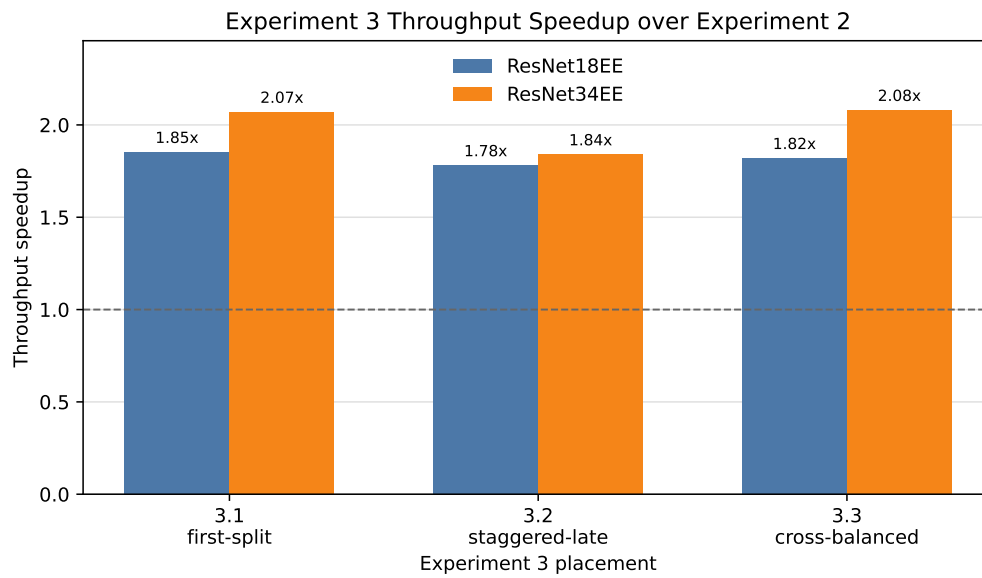
Πίνακας 5.5: Σύγκριση του Experiment 1.3 με το αφελές multi-model Experiment 2 στο CIFAR-10.1.

Experiment	Model	Accuracy (%)	Avg lat. (s)	Thr. total (tasks/s)	Thr./inst. (tasks/s)	Worker util. W1/W2/W3 (%)
1.3	ResNet18EE	75.45	0.3440	2.804	2.804	66.22/3.16/1.31
2	ResNet18EE	75.45	0.7043	2.838	1.419	154.65/5.68/2.14
1.3	ResNet34EE	77.35	0.4192	2.316	2.316	69.70/4.38/1.73
2	ResNet34EE	77.35	0.8922	2.241	1.120	160.83/8.26/2.81

Ο Πίνακας 5.5 δείχνει ότι το αφελές multi-model σενάριο συμπεριφέρεται στο CIFAR-10.1 όπως και στο CIFAR-10: η ακρίβεια μένει αμετάβλητη, αλλά η προσθήκη δεύτερου logical instance δεν οδηγεί σε ανάλογη αύξηση της συνολικής χωρητικότητας. Το throughput ανά instance μειώνεται περίπου στο μισό, ενώ ο worker1 γίνεται ακόμη πιο έντονο bottleneck. Άρα το πρόβλημα της αφελούς κοινής διαδρομής δεν είναι ιδιαιτερότητα του αρχικού test set.

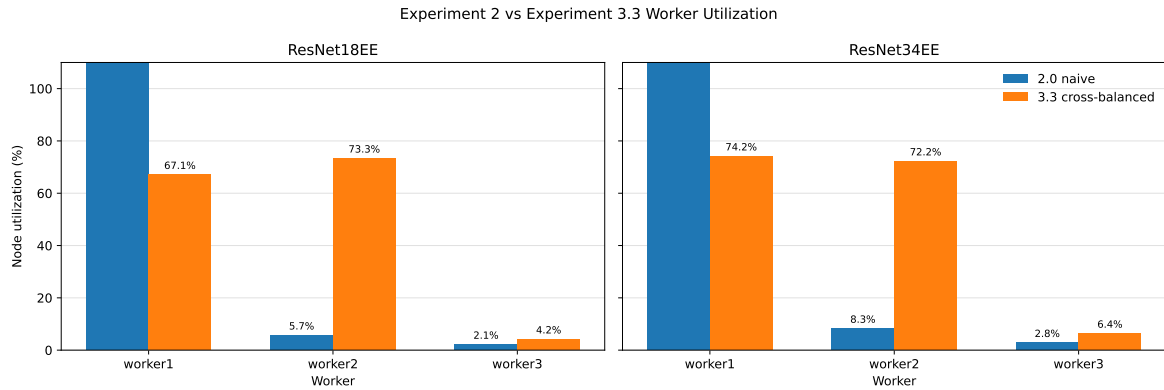
Πίνακας 5.6: Experiment 3 latency και throughput σε σχέση με το Experiment 2 στο CIFAR-10.1.

Experiment	Placement	Model	Avg Latency (s)	Throughput	Inference Time Reduction (%)
3.1	first-split	ResNet18EE	0.3795	5.262	46.12
3.2	staggered-late	ResNet18EE	0.3948	5.058	43.94
3.3	cross-balanced	ResNet18EE	0.3864	5.168	45.14
3.1	first-split	ResNet34EE	0.4304	4.641	51.76
3.2	staggered-late	ResNet34EE	0.4842	4.125	45.73
3.3	cross-balanced	ResNet34EE	0.4282	4.664	52.00



Εικόνα 5.10: Throughput speedup των στρατηγιών του Experiment 3 σε σχέση με το Experiment 2 στο CIFAR-10.1.

Ο Πίνακας 5.6 και η Εικόνα 5.10 δείχνουν ότι οι memory-aware τοποθετήσεις βελτιώνουν και στο CIFAR-10.1 το throughput σε σχέση με το αφελές Experiment 2. Η βασική τάση είναι ίδια με το CIFAR-10: η αποφυγή της κοινής αρχικής διαδρομής αυξάνει καθαρά τη συνολική ικανότητα εξυπηρέτησης. Η κατάταξη των placements, όμως, διαφοροποιείται ελαφρά. Στο CIFAR-10.1, το Exp3.1 δίνει το μεγαλύτερο throughput για το ResNet18EE, ενώ το Exp3.3 είναι οριακά καλύτερο για το ResNet34EE. Επομένως, το συμπέρασμα δεν είναι ότι ένα placement κυριαρχεί σε όλα τα datasets, αλλά ότι η memory-aware τοποθέτηση παραμένει αναγκαία και η τελική επιλογή επηρεάζεται από τη ροή των δειγμάτων.



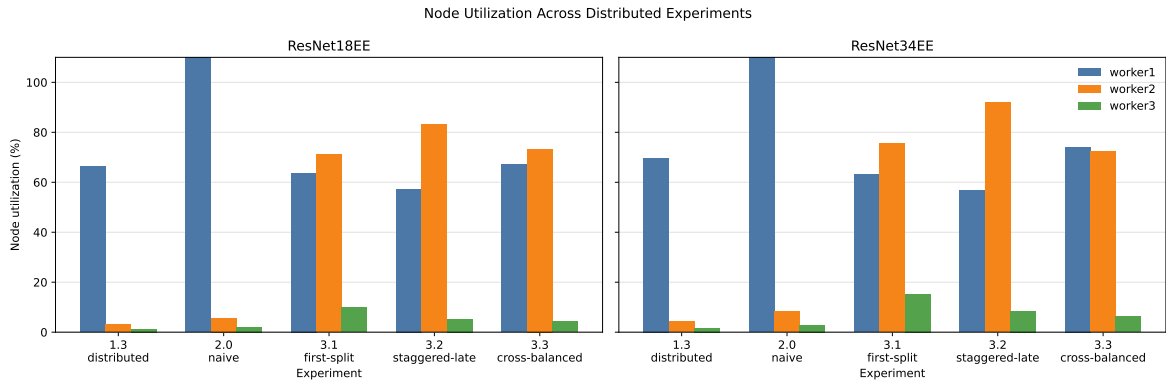
Εικόνα 5.11: Σύγκριση αξιοποίησης workers ανάμεσα στο Experiment 2 και στο Exp3.3 για CIFAR-10.1.

Η Εικόνα 5.11 δείχνει γιατί το Exp3.3 είναι η πιο πειστική συνολική επιλογή στο CIFAR-10.1. Στο αφελές Exp2, και τα δύο logical model instances ξεκινούν από τον ίδιο worker, με αποτέλεσμα ο worker1 να συγκεντρώνει application-level utilization 154.65% για το ResNet18EE και 160.83% για το ResNet34EE. Στο Exp3.3, τα αρχικά partitions των δύο instances ξεκινούν σε διαφορετικούς workers και τα επόμενα partitions διασταντώνονται σε διαφορετική σειρά. Έτσι, η αξιοποίηση μεταφέρεται σε δύο ενεργούς workers: για το ResNet18EE οι worker1 και worker2 βρίσκονται στο 67.07% και 73.34%, ενώ για το ResNet34EE στο 74.15% και 72.25%.

Η διαφοροποίηση αυτή συνδέεται με την κατανομή των exits στο CIFAR-10.1. Σε σχέση με το CIFAR-10, λιγότερα δείγματα τερματίζουν στο πρώτο exit και περισσότερα συνεχίζουν προς βαθύτερα partitions. Για το ResNet18EE, το ποσοστό εργασιών που φτάνει πέρα από το αρχικό partition αυξάνεται από 7.78% σε 13.10%, ενώ για το ResNet34EE από 4.59% σε 8.90%. Η ροή δειγμάτων είναι δηλαδή λίγο πιο απαιτητική για τα βαθύτερα partitions. Σε αυτή τη συνθήκη, το Exp3.2 δεν είναι ιδανικό, επειδή μεταφέρει υπερβολικά μεγάλο μέρος της ροής στον worker2: ο ίδιος worker φιλοξενεί συχνά ενεργά partitions και φτάνει σε 83.10% για το ResNet18EE και 91.99% για το ResNet34EE. Το bottleneck απλώς μετακινείται από τον worker1 στον worker2.

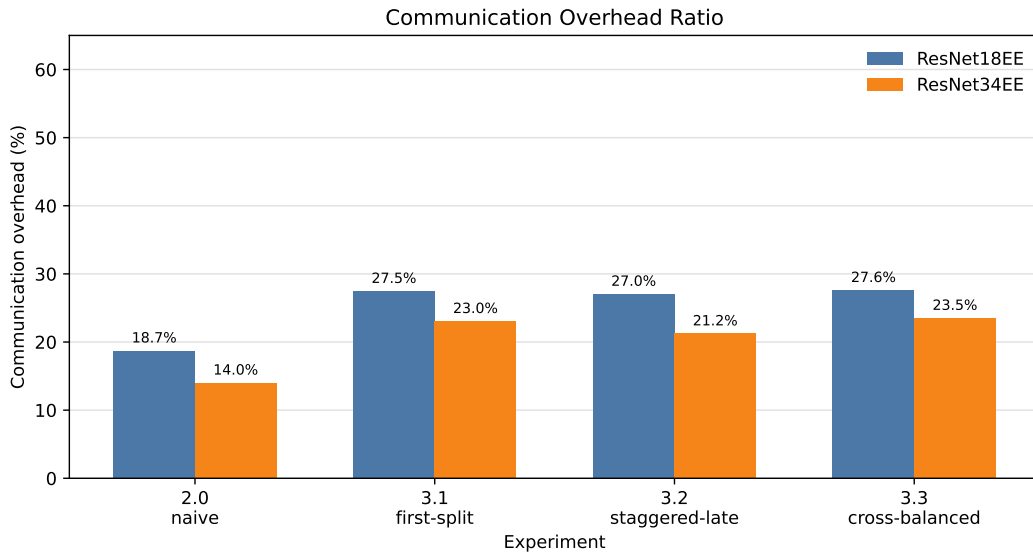
Το Exp3.3 αποφεύγει αυτή τη συγκέντρωση καλύτερα. Δεν είναι το ταχύτερο σε κάθε μεμονωμένη μέτρηση, αφού το Exp3.1 είναι οριακά καλύτερο στο ResNet18EE. Ωστόσο, το Exp3.3 δίνει το καλύτερο throughput στο βαθύτερο ResNet34EE και μένει πολύ κοντά στο Exp3.1 για το ResNet18EE. Αυτό το κάνει πιο σταθερή συνολική επιλογή στο CIFAR-10.1: αξιοποιεί δύο workers χωρίς να δημιουργεί τόσο έντονο νέο bottleneck και χειρίζεται καλύτερα την αυξημένη ροή προς τα βαθύτερα partitions. Το Exp3.1

ακολουθεί πρακτικά αμέσως μετά, επειδή επίσης διαχωρίζει τα αρχικά partitions των δύο instances, αλλά συγκεντρώνει περισσότερο τη βαθύτερη εκτέλεση στον spare worker.



Εικόνα 5.12: Αξιοποίηση workers σε όλα τα distributed πειράματα στο CIFAR-10.1.

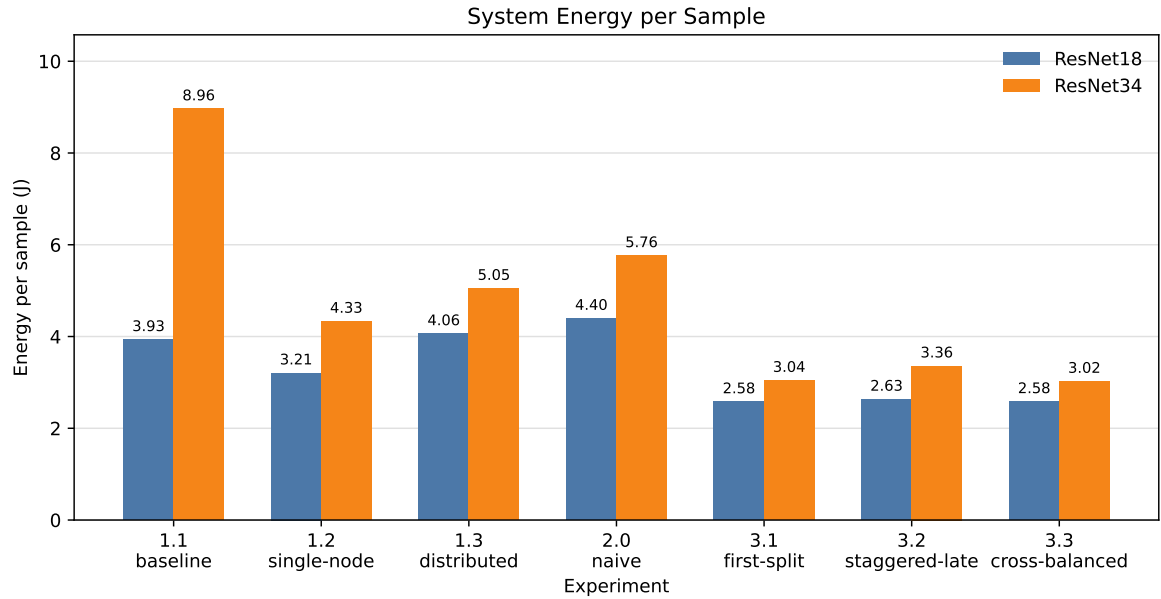
Η συνολική εικόνα της Εικόνας 5.12 επιβεβαιώνει την ίδια ερμηνεία για όλα τα distributed runs. Τα Exp3.1 και Exp3.3 είναι τα πιο ανταγωνιστικά στο CIFAR-10.1, ενώ το Exp3.2 εμφανίζει εντονότερη συγκέντρωση στον worker2. Το γενικό συμπέρασμα παραμένει ίδιο με το CIFAR-10: η ρητή τοποθέτηση των partitions είναι απαραίτητη για να αποφευχθεί η συμφόρηση του αφελούς multi-model pipeline, αλλά η καλύτερη συγκεκριμένη τοποθέτηση εξαρτάται από το exit distribution της ροής δειγμάτων.



Εικόνα 5.13: Ποσοστό communication/runtime overhead στα Experiments 2 και 3 για CIFAR-10.1.

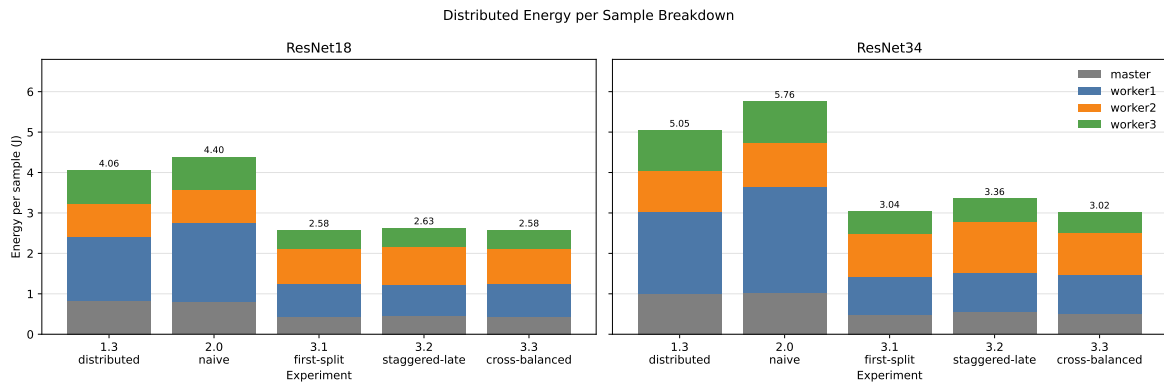
Η Εικόνα 5.13 χρειάζεται πιο προσεκτική ανάγνωση από την αντίστοιχη του CIFAR-10, επειδή ο απόλυτος χρόνος επικοινωνίας και το communication/runtime overhead ratio δεν κινούνται με τον ίδιο τρόπο. Στις memory-aware στρατηγικές, ο μέσος χρόνος επικοινωνίας μειώνεται: για το ResNet18EE από 0.1319 s στο Exp2 σε περίπου 0.104–0.107 s, και για το ResNet34EE από 0.1250 s σε περίπου 0.099–0.103 s. Ωστόσο, το συνολικό latency μειώνεται ακόμη περισσότερο, οπότε το communication/runtime overhead ως ποσοστό του latency παραμένει υψηλό και στα Experiment 3 runs εμφανίζεται στην περιοχή 21–28%. Συνεπώς, οι memory-aware τοποθετήσεις μειώνουν το απόλυτο κόστος

επικοινωνίας, αλλά στο μικρότερο σενάριο εκτέλεσης του CIFAR-10.1 η επικοινωνία εξακολουθεί να αποτελεί σημαντικό μέρος του end-to-end χρόνου.



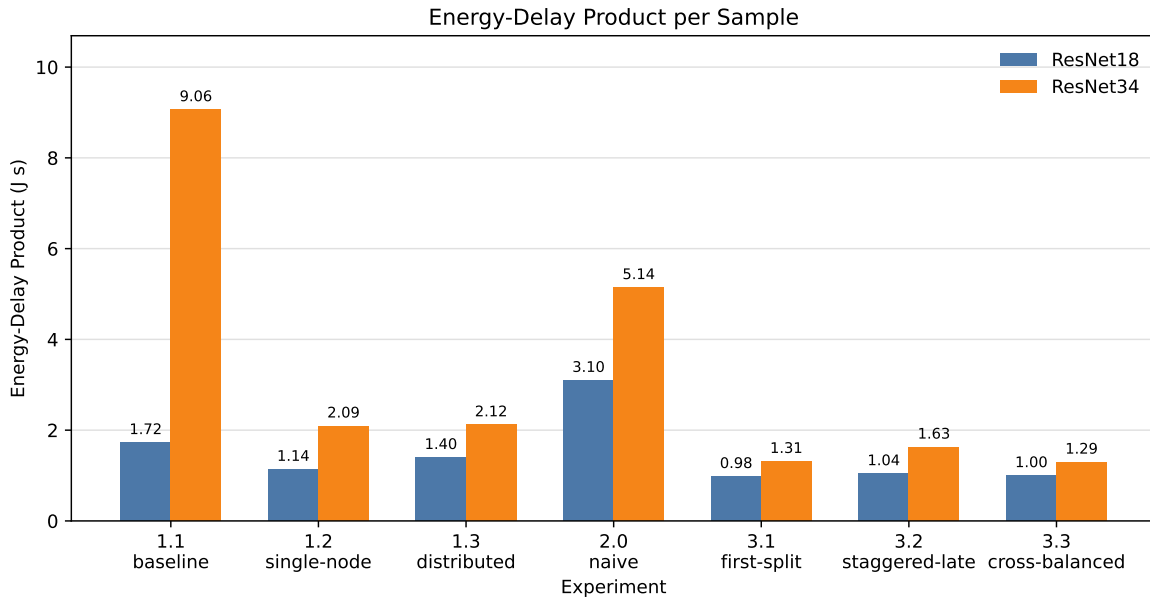
Εικόνα 5.14: Ενέργεια συστήματος ανά inference εργασία στο CIFAR-10.1.

Η Εικόνα 5.14 δείχνει ότι τα memory-aware multi-model runs μειώνουν και την ενέργεια ανά inference εργασία. Στο Exp2, η ενέργεια ανά inference εργασία είναι 4.40 J για το ResNet18EE και 5.76 J για το ResNet34EE. Στο Experiment 3, οι αντίστοιχες τιμές πέφτουν περίπου στα 2.58–2.63 J για το ResNet18EE και στα 3.02–3.36 J για το ResNet34EE. Άρα η βελτίωση του throughput δεν επιτυγχάνεται με αύξηση της ενεργειακής δαπάνης, αλλά συνοδεύεται από χαμηλότερο ενεργειακό κόστος ανά inference εργασία.



Εικόνα 5.15: Ανάλυση ενέργειας ανά inference εργασία σε master και workers για τα distributed πειράματα στο CIFAR-10.1.

Η Εικόνα 5.15 δείχνει ότι η ενέργεια ακολουθεί την ανακατανομή του φόρτου. Στο Exp2, ο worker1 έχει τη μεγαλύτερη συνεισφορά, καθώς δέχεται και τα δύο logical instances στα πρώτα partitions. Στο Experiment 3, η ενέργεια μεταφέρεται περισσότερο προς τους workers που φιλοξενούν τα συχνά ενεργά partitions. Παρόλα αυτά, το συνολικό ύψος των bars είναι μικρότερο από το Exp2, κάτι που επιβεβαιώνει ότι η καλύτερη τοποθέτηση μειώνει το συνολικό κόστος και όχι απλώς το μεταφέρει σε άλλον worker.



Εικόνα 5.16: Energy-Delay Product ανά inference εργασία στο CIFAR-10.1.

Η Εικόνα 5.16 συνοψίζει τη σχέση χρόνου και ενέργειας. Το Exp2 έχει EDP 3.10 J s για το ResNet18EE και 5.14 J s για το ResNet34EE. Στο Experiment 3, οι τιμές πέφτουν περίπου στα 0.98–1.04 J s για το ResNet18EE και στα 1.29–1.63 J s για το ResNet34EE. Συνεπώς, η συμπληρωματική αξιολόγηση στο CIFAR-10.1 επιβεβαιώνει το βασικό συμπέρασμα του Κεφαλαίου: οι memory-aware τοποθετήσεις βελτιώνουν ταυτόχρονα throughput, latency και ενεργειακή αποδοτικότητα, αν και η ακριβής καλύτερη στρατηγική μπορεί να μεταβληθεί ανάλογα με το dataset και τη ροή δειγμάτων.

Κεφάλαιο 6

Συζήτηση

Το προηγούμενο κεφάλαιο παρουσίασε τα πειραματικά αποτελέσματα για single-node, distributed και multi-model εκτέλεση early-exit ResNet μοντέλων. Στο παρόν κεφάλαιο τα αποτελέσματα αυτά ερμηνεύονται συνολικά. Η έμφαση μεταφέρεται από τις αριθμητικές τιμές κάθε πειράματος στα συμπεράσματα που προκύπτουν για τον σχεδιασμό καταναεμημένων edge inference συστημάτων.

6.1 Ερωτήματα Συζήτησης και Άξονες Ερμηνείας

Η συζήτηση που ακολουθεί δεν επαναλαμβάνει αναλυτικά τις αριθμητικές τιμές του προηγούμενου κεφαλαίου, αλλά τις οργανώνει γύρω από τρία κεντρικά ερωτήματα. Τα ερωτήματα αυτά λειτουργούν ως αφετηρία για την ερμηνεία των αποτελεσμάτων και οδηγούν στα βασικά συμπεράσματα της εργασίας.

1. Ποιο είναι το πραγματικό όφελος των early-exit μοντέλων όταν εκτελούνται ως ροές εργασιών inference σε περιορισμένο edge hardware, και πώς αυτό το όφελος σχετίζεται με τον συμβιβασμό ανάμεσα σε accuracy, latency και throughput;
2. Γιατί η καταναεμημένη εκτέλεση ενός early-exit μοντέλου, είτε ως single-model pipeline είτε ως αφελής multi-model επέκταση, δεν οδηγεί αυτόματα σε καλύτερη αξιοποίηση του cluster;
3. Σε ποιον βαθμό η memory-aware τοποθέτηση των partitions μπορεί να βελτιώσει τη συμπεριφορά του συστήματος χωρίς αλλαγή στο εκπαιδευμένο μοντέλο, στα weights ή στην πολιτική early-exit, και ποιες σχεδιαστικές αρχές προκύπτουν για edge inference συστήματα;

Με βάση αυτά τα ερωτήματα, το κεφάλαιο αναπτύσσεται σε τρεις ερμηνευτικούς άξονες. Ο πρώτος αφορά το early-exit ως μηχανισμό προσαρμοστικού υπολογισμού. Ο δεύτερος αφορά τη συστημική επίδραση της καταναεμημένης εκτέλεσης, του communication/runtime overhead και της ανισορροπίας φόρτου. Ο τρίτος αφορά τον ρόλο του placement ως μηχανισμού βελτίωσης της πολυ-μοντελικής εκτέλεσης και ως βάση για γενικότερες σχεδιαστικές αρχές.

6.2 Το Early-Exit ως Μηχανισμός Προσαρμοστικού Υπολογισμού

Το πρώτο ερώτημα του κεφαλαίου αφορά το πραγματικό όφελος των early-exit μοντέλων σε περιορισμένο edge hardware. Τα αποτελέσματα του Experiment 1 δείχνουν ότι το όφελος αυτό είναι ουσιαστικό σε single-node εκτέλεση: σε σχέση με τα baseline ResNet, οι early-exit εκδοχές μειώνουν τον μέσο χρόνο inference και αυξάνουν το throughput, επειδή δεν εκτελούν υποχρεωτικά ολόκληρο το δίκτυο για κάθε είσοδο.

Η αιτία είναι ότι το μοντέλο προσαρμόζει το βάθος εκτέλεσης στη δυσκολία κάθε δείγματος. Στο CIFAR-10, η πλειονότητα των δειγμάτων τερματίζει στα πρώτα exits, άρα η μέση εκτέλεση είναι πολύ πιο ρηχή από τη μέγιστη δυνατή. Με αυτόν τον τρόπο, όσο περισσότερα δείγματα τερματίζουν σε πρώιμα exits τόσο μειώνεται το latency, αλλά ταυτόχρονα μειώνεται και η ακρίβεια, οδηγώντας σε ένα accuracy-latency trade-off.

Η αξιολόγηση στο CIFAR-10.1 δείχνει ότι η κατανομή των εξόδων δεν καθορίζεται μόνο από την αρχιτεκτονική του μοντέλου, αλλά επηρεάζεται και από τη δυσκολία και την κατανομή των δεδομένων εισόδου. Το ίδιο εκπαιδευμένο μοντέλο, με την ίδια πολιτική εξόδου, οδηγεί σε διαφορετικό exit distribution όταν αλλάζει η κατανομή των inputs. Στο δυσκολότερο ανεξάρτητο test set, περισσότερα δείγματα συνεχίζουν σε βαθύτερα partitions, άρα μεταβάλλεται και το πραγματικό βάθος εκτέλεσης.

Το κρίσιμο σημείο για τη συνέχεια είναι ότι η ακρίβεια των early-exit μοντέλων παραμένει ίδια όταν μεταβαίνουμε σε distributed και multi-model εκτέλεση. Άρα, οι διαφορές ανάμεσα σε single-node, distributed και multi-model runs δεν οφείλονται σε διαφορετικές προβλέψεις του μοντέλου ή σε αλλαγή της πολιτικής early-exit, αλλά στον τρόπο με τον οποίο η ίδια δυναμική ροή εκτελείται πάνω στην υποδομή.

6.3 Κατανεμημένη Εκτέλεση και Ανισορροπία Φόρτου

Το δεύτερο ερώτημα του κεφαλαίου αφορά το γιατί η κατανεμημένη εκτέλεση ενός early-exit μοντέλου, είτε ως single-model είτε ως αφελής multi-model επέκταση, δεν οδηγεί αυτόματα σε καλύτερη αξιοποίηση του cluster. Τα αποτελέσματα δείχνουν ότι το πρόβλημα δεν βρίσκεται στη λειτουργικότητα του μοντέλου. Η ακρίβεια παραμένει σταθερή όταν αλλάζει μόνο η τοπολογία εκτέλεσης. Η διαφορά προκύπτει από τον τρόπο με τον οποίο η early-exit ροή αλληλεπιδρά με τα partitions, τους workers και το communication path.

Στο distributed single-model run, το ίδιο early-exit μοντέλο διασπάται σε partitions και εκτελείται σε πολλούς workers. Η διάσπαση αυτή προσθέτει επικοινωνία, serialization, forwarding και συντονισμό. Για να είναι ωφέλιμη, ο πρόσθετος υπολογισμός που μεταφέρεται στους workers που φιλοξενούν τα βαθύτερα partitions πρέπει να αντισταθμίζει αυτό το κόστος. Η σύγκριση CIFAR-10 και CIFAR-10.1 κάνει το σημείο πιο καθαρό για τη σταθερή single-model τοπολογία: όταν περισσότερα δείγματα συνεχίζουν σε βαθύτερα partitions, αυξάνεται η σημασία του communication path, επειδή περισσότερες εργασίες απαιτούν μετάδοση ενδιάμεσων ενεργοποιήσεων. Ταυτόχρονα, αυξάνεται και

η πραγματική εργασία που μπορεί να εκτελεστεί κατανεμημένα. Επομένως, το ερώτημα δεν είναι αν το distributed inference είναι γενικά καλύτερο ή χειρότερο, αλλά αν η συγκεκριμένη ροή δειγμάτων δημιουργεί αρκετό χρήσιμο υπολογισμό στα βαθύτερα partitions ώστε να δικαιολογεί το πρόσθετο κόστος εκτέλεσης.

Η βασική δυσκολία είναι ότι η early-exit ροή δεν τροφοδοτεί όλα τα partitions με τον ίδιο ρυθμό. Το πρώτο partition δέχεται κάθε δείγμα και περιλαμβάνει τα πρώτα σημεία λήψης απόφασης. Τα επόμενα partitions ενεργοποιούνται μόνο για τα δείγματα που δεν έχουν ήδη τερματίσει. Έτσι, ακόμη και όταν το μοντέλο έχει χωριστεί σε διαδοχικά partitions, η πραγματική ροή παραμένει άνιση. Η χαμηλή αξιοποίηση των workers που φιλοξενούν τα βαθύτερα partitions δεν είναι σφάλμα της υλοποίησης, αλλά άμεση συνέπεια του exit distribution.

Η ίδια ανισορροπία γίνεται εντονότερη στο αφελές multi-model σενάριο. Το Experiment 2 επαναχρησιμοποιεί την ίδια διαδρομή για δύο logical model instances. Από την πλευρά του μοντέλου, η εκτέλεση είναι ισοδύναμη με το distributed single-model run. Από την πλευρά του συστήματος, όμως, και οι δύο ροές στέλνουν όλες τις εργασίες inference στον ίδιο worker. Ο worker που φιλοξενεί το πρώτο partition δέχεται δύο φορές τη συχνότερη περιοχή της ροής δειγμάτων, ενώ οι workers που φιλοξενούν τα βαθύτερα partitions παραμένουν πολύ λιγότερο ενεργοί.

Το αποτέλεσμα είναι ότι η προσθήκη δεύτερου logical instance δεν οδηγεί σε ανάλογη αύξηση χωρητικότητας. Το συνολικό throughput αυξάνεται ελάχιστα, ενώ το throughput ανά instance μειώνεται και το communication/runtime overhead παραμένει σημαντικό μέρος του end-to-end χρόνου. Το ίδιο μοτίβο εμφανίζεται και στο CIFAR-10.1. Άρα, αποτελεί δομική συνέπεια της αναπαραγωγής της ίδιας front-heavy pipeline διαδρομής.

Συνεπώς, η κατανεμημένη early-exit εκτέλεση πρέπει να αντιμετωπίζεται ως πρόβλημα κατανομής πόρων. Ο αναμενόμενος φόρτος ενός worker δεν εξαρτάται μόνο από το κόστος του partition που φιλοξενεί, αλλά και από την πιθανότητα να φτάσει ένα δείγμα σε αυτό το partition. Αν η τοποθέτηση αγνοεί αυτή τη ροή, μπορεί να προσθέσει πρόσθετο κόστος χωρίς ουσιαστική παράλληλη αξιοποίηση. Αντίθετα, όταν η πιθανότητα ενεργοποίησης κάθε partition χρησιμοποιείται ως πληροφορία σχεδιασμού, το split computing μπορεί να γίνει μέρος μιας πιο αποδοτικής κατανομής πόρων.

6.4 Placement ως Πολυκριτήριο Πρόβλημα Βελτιστοποίησης

Το τρίτο ερώτημα αφορά τον βαθμό στον οποίο η τοποθέτηση των partitions μπορεί να βελτιώσει τη συμπεριφορά του συστήματος χωρίς αλλαγή στο εκπαιδευμένο μοντέλο, στα weights ή στην πολιτική early-exit. Το Experiment 3 απαντά σε αυτό το ερώτημα αλλάζοντας μόνο τη διαδρομή που ακολουθούν τα partitions πάνω στους workers. Η παρέμβαση είναι καθαρά συστημική, αλλά τα αποτελέσματα δείχνουν ότι μπορεί να αλλάξει ουσιαστικά το throughput, το latency, το communication/runtime overhead και την ενεργειακή αποδοτικότητα.

Το βασικό εύρημα είναι ότι όλες οι memory-aware στρατηγικές βελτιώνουν την αφελή πολυ-μοντελική εκτέλεση. Αυτό επιβεβαιώνεται και στο CIFAR-10 και στο CIFAR-10.1. Άρα, το placement δεν είναι δευτερεύουσα λεπτομέρεια υλοποίησης, αλλά βασική παράμετρος απόδοσης του συστήματος. Η ίδια early-exit πολιτική μπορεί να οδηγήσει σε πολύ διαφορετική συμπεριφορά, ανάλογα με το πού τοποθετούνται τα πιο ενεργά partitions.

Το πιο σημαντικό συμπέρασμα, όμως, είναι ότι το καλύτερο placement δεν είναι απόλυτο. Στο CIFAR-10, το Expr3.2 φαίνεται να είναι η πιο ισορροπημένη και αποτελεσματική επιλογή, κυρίως επειδή βελτιώνει έντονα το βαθύτερο ResNet34EE και μειώνει αποτελεσματικά το bottleneck του αφελούς baseline. Στο CIFAR-10.1, όμως, η κατάταξη αλλάζει: το Expr3.3 γίνεται η πιο αποδοτική επιλογή, ενώ το Expr3.1 ακολουθεί πολύ κοντά.

Η αιτία βρίσκεται στην κατανομή των exits. Στο CIFAR-10.1 λιγότερα δείγματα τερματίζουν στα πρώτα exits και περισσότερα συνεχίζουν προς βαθύτερα partitions. Αυτό αλλάζει το ποσοστό των εργασιών inference που φτάνει σε κάθε partition και, επομένως, αλλάζει τον αναμενόμενο φόρτο κάθε worker. Σε αυτή τη συνθήκη, το Expr3.2 μειώνει το αρχικό bottleneck του worker1, αλλά τείνει να μεταφέρει μεγάλο μέρος της πίεσης στον worker2. Το Expr3.3 κατανέμει πιο ισορροπημένα τη ροή μεταξύ των δύο βασικών ενεργών workers και γι' αυτό είναι πιο σταθερό στη δυσκολότερη ροή δειγμάτων.

Η παρατήρηση αυτή δείχνει ότι η απόδοση της τοποθέτησης δεν εξαρτάται μόνο από το υπολογιστικό κόστος των partitions, τη μνήμη των workers και το communication/runtime overhead. Επηρεάζεται επίσης από το data distribution και από τα thresholds που καθορίζουν πότε ένα δείγμα τερματίζει πρόωρα. Το διαφορετικό exit distribution στο CIFAR-10.1 επιβεβαιώνει ακριβώς αυτή την εξάρτηση.

Για αυτό, το placement πρέπει να αντιμετωπίζεται ως πολυκριτήριο πρόβλημα βελτιστοποίησης. Το utilization δείχνει αν η τοποθέτηση δημιουργεί bottleneck ή αφήνει διαθέσιμους πόρους ανενεργούς. Το communication/runtime overhead δείχνει πόσο από τον end-to-end χρόνο δαπανάται σε μη υπολογιστικό κόστος, τόσο σε απόλυτη διάρκεια όσο και ως ratio. Η ενέργεια ανά inference εργασία και το Energy-Delay Product δείχνουν αν η βελτίωση σε throughput συνοδεύεται από πραγματικά αποδοτικότερη λειτουργία. Καμία από αυτές τις μετρικές δεν αρκεί μόνη της.

Το αποτέλεσμα αυτό ενισχύει την αρχική υπόθεση της εργασίας. Εφόσον η βέλτιστη τοποθέτηση εξαρτάται από τη ροή που παράγει το ίδιο το early-exit μοντέλο για την εκάστοτε κατανομή εισόδων, χρειάζεται ένας αλγόριθμος τοποθέτησης των partitions στους workers. Ένας τέτοιος αλγόριθμος δεν θα πρέπει να βλέπει το δίκτυο μόνο ως ακολουθία layers, αλλά ως δυναμικό γράφο εκτέλεσης με πιθανότητες ενεργοποίησης, κόστη επικοινωνίας και περιορισμούς πόρων.

6.5 Σχεδιαστικές Αρχές

Τα αποτελέσματα οδηγούν σε μια σειρά από σχεδιαστικές αρχές για early-exit inference σε edge περιβάλλοντα. Οι αρχές αυτές δεν αποτελούν πλήρη αλγόριθμο placement, αλλά συνοψίζουν τι πρέπει να μετρά και να λαμβάνει υπόψη ένα τέτοιο σύστημα.

Πρώτον, η κατανομή των exits πρέπει να αντιμετωπίζεται ως runtime profile του μοντέλου. Σε ένα συμβατικό DNN, κάθε partition εκτελείται για όλα τα δείγματα. Σε ένα early-exit DNN, τα βαθύτερα partitions βλέπουν μόνο τα δείγματα που δεν τερμάτισαν νωρίτερα. Επομένως, το κόστος ενός partition πρέπει να σταθμίζεται με την πιθανότητα ενεργοποίησής του.

Δεύτερον, τα πρώτα partitions ενός early-exit μοντέλου έχουν δυσανάλογη σημασία. Δέχονται σχεδόν όλες τις εργασίες inference και περιλαμβάνουν τις πρώτες αποφάσεις εξόδου, ακόμη και αν δεν είναι τα βαρύτερα από άποψη layers ή παραμέτρων. Σε multi-model σενάρια, η τοποθέτηση πολλών αρχικών partitions στον ίδιο worker είναι πιθανό να δημιουργήσει bottleneck. Η διασπορά τους σε διαφορετικούς workers είναι η ελάχιστη συνθήκη για καλύτερη κλιμάκωση, αλλά δεν αρκεί από μόνη της.

Τρίτον, το placement πρέπει να είναι workload-aware και όχι μόνο memory-aware. Η μνήμη και η χωρητικότητα των workers καθορίζουν ποιες τοποθετήσεις είναι εφικτές, αλλά η κατανομή των exits καθορίζει ποιες είναι αποδοτικές. Η διαφοροποίηση ανάμεσα σε CIFAR-10 και CIFAR-10.1 δείχνει ότι η ίδια τοποθέτηση μπορεί να αλλάξει θέση στην κατάταξη όταν διαφορετικό ποσοστό δειγμάτων συνεχίζει προς τα βαθύτερα partitions. Για αυτό, ένας αλγόριθμος placement πρέπει να λαμβάνει υπόψη όχι μόνο τους περιορισμούς μνήμης και επικοινωνίας, αλλά και το data distribution και τα thresholds που διαμορφώνουν τη ροή των δειγμάτων.

Τέταρτον, το utilization πρέπει να ερμηνεύεται μαζί με τη ροή των εργασιών inference. Χαμηλή αξιοποίηση ενός worker δεν είναι απαραίτητα ανεπιθύμητη αν ο worker φιλοξενεί partition που ενεργοποιείται μόνο για δύσκολα δείγματα. Υψηλή αξιοποίηση δεν είναι απαραίτητα πρόβλημα αν οδηγεί σε υψηλό throughput χωρίς μεγάλο communication/runtime overhead. Το κρίσιμο ερώτημα είναι αν υπάρχει αχρείαστη συγκέντρωση φόρτου που αφήνει άλλους διαθέσιμους πόρους ουσιαστικά ανενεργούς.

Πέμπτον, το communication/runtime overhead πρέπει να αξιολογείται τόσο σε απόλυτο χρόνο όσο και ως ratio. Ο απόλυτος χρόνος δείχνει το πραγματικό κόστος των μεταβάσεων μεταξύ partitions. Το ratio δείχνει πόσο μεγάλο μέρος του end-to-end χρόνου δεν είναι καθαρός υπολογισμός. Χρειάζονται και οι δύο όψεις, επειδή μια βελτίωση στο συνολικό latency μπορεί να αλλάξει την ερμηνεία του ποσοστού communication/runtime overhead.

Έκτον, η ενεργειακή αποδοτικότητα πρέπει να κανονικοποιείται ανά inference εργασία. Τα multi-model runs εκτελούν περισσότερες εργασίες από τα single-model runs, οπότε η συνολική ενέργεια δεν αρκεί για δίκαιη σύγκριση. Η ενέργεια ανά inference εργασία και το Energy-Delay Product συνδέουν το ενεργειακό κόστος με την παραγωγικότητα και το latency του συστήματος.

Κεφάλαιο 7

Συμπεράσματα και Μελλοντική Εργασία

Το παρόν κεφάλαιο ολοκληρώνει τη διπλωματική εργασία, παρουσιάζοντας κατευθύνσεις για μελλοντική επέκταση και συνοψίζοντας την κεντρική θέση που ανέδειξε η πειραματική μελέτη: ότι τα early-exit μοντέλα σε καταναμημένο edge περιβάλλον πρέπει να αντιμετωπίζονται ως δυναμικοί φόρτοι εργασίας, των οποίων η αποδοτικότητα εξαρτάται από την πολιτική εξόδου, το partitioning, το placement και τους περιορισμούς της υποδομής.

7.1 Μελλοντική Εργασία

Μια πρώτη κατεύθυνση μελλοντικής εργασίας είναι η αξιολόγηση σε περισσότερα datasets και αρχιτεκτονικές. Το CIFAR-10 επέτρεψε ελεγχόμενη και επαναλήψιμη αξιολόγηση, αλλά ένα δυσκολότερο dataset, όπως το CIFAR-100, θα μπορούσε να αλλάξει σημαντικά την κατανομή των exits. Αν λιγότερα δείγματα τερματίζουν στα πρώτα exits, τότε η αξιοποίηση των workers που φιλοξενούν τα βαθύτερα partitions και το βέλτιστο placement θα μεταβληθούν. Αντίστοιχα, η μελέτη αρχιτεκτονικών όπως MobileNet, EfficientNet ή μεγαλύτερα ResNet θα έδινε πληρέστερη εικόνα για το πώς το βάθος και το μέγεθος των intermediate tensors επηρεάζουν το split inference.

Μια δεύτερη κατεύθυνση είναι η συστηματική μελέτη της πολιτικής early-exit. Στην παρούσα εργασία χρησιμοποιήθηκε σταθερό threshold $\tau = 0.9$, ώστε οι τοπολογίες να συγκρίνονται με την ίδια συμπεριφορά μοντέλου. Μελλοντική εργασία μπορεί να εξετάσει threshold sweeps, confidence calibration και δυναμικές πολιτικές εξόδου που λαμβάνουν υπόψη latency budgets, τρέχον load ή ενεργειακούς περιορισμούς. Έτσι θα μπορούσε να αποτυπωθεί ολόκληρη η καμπύλη accuracy-latency-energy για κάθε placement.

Πρόσφατη εργασία εξετάζει αυτή την κατεύθυνση μοντελοποιώντας την επιλογή του early-exit threshold ως πρόβλημα multi-armed bandit και συγκρίνοντας διαφορετικές παραλλαγές Upper Confidence Bound, όπως UCB-V, UCB-Tuned, UCB-Bayes και UCB-BwK. Τα αποτελέσματα δείχνουν ότι η επιλογή της πολιτικής εξερεύνησης-εκμετάλλευσης μπορεί να επηρεάσει τον συμβιβασμό accuracy-latency-energy, γεγονός που ενισχύει τη σημασία μελλοντικής μελέτης δυναμικών πολιτικών εξόδου [23].

Μια τρίτη κατεύθυνση είναι η ανάπτυξη δυναμικού placement μηχανισμού. Τα placements του Experiment 3 είναι στατικά και επιλεγμένα εκ των προτέρων. Σε πραγματικό σύστημα, όμως, ο φόρτος των workers, το bandwidth και οι ρυθμοί άφιξης αιτημάτων μπορούν να αλλάζουν στον χρόνο. Ένας online scheduler θα μπορούσε να μετακινεί ή να ενεργοποιεί partitions με βάση μετρήσεις utilization, queue length, communication/runtime overhead και exit ratios. Η κατεύθυνση αυτή συνδέεται με resource-aware προσεγγίσεις όπως τα graph-based deployment models της σχετικής βιβλιογραφίας.

Μια τέταρτη κατεύθυνση είναι η αξιολόγηση σε πιο ετερογενές hardware. Η παρούσα υποδομή αποτελείται από Raspberry Pi workers με παρόμοια χαρακτηριστικά. Σε πραγματικά edge συστήματα μπορεί να συνυπάρχουν CPU-only κόμβοι, GPU-enabled κόμβοι, gateways και ισχυρότεροι edge servers. Η ετερογένεια αυτή θα έκανε το placement πιο σύνθετο, επειδή το ίδιο partition μπορεί να έχει διαφορετικό χρόνο εκτέλεσης και ενεργειακό κόστος σε κάθε κόμβο.

Μια πέμπτη κατεύθυνση είναι η συστηματική μελέτη διαφορετικών partition boundaries. Στην παρούσα εργασία τα δύο πρώτα exits τοποθετούνται στο πρώτο partition, επιλογή που εξυπηρετεί πειραματικό στόχο. Ωστόσο, για πιο κατανομημένο φόρτο, το ίδιο το partitioning θα πρέπει να εξεταστεί ως μεταβλητή του προβλήματος. Διαφορετικά split points θα μπορούσαν να μεταφέρουν μέρος της συχνά ενεργής εκτέλεσης σε workers που φιλοξενούν βαθύτερα partitions, να αλλάζουν το μέγεθος των intermediate tensors και να επηρεάσουν ταυτόχρονα latency, communication/runtime overhead, utilization και ενέργεια. Επομένως, μελλοντική εργασία μπορεί να μελετήσει από κοινού την επιλογή των partition boundaries και την τοποθέτηση των partitions στους workers, με βάση την κατανομή των exits, το data distribution, τα thresholds, το compute cost κάθε partition και το κόστος επικοινωνίας.

Μια ακόμη σημαντική επέκταση είναι η ενσωμάτωση συστήματος monitoring μέσω Grafana. Στην παρούσα εργασία οι μετρικές συλλέγονται κυρίως ως αρχεία αποτελεσμάτων μετά την ολοκλήρωση κάθε run. Σε μια πιο πλήρη υποδομή, κάθε worker και ο master θα μπορούσαν να εξάγουν runtime μετρικές, όπως CPU utilization, μνήμη, network traffic, latency ανά partition, queue length, exit ratios και communication/runtime overhead, σε ένα σύστημα παρακολούθησης με dashboards. Η χρήση Grafana, ενδεχομένως σε συνδυασμό με Prometheus, θα επέτρεπε real-time παρατήρηση της συμπεριφοράς του distributed inference pipeline, ευκολότερο εντοπισμό bottlenecks και καλύτερη τεκμηρίωση των αποφάσεων placement. Επιπλέον, τέτοια παρατηρησιμότητα θα αποτελούσε απαραίτητη βάση για μελλοντικούς online schedulers που προσαρμόζουν δυναμικά την κατανομή των partitions.

Επιπλέον, η ενεργειακή αξιολόγηση μπορεί να βελτιωθεί με εξωτερική μέτρηση ισχύος. Οι εκτιμήσεις του CodeCarbon είναι χρήσιμες για συγκριτική ανάλυση, αλλά ένα power meter ανά κόμβο θα επέτρεπε ακριβέστερη αποτύπωση της κατανάλωσης, ειδικά σε περιπτώσεις όπου οι workers έχουν χαμηλό utilization αλλά παραμένουν ενεργοί.

Τέλος, μελλοντική εργασία μπορεί να εξετάσει τεχνικές μείωσης του επικοινωνιακού κόστους. Η συμπίεση intermediate tensors, η εισαγωγή bottleneck layers, η χρήση αποδοτικότερου serialization protocol ή η μετάβαση από HTTP σε πιο ελαφρύ RPC μηχανισμό θα μπορούσαν να μειώσουν το κόστος επικοινωνίας της distributed εκ-

τέλεσης. Οι τεχνικές αυτές είναι ιδιαίτερα σημαντικές όταν τα split points παράγουν ενεργοποιήσεις μεγαλύτερες από την αρχική είσοδο ή όταν το δίκτυο έχει περιορισμένο bandwidth.

7.2 Συνολικά Συμπεράσματα

Η βασική αξία της εργασίας είναι ότι αντιμετωπίζει τα early-exit μοντέλα όχι μόνο ως αρχιτεκτονικές νευρωνικών δικτύων, αλλά ως δυναμικούς φόρτους εργασίας που εκτελούνται πάνω σε συγκεκριμένη edge υποδομή. Η απόδοση ενός τέτοιου συστήματος δεν προκύπτει μόνο από το αν το μοντέλο έχει ενδιάμεσες εξόδους, αλλά από το πώς οι εξοδοί αυτές αλλάζουν τη ροή των δειγμάτων μέσα στο runtime.

Τα αποτελέσματα δείχνουν ότι το early-exit μπορεί να μειώσει σημαντικά τον μέσο υπολογισμό και την ενέργεια σε single-node εκτέλεση. Δείχνουν επίσης ότι η κατανεμημένη εκτέλεση χρειάζεται προσεκτικό partitioning, δηλαδή σχεδιασμό του τι περιλαμβάνει κάθε partition, επειδή διαφορετικά το communication/runtime overhead και η ανισορροπία φόρτου μπορούν να περιορίσουν το όφελος. Στα multi-model σενάρια, η τοποθέτηση των partitions γίνεται ακόμη πιο σημαντική, καθώς πολλαπλές ροές ανταγωνίζονται για τους ίδιους workers.

Συνολικά, η εργασία υποστηρίζει ότι ο σχεδιασμός αποδοτικών edge inference συστημάτων πρέπει να συνδυάζει τρία επίπεδα: την πολιτική early-exit του μοντέλου, τη διάσπαση του δικτύου σε partitions και την τοποθέτηση αυτών των partitions στους διαθέσιμους workers. Μόνο όταν τα τρία αυτά επίπεδα εξετάζονται μαζί μπορεί ένα early-exit DNN να αξιοποιηθεί αποτελεσματικά σε κατανεμημένο edge περιβάλλον.

Βιβλιογραφία

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-Based Learning Applied to Document Recognition”, *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: [10.1109/5.726791](https://doi.org/10.1109/5.726791).
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks”, in *Adv. Neural Inf. Process. Syst.*, vol. 25, 2012, pp. 1097–1105.
- [3] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition”, in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, arXiv:1409.1556.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition”, in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778, doi: [10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90).
- [5] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks”, in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, Proc. Mach. Learn. Res., vol. 97, 2019, pp. 6105–6114, arXiv:1905.11946.
- [6] S. Teerapittayanon, B. McDanel, and H. T. Kung, “BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks”, in *Proc. 23rd Int. Conf. Pattern Recognit. (ICPR)*, 2016, pp. 2464–2469, doi: [10.1109/ICPR.2016.7900006](https://doi.org/10.1109/ICPR.2016.7900006).
- [7] H. Rahmath P, V. Srivastava, K. Chaurasia, R. G. Pacheco, and R. S. Couto, “Early-Exit Deep Neural Network: A Comprehensive Survey”, *ACM Comput. Surv.*, vol. 57, no. 3, Art. no. 75, 2024, doi: [10.1145/3698767](https://doi.org/10.1145/3698767).
- [8] Y. Matsubara, M. Levorato, and F. Restuccia, “Split Computing and Early Exiting for Deep Learning Applications: Survey and Research Challenges”, *ACM Comput. Surv.*, vol. 55, no. 5, Art. no. 90, 2022, doi: [10.1145/3527155](https://doi.org/10.1145/3527155).
- [9] C. Singhal, Y. Wu, F. Malandrino, M. Levorato, and C. F. Chiasserini, “Resource-Aware Deployment of Dynamic DNNs over Multi-Tiered Interconnected Systems”, 2024, arXiv:2404.08060.
- [10] R. Mohanasundaram, A. S. Malhotra, R. Arun, and P. S. Periasamy, “Deep Learning and Semi-Supervised and Transfer Learning Algorithms for Medical Imaging”, in *Deep Learning and Parallel Computing Environment for Bioengineering Systems*, Elsevier, 2019, pp. 139–151, doi: [10.1016/B978-0-12-816718-2.00015-4](https://doi.org/10.1016/B978-0-12-816718-2.00015-4).
- [11] M. Gopinath and S. C. Sethuraman, “A Comprehensive Survey on Deep Learning-Based Malware Detection Techniques”, *Comput. Sci. Rev.*, vol. 47, Art. no. 100529, 2023, doi: [10.1016/j.cosrev.2022.100529](https://doi.org/10.1016/j.cosrev.2022.100529).
- [12] H. F. Nweke, Y. W. Teh, M. A. Al-garadi, and U. R. Alo, “Deep Learning Algorithms for Human Activity Recognition Using Mobile and Wearable Sensor Networks: State of the Art and Research Challenges”, *Expert Syst. Appl.*, vol. 105, pp. 233–261, 2018, doi: [10.1016/j.eswa.2018.03.056](https://doi.org/10.1016/j.eswa.2018.03.056).

- [13] Q. Zhang, L. T. Yang, Z. Chen, and P. Li, “A Survey on Deep Learning for Big Data”, *Inf. Fusion*, vol. 42, pp. 146–157, 2018, doi: [10.1016/j.inffus.2017.10.006](https://doi.org/10.1016/j.inffus.2017.10.006).
- [14] S. Zahia, M. B. Garcia Zapirain, X. Sevillano, A. González, P. J. Kim, and A. Elmaghraby, “Pressure Injury Image Analysis with Machine Learning Techniques: A Systematic Review on Previous and Possible Future Methods”, *Artif. Intell. Med.*, vol. 102, Art. no. 101742, 2020, doi: [10.1016/j.artmed.2019.101742](https://doi.org/10.1016/j.artmed.2019.101742).
- [15] M. Mohseni-Dargah, Z. Falahati, B. Dabirmanesh, P. Nasrollahi, and K. Khajeh, “Machine Learning in Surface Plasmon Resonance for Environmental Monitoring”, in *Artificial Intelligence and Data Science in Environmental Sensing*, Elsevier, 2022, pp. 269–298, doi: [10.1016/B978-0-323-90508-4.00012-5](https://doi.org/10.1016/B978-0-323-90508-4.00012-5).
- [16] H. Rahmath P, K. Chaurasia, and A. Bansal, “Strategies at a Glance: A Comparative Analysis of Training Techniques for Optimizing Early-Exit Deep Neural Networks”, *Neural Netw.*, vol. 192, Art. no. 107970, 2025, doi: [10.1016/j.neunet.2025.107970](https://doi.org/10.1016/j.neunet.2025.107970).
- [17] A. Alotaibi and A. Barnawi, “Securing Massive IoT in 6G: Recent Solutions, Architectures, Future Directions”, *Internet Things*, vol. 22, Art. no. 100715, 2023, doi: [10.1016/j.iot.2023.100715](https://doi.org/10.1016/j.iot.2023.100715).
- [18] T. Noguchi and T. Azumi, “3D Point Cloud Object Detection on Edge Devices for Split Computing”, in *Proc. 3rd Real-time And intelliGent Edge Computing Workshop (RAGE)*, 2024, pp. 1–6, doi: [10.1109/RAGE62451.2024.00009](https://doi.org/10.1109/RAGE62451.2024.00009).
- [19] A. Bakhtiarnia, N. Milosevic, Q. Zhang, D. Bajovic, and A. Iosifidis, “Dynamic Split Computing for Efficient Deep Edge Intelligence”, 2022, arXiv:2205.11269.
- [20] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images”, Univ. Toronto, Toronto, ON, Canada, 2009.
- [21] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, “Do CIFAR-10 Classifiers Generalize to CIFAR-10?”, 2018, arXiv:1806.00451.
- [22] modestyachts. “CIFAR-10.1”. GitHub. Accessed: Jun. 13, 2026. [Online]. Available: <https://github.com/modestyachts/CIFAR-10.1>
- [23] G. Papanikolaou, I. Kontopoulos, and K. Tserpes, “A Comparative Analysis on the Performance of Upper Confidence Bound Algorithms in Adaptive Deep Neural Networks”, in *Proc. IEEE Int. Conf. Smart Computing (SMARTCOMP)*, 2026, arXiv:2604.24810.

Δήλωση χρήσης εργαλείων Τεχνητής Νοημοσύνης

Κατά την εκπόνηση της παρούσας διπλωματικής εργασίας έγινε χρήση εργαλείων Τεχνητής Νοημοσύνης βασισμένων σε μεγάλα γλωσσικά μοντέλα, και συγκεκριμένα του Chat-GPT/Codex, ως υποστηρικτικών εργαλείων και όχι ως πηγών του περιεχομένου της εργασίας. Τα εργαλεία αυτά αξιοποιήθηκαν για τη γλωσσική επιμέλεια και βελτίωση της σαφήνειας του κειμένου, για την αποσαφήνιση και καλύτερη κατανόηση επιμέρους τεχνικών εννοιών, για την υποστήριξη στη διατύπωση και οργάνωση τμημάτων της εργασίας, καθώς και για τη μορφοποίηση και τεχνική επεξεργασία του κειμένου σε L^AT_EX.

Σε κάθε περίπτωση, το περιεχόμενο που προέκυψε με τη βοήθεια των εργαλείων αυτών ελέγχθηκε, αξιολογήθηκε και διαμορφώθηκε από τον συγγραφέα. Η επιλογή της βιβλιογραφίας, η μεθοδολογία, ο σχεδιασμός και η υλοποίηση των πειραμάτων, η ανάλυση των αποτελεσμάτων και τα συμπεράσματα της εργασίας αποτελούν προϊόν προσωπικής εργασίας του συγγραφέα, ο οποίος φέρει την πλήρη και αποκλειστική ευθύνη για το τελικό περιεχόμενο.